

DAKE: Bandwidth-Efficient (U)AKE from Double-KEM

Hugo Beguin¹, Céline Chevalier^{2,3}, Guirec Lebrun^{2,5}, Thomas Legavre^{1,4,5},
Thomas Ricosset¹, Maxime Roméas⁵, and Éric Sageloli^{1,2}

¹ Thales, Gennevilliers, France

² DIENS, École normale supérieure, CNRS, PSL University, Inria, Paris, France

³ CRED, Paris-Panthéon-Assas University

⁴ Sorbonne Université, CNRS, LIP6, Paris, France

⁵ ANSSI, Paris, France

✉ thomas.ricosset@thalesgroup.com

✉ eric.sageloli@protonmail.com

Abstract. Bandwidth remains a major bottleneck in post-quantum cryptography, particularly for authenticated key exchange (AKE) protocols. In this work, we present **DAKE**, a bandwidth-efficient AKE framework built from double-KEM constructions. **DAKE** comes in two main versions achieving, respectively, weak and full perfect forward secrecy, as well as explicit authentication. It further admits two variants: a unilateral version, and another where a signature scheme replaces a KEM. They are proven secure in the standard model under eCK^w and eCK-PFS , two strong variants of the extended Canetti–Krawczyk framework.

DAKE employs a double-KEM, a primitive that encapsulates a single key under two public keys simultaneously. Such constructions can achieve smaller encapsulation sizes than two independent KEM encapsulations, offering a significant bandwidth advantage.

To facilitate the design of double-KEMs compatible with **DAKE**, we introduce a chosen-key Fujisaki–Okamoto (CK-FO) transform proven in the QROM, which upgrades IND-CPA double-PKEs to IND-CCA double-KEMs while ensuring the one-sided chosen-key security required by **DAKE**. As a concrete instantiation, we propose **Maul**, a compact double-KEM derived from ML-KEM under the Hint-MLWE assumption. **Maul** reuses ciphertext components to cut encapsulation size by up to 42% compared to two parallel ML-KEMs. When instantiated with **Maul**, **DAKE** achieves overall communication reductions of about 16% (mutual authentication) and 21% (unilateral), outperforming both the double-KEM AKE of Xue *et al.* (ASIACRYPT 2018) and standard ML-KEM-based AKEs.

Keywords: Post-quantum (U)AKE · Double-KEM · eCK · ML-KEM

1 Introduction

In order to face the looming threat of large-scale quantum computers against public-key cryptography, the U.S. National Institute of Standards and Technology (NIST) launched in 2016 a standardization process to select post-quantum

encryption and signature primitives resistant to quantum attacks. In particular, key encapsulation mechanisms (KEMs) were chosen to ensure the encryption functionality. KEMs serve as key exchange primitives: a sender uses an encapsulation algorithm to produce a shared secret and a ciphertext, which is then sent to the recipient. The recipient runs a decapsulation algorithm to recover the same shared secret, enabling secure communication. In mid-2022, NIST announced the first post-quantum standards: Kyber/ML-KEM [54] for key encapsulation, and Dilithium/ML-DSA [55], Falcon/FN-DSA [27], and SPHINCS⁺/SLH-DSA [56] for digital signatures. More recently, in 2025, NIST also selected HQC [30]—a code-based KEM—as an additional standardized alternative to the lattice-based ML-KEM, in order to diversify the set of hardness assumptions.

Quantum Security Comes at a Cost. While post-quantum (PQ) primitives offer conjectured security against quantum-capable adversaries, they often impose large communication costs. For instance, ML-KEM-512 (128-bit security) has 800-byte public keys and 768-byte ciphertexts, compared to the 32-byte keys and ciphertexts of classical Curve25519 DH-KEM [5]. PQ signatures are even larger, often several kilobytes, which makes signature-based authenticated key exchange (AKE) particularly expensive.

Signature-Free AKEs. To mitigate this overhead, signature-free AKEs rely on KEMs or public-key encryptions (PKEs) for both key exchange and authentication. Early constructions stem from [11] and were later strengthened to the CK+ model in [28].⁶ More recent works [33, 43] improved computational efficiency by employing PKEs instead of KEMs, applying the Fujisaki–Okamoto transform at the protocol level. Yet, communication costs remain essentially unchanged. These designs have inspired practical adaptations, including post-quantum WireGuard [34] and KEMTLS [51], now under IETF standardization [57]. Related work [12] explores modular AKEs with various authentication mechanisms, including KEM-based ones, and proves their security in the CK+ model.

From Two KEMs to a Double-KEM. Even in the unilaterally authenticated setting, achieving (weak) perfect forward secrecy requires two KEMs—one for key agreement and another for authentication. Greater efficiency can be achieved through a dedicated primitive, the *double-KEM* [59], which replaces two independent KEMs with a single mechanism encapsulating under two public keys simultaneously. This design offers better compactness than parallel composition and underlies the double-KEM-based AKE of [59], proven secure in the random oracle model (ROM) and in the standard model under the [IND-CK-CCA, IND-CK-CPA] assumption. The same work introduces *Twin-Kyber*, a size-optimized double-KEM derived from ML-KEM/Kyber [10, 54], proven [IND-CK-CCA, IND-CK-CCA]-secure in the ROM. Notably, *Twin-Kyber* is obtained by applying a Fujisaki–

⁶ [11] proved security in the CK model [13], while [28] extended this to CK+ [39], capturing weak forward secrecy (wFS) and maximal exposure attacks (MEX).

Okamoto transformation to a $[\text{IND-CK-CPA}, \text{IND-CK-CPA}]$ -secure double-key double-message PKE.

Security notions for double-KEMs extend those of traditional KEMs. The standard notions of IND-CPA and IND-CCA are generalized to one-sided variants such as $[\text{IND-CPA}, -]$ and $[\text{IND-CCA}, -]$, where the adversary is given the right secret key sk_R during the security game. In this setting, the encapsulated key k must remain secret even if sk_R is known. Stronger notions, denoted $[\text{IND-CK-CPA}, -]$ and $[\text{IND-CK-CCA}, -]$, additionally allow the adversary to specify the right public key pk_R used during encapsulation.⁷ Analogous notions are defined for the right side, denoted $[-, \text{IND-CK-CPA}]$ and $[-, \text{IND-CK-CCA}]$, where the adversary chooses the left public key pk_L . These can also be extended to the double-key, double-message public-key encryption (2K2M-PKE) setting.

1.1 Our Contributions

We introduce DAKE, DAKE^σ , and DUAKE, three bandwidth-efficient AKE protocols built from $[\text{IND-CK-CCA}, \text{IND-CCA}]$ double-KEMs. The underlying double-KEM can be instantiated either using two standard IND-CCA KEMs or more compact double-KEM constructions. Each protocol comes in a two-message version achieving weak perfect forward secrecy (wPFS) and a three-message version achieving full PFS, both proven secure in the standard model under the strong eCK^w and eCK-PFS variants [18] of the eCK model [41] respectively. These variants capture attack scenarios that are not covered by other generic post-quantum (U)AKes analyzed in the literature [10, 29, 33, 51].

DAKE provides mutual authentication by combining a double-KEM with a standard KEM, where the double-KEM ensures initiator authentication, while the standard KEM ensures responder authentication. Responder authentication can alternatively be achieved using a signature scheme, yielding the variant DAKE^σ . We also propose DUAKE, a unilaterally authenticated variant in which only the initiator is authenticated.

Beyond key secrecy, we analyze additional properties, including *inverse match soundness*—which prevents *unknown key-share* (UKS) attacks [9]—and *explicit authentication*—which ensures that any completed session has a unique honest partner. Among the generic post-quantum (U)AKes in the literature, the only one we are aware of that analyzes explicit authentication is [51].

To ease the construction of double-KEMs compatible with DAKE and its variants, we introduce a chosen-key Fujisaki–Okamoto (CK-FO) transform tailored to 2K2M-PKEs. Our transform is proven secure in the quantum random oracle model (QROM), achieving CCA security while additionally providing chosen-key security for the left encapsulation, thereby yielding a $[\text{IND-CK-CCA}, \text{IND-CCA}]$ double-KEM, the central ingredient of DAKE. To the best of our knowledge, this is the first extension of the FO paradigm to this setting, enabling constructions of double-KEMs from a wide class of schemes that lack chosen-key security.

⁷ These stronger notions are sometimes directly referred to as $[\text{IND-CPA}, -]$ and $[\text{IND-CCA}, -]$ in the literature, e.g., in [59].

As a concrete instantiation, we propose **Maul**, a compact lattice-based double-KEM derived from Kyber/ML-KEM. **Maul** is obtained by applying our CK-FO transform to Maul_{PKE} , a double-key double-message PKE built by factoring the large polynomial vector $\mathbf{u}$ from Kyber ciphertexts, reducing ciphertext size by up to 42% compared with two parallel ML-KEM instances. We prove Maul_{PKE} [IND-CPA, IND-CPA]-secure under MLWE and hint-MLWE [38], and its CK-FO transformation yields **Maul**, a compact [IND-CK-CCA, IND-CCA] double-KEM. Instantiating **DAKE** with **Maul** achieves roughly 20% global communication savings over the double-KEM-based AKE of Xue et al. [59] and over classical ML-KEM-based AKEs, highlighting the practical advantages of tightly integrated double-KEM designs.

1.2 Technical Overview

Our authenticated key exchange protocol, denoted **DAKE** (for Double-KEM-based AKE), together with its variants DAKE^σ and **DUAKE**, forms a versatile family of AKE constructions built upon the notion of a double-KEM.

A compact double-KEM enables, at essentially the cost of a single standard KEM encapsulation, the derivation of a shared key that depends on two distinct public keys—an operation that would otherwise require two separate encapsulations. We provide such a construction with **Maul**, our double-KEM variant of ML-KEM constructed via our CK-FO transform. **DAKE** and its variants exploit this property to achieve a substantial reduction in bandwidth compared to conventional (U)AKEs.

AKE from Double-KEM. The main protocol, **DAKE**, provides mutual authentication by combining a double-KEM with a standard KEM, as specified in Figure 1, where the KEM ensures authentication of the responder. **DAKE** has two variants: DAKE_2 , a two-message protocol achieving weak Perfect Forward Secrecy (wPFS), and DAKE_3 , a three-message protocol achieving full PFS. The PFS ensures that a session key remains secure even if the long-term secrets used in its derivation are compromised after the session completes, the wPFS is discussed below.

All our protocols achieve key secrecy in the standard model: DAKE_2 is proven secure in the eCK^w model, and DAKE_3 in the eCK-PFS model. These state-of-the-art security models refine the classical eCK model by more precisely capturing the effects of both passive and active adversarial behavior. In most other models, weak PFS requires that a session key remains secure even if the long-term secrets used in its derivation are compromised, under the additional assumptions that the adversary neither (1) modifies messages sent by the session nor (2) forges or alters received messages. In contrast, eCK^w and eCK-PFS ensure key secrecy even if the adversary tampers with messages sent by the session, as long as the received messages remain authentic (*i.e.*, neither forged nor modified). These refinements are enabled by the introduction of the *origin session* notion.

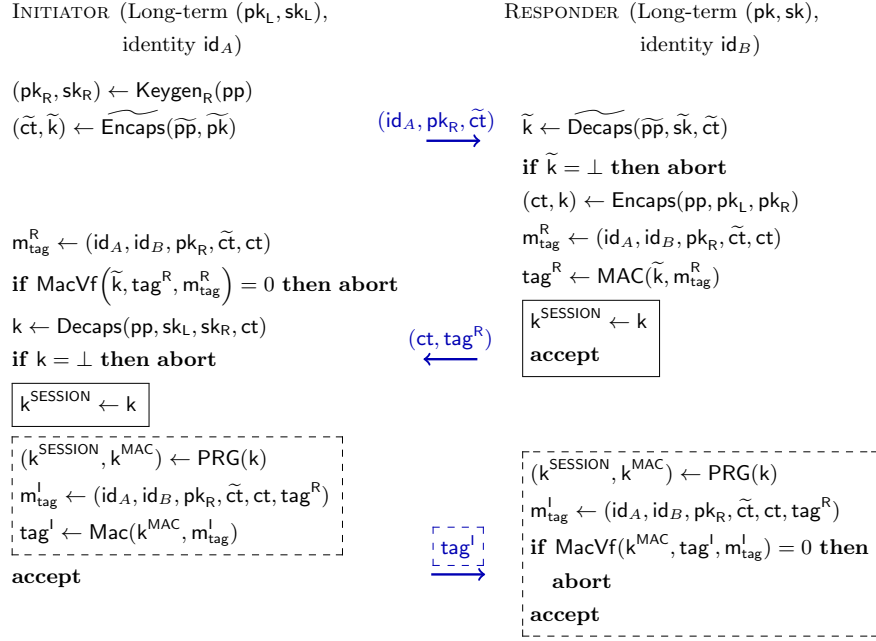


Fig. 1: The DAKE protocol. Public parameters are (1) $(pp, \widetilde{pp})$ with $pp \leftarrow \text{Setup}()$ and $\widetilde{pp} \leftarrow \widetilde{\text{Setup}}()$, (2) all party identities, and (3) a dictionary mapping identities to public keys. The initiator chooses its peer before sending the first message. DAKE₂ (*resp.* DAKE₃) is obtained by removing the dotted (*resp.* solid) boxes.

Responder authentication can be achieved either via an additional standard KEM—as in the baseline DAKE, which typically minimizes overall bandwidth—or via a signature scheme, yielding the variant DAKE ^{σ} (see Appendix D.1). The latter is especially relevant when the initiator’s public key is unknown to the responder and therefore the initiator must attach a certificate to its first message. Although replacing a KEM ciphertext with a signature usually increases total communication, it reduces the initiator’s first-message size, balancing communication costs when certificates or other extra data must be sent upfront.

Unilaterally authenticated AKEs (UAKes), in which only one party is authenticated, have been studied in several contexts such as Tor and TLS [21, 26, 31, 36, 40, 51, 53]. DUAKE, specified in Figure 2, is our unilateral variant of DAKE, tailored for such settings. To analyze its security, we introduce the u-eCK^w and u-eCK-PFS models—unilateral adaptations of the eCK^w and eCK-PFS frameworks. Essentially, u-eCK^w and u-eCK-PFS remove from eCK^w and eCK-PFS the attack scenarios that cannot be prevented by a unilateral protocol (for instance, attacks where an active adversary impersonates a party that possesses no long-term keys). While several security models for UAKE have been pro-

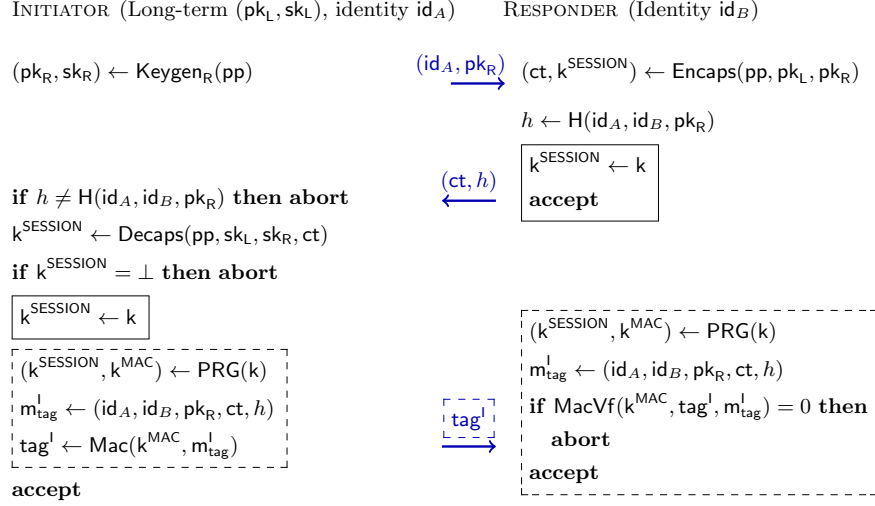


Fig. 2: The DUAKE protocol. Public parameters are (1) $pp \leftarrow \text{Setup}()$, (2) all party identities, and (3) a dictionary mapping initiators to their public keys. The initiator chooses its peer before sending the first message. DUAKE_2 (*resp.* DUAKE_3) is obtained by removing the dotted (*resp.* solid) boxes.

posed (*e.g.*, [31] presents a unilateral extension of the $\mathbf{eCK}$ model), our models strengthen these definitions by incorporating the refinements introduced in $\mathbf{eCK}^w$ and $\mathbf{eCK}\text{-PFS}$, thereby capturing a broader range of attack scenarios. Practical use cases of DUAKE are discussed in more detail in Appendix I.

The key secrecy guarantees provided by the security models we rely on offer resistance to *maximal exposure* (MEX) and *key-compromise impersonation* (KCI) attacks. They also imply a form of *implicit authentication* by ensuring that the session key remains secret under active attacks whenever the peer of the session is not corrupted (in which case the peer is said to be implicitly authenticated)⁸. More precisely, both the initiator and responder are implicitly authenticated in $\text{DAKE}_2, \text{DAKE}_3, \text{DAKE}_2^\sigma$ and DAKE_3^σ , whereas only the initiator benefits from implicit authentication in $\text{DUAKE}_2, \text{DUAKE}_3$.

To capture security across a broad range of settings, we further study several additional properties for all our AKE protocols, Table 1 summarizing which ones are achieved by the different variants of DAKE. Specifically, we consider *match soundness*⁹, inverse match soundness (which prevents UKS attacks), and *explicit authentication*. We refine the latter by distinguishing between *Init-explicit*

⁸ Some additional conditions that would trivially reveal the session key to the adversary are omitted here for simplicity.

⁹ As informally explained in [18], it requires that two parties who complete matching sessions (i. e., the intended communication partners) compute the same session key.

Table 1: Additional security properties of our AKE protocols. An asterisk (✓*) indicates that the corresponding property can be achieved through a minor modification that does not affect the exchanged messages. See Section 3.3 for details on how inverse-match soundness can be obtained using an additional KDF.

Protocol	PFS	wPFS	(Match Snd., Inv. Match)	Init-Explicit Authent.	Resp-Explicit Authent.
DAKE ₂	✗	✓	(✓, ✓*)	✗	✓
DAKE ₃	✓	✓	(✓, ✓*)	✓	✓
DAKE ₂ ^σ	✗	✓	(✓, ✓*)	✗	✓
DAKE ₃ ^σ	✓	✓	(✓, ✓*)	✓	✓
DUAKE ₂	✗	✓	(✓, ✓*)	✗	✗
DUAKE ₃	✓	✓	(✓, ✓*)	✓	✗

and *Resp-explicit* authentication. In short, *Init-explicit* (resp. *Resp-explicit*) authentication ensures that if the peer of a responder (resp. initiator) session is not corrupted (*i.e.*, its long-term secret key is unknown to the adversary) and this session has completed the protocol, then there exists an initiator (resp. responder) session that has honestly executed the protocol with it. We show that *Init-explicit* authentication is provided by DAKE₃, DAKE₃^σ and DUAKE₃, while *Resp-explicit* authentication is ensured by DAKE₂, DAKE₃, DAKE₂^σ and DAKE₃^σ.

Chosen-key FO Transform. Our Chosen-Key Fujisaki–Okamoto (CK-FO) transform enables the instantiation of DAKE and its variants from a broad class of underlying primitives by providing a generic method to transform a wide class of [IND-CPA, IND-CPA] double-key double-message PKE (2K2M-PKE) schemes into [IND-CK-CCA, IND-CCA] double-KEMs. The resulting construction guarantees chosen-key CCA security on one side and standard CCA security on the other—precisely the security properties required for instantiating DAKE and its variants. Although our concrete instantiation is lattice-based, the technique also paves the way for compact double-KEMs under other assumptions, such as code-based constructions inspired by HQC [30].

A central idea of the CK-FO transform is to restrict attention to a subclass of 2K2M-PKE constructions derived from two underlying PKEs, and to leverage not only the security of the combined scheme but also that of its constituents. The transform admits proofs of security in both the random oracle model (ROM) and the quantum random oracle model (QROM), the latter via semi-classical oracle techniques. Beyond its conceptual novelty, the CK-FO transform significantly broadens the design space: it enables double-KEMs to be instantiated from compact [IND-CPA, IND-CPA] 2K2M-PKEs that do not possess any chosen-key security.

Lattice-based Compact Double-KEM. While DAKE, DAKE^σ , and DUAKE are generic constructions, their practical efficiency depends on the availability of compact double-KEMs. Prior work such as Twin-Kyber [59] demonstrated feasibility but left ample room for optimization. To validate our framework, we introduce **Maul**, a $[\text{IND-CK-CCA}, \text{IND-CCA}]$ double-KEM derived from ML-KEM/Kyber, and use it to instantiate DAKE. By exploiting structural asymmetries in Kyber ciphertexts, **Maul** reduces ciphertext size by up to 42% compared to two parallel ML-KEMs, thereby minimizing message sizes in complete AKE handshakes. Table 2 compares **Maul**’s ciphertext and public-key sizes with those of two parallel ML-KEMs and Twin-Kyber [59].

Table 2: Sizes (bytes) of the ciphers and public keys of our double-KEM **Maul**, compared to two parallel ML-KEM instances and to Twin-Kyber. Security levels follow the categories defined in the NIST PQC call for proposals¹⁰.

	Security Level	Maul (this work)	Twin-Kyber [59]	ML-KEM [54]
Ciphertext	1	896	–	1,536
	3	1,440	–	2,176
	5	1,856	2,464	3,136
Public Key	1	826	–	800
	3	1,240	–	1,184
	5	1,691	1,568	1,568

Maul results from applying the CK-FO transform to Maul_{PKE} , a 2K2M-PKE that we now describe. At a high level, ML-KEM (Kyber) ciphertexts consist of two parts: a vector component $\mathbf{u}$ and a scalar component v , where $\mathbf{u}$ encodes ephemeral randomness and v encodes the message. These components are highly asymmetric in size; for instance, in ML-KEM-768, $\mathbf{u}$ occupies 950 bytes, while v is only 128 bytes. Maul_{PKE} exploits this imbalance by factoring the larger vector component $\mathbf{u}$, yielding ciphertexts of the form $(\mathbf{u}, v_1, v_2)$ that encrypt two messages while significantly reducing overall ciphertext size. A related construction was proposed in [37], though its security analysis was limited to a setting where the adversary is not given access to any secret key.

The security of Maul_{PKE} is based on the Hint-MLWE assumption, a natural variant of MLWE in which the adversary receives a partial linear hint about the secret and error. To illustrate this, consider a simplified version of the $[\text{IND-CPA}, \text{IND-CPA}]$ Maul_{PKE} encrypting two messages m_L and m_R as follows:

$$(\mathbf{u} = \mathbf{A}^T \mathbf{s} + \mathbf{e}, v_1 = \mathbf{t}_L^T \mathbf{s} + e_1 + \lceil q/2 \rceil m_L, v_2 = \mathbf{t}_R^T \mathbf{s} + e_2 + \lceil q/2 \rceil m_R),$$

¹⁰ <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/call-for-proposals>

where $\mathbf{t}_L = \mathbf{A}\mathbf{s}_L + \mathbf{e}_L$ and $\mathbf{t}_R = \mathbf{A}\mathbf{s}_R + \mathbf{e}_R$ are public keys derived from secret keys $\mathbf{s}_L, \mathbf{s}_R$ and errors $\mathbf{e}_L, \mathbf{e}_R$. When the adversary knows the right-hand secret key $(\mathbf{s}_R, \mathbf{e}_R)$, the component $(\mathbf{u}, v_1)$ becomes a MLWE instance with an additional known noisy linear relation: $v_2 - \mathbf{s}_R^T \mathbf{u} - \lceil q/2 \rceil m_R = (\mathbf{e}_R^T, -\mathbf{s}_R^T)(\begin{smallmatrix} s \\ e \end{smallmatrix}) + e_2$.

This precisely matches the definition of Hint-MLWE, and under this assumption, the left ciphertext $(\mathbf{u}, v_1)$ remains indistinguishable from uniform, thus ensuring the security of m_L .

We instantiate **Maul** with concrete parameters and estimate its security by adapting the heuristic framework of [22], which models non-Gaussian error distributions as Gaussians and disregards certain discrete-Gaussian-specific factors in the reduction. To support robust parameter selection, we complement this analysis with an evaluation of the best known attacks against Hint-MLWE.

The techniques underlying **Maul** can also be directly adapted to integer lattices, yielding a FrodoKEM-inspired [45] double-KEM with analogous security guarantees under the LWE and Hint-LWE assumptions; we leave a detailed treatment of this variant to future work. We further believe that these techniques could extend to a broader class of ElGamal-style KEMs, such as HQC [30], which has been selected for NIST standardization.

Performance of DAKE Instantiations. Beyond the security properties they guarantee, the strength of DAKE, DAKE^σ , and DUAKE lies in their ability to transform compact double-KEMs into bandwidth-efficient (U)AKE protocols. Instantiated with **Maul**, they demonstrate this potential concretely. Table 3 reports the message sizes for DAKE, DAKE^σ , and DUAKE instantiated with **Maul**, under the assumption that MAC tags, hash outputs, and identifiers have a bit-length twice the security parameter.

Table 3: Message sizes (bytes) for DAKE, DAKE^σ , and DUAKE using **Maul**, ML-KEM, and ML-DSA. The optional third message corresponds to the full-PFS variants. Security levels follow NIST PQC categories.

Protocol	Sec. Level	Msg. 1 (Init.)	Msg. 2 (Resp.)	Msg. 3 (Init.)
DAKE	1	1,626	928	32
	3	2,376	1,488	48
	5	3,323	1,920	64
DAKE^σ	1	858	3,316	32
	3	1,288	4,749	48
	5	1,755	6,483	64
DUAKE	1	858	928	32
	3	1,288	1,488	48
	5	1,755	1,920	64

For comparison, existing ML-KEM-based AKE protocols (e.g., [10, 29]) require transmitting at least one ephemeral public key and three ciphertexts, amounting to 6,272 bytes at NIST 5 security. The Twin-Kyber-based AKE of [59] instead transmits one ephemeral public key and two double-KEM ciphertexts, totaling 6,496 bytes at the same security level, since it employs two double-KEMs in parallel rather than combining a double-KEM with a standard KEM as in DAKE. In contrast, DAKE instantiated with Maul achieves approximately 16% overall communication savings (5,243 bytes) compared to these protocols (NIST 5), with an average reduction of 16% across NIST 1-5 security levels.

Similarly, existing ML-KEM-based UAE protocols (e.g., [10, 51]) require transmitting at least one ephemeral public key and two ciphertexts, for a total of 4,704 bytes at NIST 5 security. In comparison, DUAKE instantiated with Maul achieves about 22% overall communication savings (3,675 bytes) relative to these protocols (NIST 5), with an average reduction of 21% across NIST 1-5 levels.

1.3 Related Works

The only generic AKE we are aware of that leverages a double-KEM is the construction of Xue et al. [59], which provides weak perfect forward secrecy (wPFS) but no explicit authentication. Although it can, in principle, be instantiated with Maul (see Appendix G), their protocol requires transmitting two double-KEM ciphertexts, which is essentially equivalent to four standard KEM encapsulations, whereas it is well known that wPFS AKEs can be realized with only three standard encapsulations [10]. Consequently, the construction does not fully exploit the bandwidth advantage that double-KEMs can offer over standard KEMs. Xue et al. [59] also proposed an FO transform in the ROM that upgrades any 2K2M-PKE with $[\text{IND-CK-CPA}, \text{IND-CK-CPA}]$ security to a double-KEM with $[\text{IND-CK-CCA}, \text{IND-CK-CCA}]$ security. Their transform, however, neither achieves one-sided chosen-key security nor extends to the QROM.

In concurrent and independent work, of which we became aware only after completing ours, Hashimoto, Katsumata, Niot, and Wiggers [32] propose a lattice-based *Reinforced*-KEM (Rebar) that also reuses the $\mathbf{u}$ component of ML-KEM ciphertexts. Unlike our double-KEM Maul, Rebar employs discrete Gaussian distributions but achieves comparable security under the same hardness assumptions. Their construction relies on an FO transform, which (when translated into the double-KEM/2K2M-PKE setting) upgrades a $[\text{IND-CK-CPA}, \text{IND-CPA}]$ 2K2M-PKE to a $[\text{IND-CK-CCA}, \text{IND-CCA}]$ double-KEM. This achieves CCA security on both sides, similar in spirit to the FO of [59], but without adding chosen-key security—our main novelty with the CK-FO transform. While Rebar is applied to optimize post-quantum WireGuard (with proofs in the eCK model), our focus is on designing a general family of (U)AKE protocols. This difference is also reflected in the analyzed properties: they study *identity hiding*, which we leave aside, whereas we emphasize explicit authentication, central to our framework.

Organization

The rest of the paper is organized as follows. Section 2 introduces the necessary notation and background. Section 3 presents our AKE protocols and their security properties. Section 4 describes the CK-FO transform and analyzes its security. Section 5 introduces Maul, establishing its security and parameter choices.

Acknowledgements

This work has been partially supported by the European Union - Next Generation EU through the France Relance program and by the French government through the France 2030 program under the project RESQUE. This work has been further supported by the French Agence Nationale de la Recherche through the SecNISQ project (ANR-21-CE47-0014).

2 Preliminaries

We denote by $\{0, 1\}^*$ the set of binary strings. For any elements a, b from a set, we sometimes write $a = b$ to denote the Boolean predicate that evaluates to 1 if $a = b$ and 0 otherwise. We use the notation $\text{Alg}(\text{arg}_1, \dots, \text{arg}_n; r)$ to denote the execution of a probabilistic algorithm Alg on inputs $\text{arg}_1, \dots, \text{arg}_n$ using randomness r . When the randomness is sampled internally by the algorithm, we omit r for brevity. When a tuple $(x_1, \dots, x_n)$ is used as an argument to a function expecting a string – such as $H((k, m))$ or $\text{Mac}(k, t, (m_1, m_2))$ – we implicitly refer to an injective concatenation of $x_1, \dots, x_n$, meaning that the original components can be unambiguously recovered from it. When the lengths of $x_1, \dots, x_n$ are fixed and publicly known, a simple concatenation suffices. Otherwise, an appropriate encoding must be applied to ensure injectivity.

Double-KEM (2K-KEM). Introduced in [59], a double-KEM, also referred to as a double-key KEM—abbreviated as 2K-KEM in this paper—is a public-key encapsulation mechanism that operates with two pairs of public and secret keys.

Formally, a 2K-KEM $\text{dKEM} = (\text{Setup}, \text{Keygen}_L, \text{Keygen}_R, \text{Encaps}, \text{Decaps})$ is a tuple of five probabilistic polynomial-time (PPT) algorithms. The $\text{Setup}()$ algorithm is responsible for generating public parameters pp . With these parameters, $\text{Keygen}_L(\text{pp})$ generates a left pair of public and secret keys $(\text{pk}_L, \text{sk}_L) \in \text{PK}_L \times \text{SK}_L$, while $\text{Keygen}_R(\text{pp})$ provides a right pair $(\text{pk}_R, \text{sk}_R) \in \text{PK}_R \times \text{SK}_R$. The $\text{Encaps}(\text{pp}, \text{pk}_L, \text{pk}_R)$ algorithm takes the public parameters and both public keys pk_L and pk_R to produce a pair $(\text{ct}, k) \in \text{CT} \times \text{Keys}$, consisting of a ciphertext and a key. The $\text{Decaps}(\text{pp}, \text{sk}_L, \text{sk}_R, \text{ct})$ algorithm then uses the public parameters, secret keys sk_L and sk_R , along with the ciphertext ct , to derive and output the key k or $\perp$ to indicate a decryption failure.

The security games we consider are defined in Figure 3. These games are variations of the indistinguishability games for KEMs. As explained in the introduction, these variations depend on three factors: (1) which side (left or right)

is considered, (2) whether we consider security with chosen-key (CK), and (3) whether a decapsulation oracle is provided to the adversary, leading to CPA or CCA variations. We say that a 2K-KEM is $[X, Y]$ secure if it is $[X, -]$ secure and $[-, Y]$ secure. The advantage in these games, following the notations of Figure 3, is expressed as $|\Pr[b = b'] - 1/2|$.

We also consider key-recovery games $[\text{Key-Recov}, \cdot]$ and $[\cdot, \text{Key-Recov}]$. By symmetry, we only define $[\text{Key-Recov}, \cdot]$. First, we suppose the existence of a function RecovPKL , such that, for any $\text{pp} \leftarrow \text{Setup}()$ and $(\text{pk}_L, \text{sk}_L) \leftarrow \text{Keygen}_L(\text{pp})$, we have $\text{pk}_L = \text{RecovPKL}(\text{pp}, \text{sk}_L)$. The advantage of the game $[\text{Key-Recov}, \cdot]$ is defined as

$$\Pr \left[\text{pk}_L = \text{RecovPKL}(\text{pp}, \text{sk}'_L) : \begin{array}{l} \text{pp} \leftarrow \text{Setup}(), (\text{pk}_L, \text{sk}_L) \leftarrow \text{Keygen}_L(\text{pp}), \\ \text{sk}'_L \leftarrow \mathcal{A}(\text{pp}, \text{pk}_L) \end{array} \right].$$

The notions of correctness and spreadness for a 2K-KEM are defined as follows:

- For $\delta \geq 0$, a 2K-KEM is δ -correct if for $\text{pp} \leftarrow \text{Setup}(), (\text{pk}_L, \text{sk}_L) \leftarrow \text{Keygen}_L(\text{pp})$, $(\text{pk}_R, \text{sk}_R) \leftarrow \text{Keygen}_R(\text{pp})$, and $(\text{ct}, k) \leftarrow \text{Encaps}(\text{pp}, \text{pk}_L, \text{pk}_R)$, the probability that $\text{Decaps}(\text{pp}, \text{sk}_L, \text{sk}_R, \text{ct}) = k$ holds is at least $1 - \delta$.
- For $\gamma > 0$, a 2K-KEM is γ -spread if, for any public parameters $\text{pp} \leftarrow \text{Setup}()$, any $\text{pk}'_L \in \text{PK}_L$, $\text{pk}'_R \in \text{PK}_R$, and any ciphertext ct' , we have: $\Pr[\text{ct} = \text{ct}' : (\text{ct}, k) \leftarrow \text{Encaps}(\text{pp}, \text{pk}'_L, \text{pk}'_R)] \leq 2^{-\gamma}$.

Double-key Double-message PKE (2K2M-PKE). We consider a variant of the double-key PKE of [59] that encrypts a pair of messages. A double-key double-message PKE (2K2M-PKE), functions as a PKE with two pairs of public and secret keys to encrypt two messages into a single ciphertext. Formally, a 2K2M-PKE $\text{DPKE} = (\text{Setup}, \text{Keygen}_L, \text{Keygen}_R, \text{dEnc}, \text{dDec})$ consists of five PPT algorithms. The $\text{Setup}()$ algorithm generates and outputs the public parameters pp . With these parameters, $\text{Keygen}_L(\text{pp})$ generates a left pair of public and secret keys $(\text{pk}_L, \text{sk}_L) \in \text{PK}_L \times \text{SK}_L$, while $\text{Keygen}_R(\text{pp})$ produces a right pair $(\text{pk}_R, \text{sk}_R) \in \text{PK}_R \times \text{SK}_R$. The $\text{dEnc}(\text{pp}, \text{pk}_L, \text{pk}_R, m_L, m_R)$ algorithm takes the public parameters, the public keys pk_L and pk_R , and two messages $(m_L, m_R) \in \mathcal{M}_L \times \mathcal{M}_R$ to generate a ciphertext $\text{ct} \in \text{CT}$. The $\text{dDec}(\text{pp}, \text{sk}_L, \text{sk}_R, \text{ct})$ algorithm then uses the public parameters, secret keys sk_L and sk_R , along with the ciphertext ct , to recover and output the messages $(m_L, m_R) \in \mathcal{M}_L \times \mathcal{M}_R$.

The security games for a 2K2M-PKE are outlined in Figure 4. As for the security games for 2K-KEM, these games are variations of the indistinguishability games for PKE. We say that a 2K2M-PKE is $[X, Y]$ secure if it is $[X, -]$ secure and $[-, Y]$ secure. The advantage in these games, following the notations of Figure 4, is expressed as $|\Pr[b = b'] - 1/2|$.

Finally, for $\delta \geq 0$, a 2K2M-PKE is δ -correct if:

$$\mathbb{E} \left(\max_{m \in \mathcal{M}_L \times \mathcal{M}_R} \Pr[\text{dDec}(\text{pp}, \text{sk}_L, \text{sk}_R, \text{dEnc}(\text{pp}, \text{pk}_L, \text{pk}_R, m)) = m] \right) \geq 1 - \delta,$$

where expected value is taken over $\text{pp} \leftarrow \text{Setup}(), (\text{pk}_L, \text{sk}_L) \leftarrow \text{Keygen}_L(\text{pp})$, and $(\text{pk}_R, \text{sk}_R) \leftarrow \text{Keygen}_R(\text{pp})$.

$[\text{IND-CK-ATK}, -]$ and $[\text{IND-ATK}, -]$	$[-, \text{IND-CK-ATK}]$ and $[-, \text{IND-ATK}]$
if $\text{ATK} = \text{CCA}$ then $\text{Oracles} = \{\mathcal{O}_{\text{CCA}_L}, \mathcal{O}_{\text{leak}_R}\}$ $\text{LeakedR} = \emptyset$ if $\text{ATK} = \text{CPA}$ then $\text{Oracles} = \emptyset$ $\text{pp} \leftarrow \text{Setup}()$ $(\text{pk}_L, \text{sk}_L) \leftarrow \text{Keygen}_L(\text{pp})$ $r_R \leftarrow \\$ \text{RandKeygenR}$ $(\text{pk}_R, \text{sk}_R) \leftarrow \text{Keygen}_R(\text{pp}; r_R)$ $(\text{state}, \text{pk}_R) \leftarrow \mathcal{A}_1^{\text{Oracles}}(\text{pp}, \text{pk}_L)$ $b \leftarrow \$ \{0, 1\}; k_1 \leftarrow \$ \text{Keys}$ $(\text{ct}, k_0) \leftarrow \text{Encaps}(\text{pp}, \text{pk}_L, \text{pk}_R)$ $b' \leftarrow \mathcal{A}^{\text{Oracles}}(\text{pp}, \text{pk}_R, \text{sk}_R, r_R, \text{pk}_L, \text{ct}, k_b)$ $b' \leftarrow \mathcal{A}_2^{\text{Oracles}}(\text{state}, \text{ct}, k_b)$ return $b = b'$ $\mathcal{O}_{\text{leak}_L}()$	if $\text{ATK} = \text{CCA}$ then $\text{Oracles} = \{\mathcal{O}_{\text{CCA}_R}, \mathcal{O}_{\text{leak}_L}\}$ $\text{LeakedL} = \emptyset$ if $\text{ATK} = \text{CPA}$ then $\text{Oracles} = \emptyset$ $\text{pp} \leftarrow \text{Setup}()$ $(\text{pk}_R, \text{sk}_R) \leftarrow \text{Keygen}_R(\text{pp})$ $r_L \leftarrow \\$ \text{RandKeygenL}$ $(\text{pk}_L, \text{sk}_L) \leftarrow \text{Keygen}_L(\text{pp}; r_L)$ $(\text{state}, \text{pk}_L) \leftarrow \mathcal{A}_1^{\text{Oracles}}(\text{pp}, \text{pk}_R)$ $b \leftarrow \$ \{0, 1\}; k_1 \leftarrow \$ \text{Keys}$ $(\text{ct}, k_0) \leftarrow \text{Encaps}(\text{pp}, \text{pk}_L, \text{pk}_R)$ $b' \leftarrow \mathcal{A}^{\text{Oracles}}(\text{pp}, \text{pk}_L, \text{sk}_L, r_L, \text{pk}_R, \text{ct}, k_b)$ $b' \leftarrow \mathcal{A}_2^{\text{Oracles}}(\text{state}, \text{ct}, k_b)$ return $b = b'$ $\mathcal{O}_{\text{leak}_R}()$
$r \leftarrow \$ \text{RandKeygenL}$ $(\text{pk}'_L, \text{sk}'_L) \leftarrow \text{Keygen}_L(\text{pp}; r)$ $\text{LeakedL} \leftarrow \text{LeakedL} \cup \{(\text{pk}'_L, \text{sk}'_L, r)\}$ return $(\text{pk}'_L, \text{sk}'_L, r)$ $\mathcal{O}_{\text{CCA}_L}(\text{pk}'_L, \text{ct}')$	$r \leftarrow \$ \text{RandKeygenR}$ $(\text{pk}'_R, \text{sk}'_R) \leftarrow \text{Keygen}_R(\text{pp}; r)$ $\text{LeakedR} \leftarrow \text{LeakedR} \cup \{(\text{pk}'_R, \text{sk}'_R, r)\}$ return $(\text{pk}'_R, \text{sk}'_R, r)$ $\mathcal{O}_{\text{CCA}_R}(\text{pk}'_L, \text{ct}')$
// with $\text{pp}, \text{ct}, \text{LeakedR}, \text{pk}_R$ and sk_L // from the security game if $(\exists \text{sk}'_R, r : (\text{pk}'_R, \text{sk}'_R, r) \in \text{LeakedR})$ $\wedge (\text{ct}' \neq \text{ct} \vee \text{pk}'_R \neq \text{pk}_R)$ then return $\text{Decaps}(\text{pp}, \text{sk}_L, \text{sk}'_R, \text{ct}')$ if $\text{pk}'_R = \text{pk}_R \wedge \text{ct}' \neq \text{ct}$ then return $\text{Decaps}(\text{pp}, \text{sk}_L, \text{sk}_R, \text{ct}')$ return $\perp$	// with $\text{pp}, \text{ct}, \text{LeakedL}, \text{pk}_L$ and sk_R // from the security game if $(\exists \text{sk}'_L, r : (\text{pk}'_L, \text{sk}'_L, r) \in \text{LeakedL})$ $\wedge (\text{ct}' \neq \text{ct} \vee \text{pk}'_L \neq \text{pk}_L)$ then return $\text{Decaps}(\text{pp}, \text{sk}'_L, \text{sk}_R, \text{ct}')$ if $\text{pk}'_L = \text{pk}_L \wedge \text{ct}' \neq \text{ct}$ then return $\text{Decaps}(\text{pp}, \text{sk}_L, \text{sk}_R, \text{ct}')$ return $\perp$

Fig. 3: Security games of 2K-KEM. $\text{ATK} \in \{\text{CPA}, \text{CCA}\}$. Solid boxes are taken in account only for chosen-key security games (*i.e.* whose name contains CK) while dashed boxes are taken in account only for non-chosen-key games.

[IND-CPA, -]	[-, IND-CPA]
$pp \leftarrow \text{Setup}()$	$pp \leftarrow \text{Setup}()$
$(pk_L, sk_L) \leftarrow \text{Keygen}_L(pp)$	$(pk_R, sk_R) \leftarrow \text{Keygen}_R(pp)$
$r_R \leftarrow \$ \text{RandKeygenR}$	$r_L \leftarrow \$ \text{RandKeygenL}$
$(pk_R, sk_R) \leftarrow \text{Keygen}_R(pp; r_R)$	$(pk_L, sk_L) \leftarrow \text{Keygen}_L(pp; r_L)$
$(\text{state}, m_{L,0}, m_{L,1}, m_R)$	$(\text{state}, m_{R,0}, m_{R,1}, m_L)$
$\leftarrow \mathcal{A}_1(pp, pk_R, sk_R, r_R, pk_L)$	$\leftarrow \mathcal{A}_1(pp, pk_L, sk_L, r_L, pk_R)$
$ct_0 \leftarrow \text{Enc}(pp, pk_L, pk_R, (m_{L,0}, m_R))$	$ct_0 \leftarrow \text{Enc}(pp, pk_L, pk_R, (m_L, m_{R,0}))$
$ct_1 \leftarrow \text{Enc}(pp, pk_L, pk_R, (m_{L,1}, m_R))$	$ct_1 \leftarrow \text{Enc}(pp, pk_L, pk_R, (m_L, m_{R,1}))$
$b \leftarrow \$ \{0, 1\}$	$b \leftarrow \$ \{0, 1\}$
$b' \leftarrow \mathcal{A}_2(\text{state}, ct_b)$	$b' \leftarrow \mathcal{A}(\text{state}, ct_b)$
return $b = b'$	return $b = b'$

Fig. 4: Security games of 2K2M-PKE.

3 Authenticated Key Exchanges from Double-KEMs

In this section we formalize and analyze three authenticated key exchange (AKE) protocols built from [IND-CK-CCA, IND-CCA] double-KEMs. We first recall the security models we use, then present the base protocol DAKE together with its two variants, DAKE^σ and DUAKE. Our treatment integrates the role of additional properties such as match soundness, inverse match soundness and explicit authentication.

3.1 The eCK Security Model and Its Variants

Our analysis builds on two strengthened variants of the extended Canetti–Krawczyk (eCK) model [41]: eCK^w and eCK-PFS [18].¹¹ The former guarantees *weak perfect forward secrecy* (wPFS), and the latter *perfect forward secrecy* (PFS). Both variants ensure key secrecy against a broader class of attack scenarios than those covered by eCK and CK^+ [39].¹²

We provide an overview of these variants; a full formalization is deferred to Appendix C. In this model, the adversary can create parties (as initiators or responders), activate sessions, and interact arbitrarily with them by sending chosen messages. Each session represents a protocol execution by a party (the actor of the session). A simple protocol execution between two parties is therefore represented by two sessions. In each session, the party that the actor intends to

¹¹ As explained in Section 3.2, the analysis of DUAKE relies on an adaptation of these models to the unilateral setting.

¹² This strengthening is due to the use of the *origin session* notion, rather than matching sessions, in certain scenarios [18].

communicate with is called its peer. Each session can be in one of these states: running, completed (the protocol finished without error), or aborted.

Multiple relations can be defined between sessions, and two are of particular importance for the security properties considered in this work:

- Informally, two sessions s, s' are said to be *matching* if they completed the protocol together. More formally, this requires that (1) they both completed, (2) they have distinct roles, (3) the peer of each session equals the actor of the other, and (4) they share the same transcript of sent and received messages.¹³
- A (not necessarily completed) session s' is said to be the *origin* of a session s if the transcript of messages received by s equals the transcript of messages sent by s' .

The eCK model and its variants consider powerful adversarial queries:

- **corrupt**: reveals a party’s long-term secret key.
- **eph-key**: reveals some ephemeral elements computed by a running or completed session.
- **session-key**: reveals the session key of a completed session.

Within this adversarial setting, we are interested in multiple security properties, each defined by the adversary’s goal.

Key Secrecy. Key secrecy is measured by a real-or-random game: the adversary chooses a *test session* s^{Test} of its choice, receives either its true session key or a uniformly sampled one, and must distinguish between the two.

While the adversary can make adversarial queries before and after receiving the challenge key, restrictions on adversarial queries are required to prevent unavoidable attacks (*e.g.*, querying **session-key** on the tested session) and leads to five attack scenarios summarized in Table 4. More precisely, **eCK^w**—guaranteeing only weak perfect forward secrecy (wPFS)—only authorizes scenarios 1 to 4 while **eCK-PFS** authorizes scenarios 1 to 6—guaranteeing full perfect forward secrecy (PFS). Moreover, both models also protect against *maximal exposure* (MEX) attacks and *key-compromise impersonation* (KCI) attacks.

The key secrecy game with the scenarios of **eCK^w** (*resp.* **eCK-PFS**) is called **eCK^w-KeySec** (*resp.* **eCK-PFS-KeySec**).

Match Soundness. Match soundness is defined through a game in which the adversary faces no restrictions on its queries. The adversary wins if it can produce either of the following:

- two completed sessions that are matching but derive different session keys.
- three distinct completed sessions that share the same message transcript.¹⁴

¹³ When comparing the transcripts of two sessions with distinct roles, sent and received messages are inverted, since the outgoing messages of one session correspond to the incoming messages of the other. This convention is applied throughout the paper when comparing transcripts of sessions with different roles.

¹⁴ Recall that when two sessions have distinct roles, transcript equality is checked up to the inversion of sent and received messages, as explained in Footnote 13.

Table 4: Summary of the attack scenarios considered. The adversary $\mathcal{A}$ is never allowed to query `session-key` on s^{Test} or on any session matching s^{Test} . Here, A denotes the actor of s^{Test} , P its peer, and s^{Origin} an origin session of s^{Test} . *Post-comp.* indicates that the corruption query is made after s^{Test} has completed, and *possible* means that the scenario includes both *yes* and *no* cases.

Scenario	corrupt(A)	corrupt(P)	eph-key(s^{Test})	Origin session s^{Origin} exists	eph-key(s^{Origin})
1	no	no	possible	possible	possible
2	yes	no	no	possible	possible
3	no	yes	possible	yes	no
4	yes	yes	no	yes	no
5	yes	post-comp.	no	no	—
6	no	post-comp.	possible	no	—

Inverse-Match Soundness. Inverse-Match Soundness is measured by a game in which the adversary faces no restrictions on its queries. The adversary wins if it produces two distinct completed sessions that output the same session key without being matching. This notion protects in particular against unknown key-share (UKS) attacks, where two parties derive the same session key but disagree on their peer identities.

Explicit Authentication. Explicit Authentication ensures that whenever a session s is completed with an uncorrupted peer, it has done so with an honest partner session whose actor is s_{Peer} —we denote by s_{Actor} and s_{Peer} the actor and peer of s . Care is needed in formalizing this notion: if s sends the final message of the exchange, the mere uncorruptedness of the peer cannot guarantee that this message is actually received by the honest partner.

For additional expressivity we distinguish two properties: *Init-explicit authentication* and *Resp-explicit authentication*. We take a role $X \in \{\text{Init}, \text{Resp}\}$ and denote by $\bar{X}$ the role distinct of X . In the X -explicit authentication game $X\text{-ExplicitAuth}$, the adversary selects a completed session s with role $\bar{X}$, under the sole restriction on adversarial queries that s_{Peer} was not corrupted before s completed. The adversary wins if there does not exist a session s' with actor s_{Peer} , peer s_{Actor} and role X , such that:

- *If the sessions with role X send the final message in the AKE: s and s' share the same message transcript, except possibly for the final message sent by s and received by s' . Note that s' is not required to be completed.*
- *If the sessions with role X do not send the final message in the AKE: s and s' share the same message transcript, and s' is completed. Note that in this case, s and s' are matching.*

3.2 Our AKE Protocols

The base protocol DAKE (Figure 1) is a mutually authenticated key exchange that comes in two versions: DAKE_2 , a two-message variant achieving wPFS in the eCK^w model, and DAKE_3 , a three-message variant achieving PFS in the eCK -PFS model.

Our analysis of DAKE accounts for an eph -key query that can reveal:

- On an initiator session, the randomness used in Keygen_R (which implies knowledge of sk_R). Moreover, $\tilde{k}$ can also be leaked after the session finished to compute its second message.
- On a responder session, nothing is revealed: we assume that the randomness of the encapsulation is securely erased immediately after its computation.

Two additional variants of DAKE are considered. The first, DAKE^σ , achieves responder authentication using a digital signature scheme instead of a KEM (Figure 16, Appendix D.1). While this typically increases overall communication bandwidth, it significantly reduces the size of the initiator’s first message. The second, DUAKE, is a unilateral version providing authentication only for the initiator (Figure 2). Its security is proven under adapted unilateral models (u-eCK^w and u-eCK -PFS), defined in Appendix C.4. Further details on both variants are provided in Appendices D.1 and D.2.

3.3 Security of Our AKE Protocols

We summarize here the security properties of DAKE, DAKE^σ , and DUAKE. Note that all of them are proven in the standard model.

Key Secrecy. It is achieved for DAKE, DAKE^σ , and DUAKE, assuming the $[\text{IND-CK-CCA}, \text{IND-CCA}]$ security of the double-KEM and standard security assumptions on the other primitives. The 2-pass versions provide weak PFS, while the 3-pass versions achieve full PFS. The results for DAKE_3 are stated in Theorem 1, while those for DAKE_2 , DAKE^σ and DUAKE are detailed in Appendix E.

In addition to the definitions provided in the main body, the theorem relies on standard security properties of a KEM, a PRG, and a MAC, which are recalled in Appendix A. We also use the *message-key bound* security of a MAC, which is ensured when a collision-resistant hash function is used as the underlying MAC primitive. This property is formalized by a game MacColl , in which the adversary must find two distinct pairs of keys and messages, $(k, m) \neq (k', m')$, and a tag tag that is valid for both pairs.

Theorem 1 (Key Secrecy of DAKE_3 in the Standard Model). *Let $\mathcal{A}$ be a PPT adversary against key secrecy of DAKE_3 . Suppose it activates at most N parties and, for each party P , at most l sessions with actor or peer P . Assume that the underlying 2K-KEM is γ -spread and δ -correct while the standard KEM is δ' -correct. Let Adv^w denote the advantage of $\mathcal{A}$ against eCK^w -KeySec and Adv^{PFS}*

its advantage against eCK-PFS-KeySec. Then, in the standard model, there exist PPT adversaries $\mathcal{B}_1, \dots, \mathcal{B}_{11}$ such that:

$$\begin{aligned} \text{Adv}^w &\leq \alpha + 4\beta + 6Nl^2 \left(\text{Adv}_{\text{dKEM}}^{[\text{IND-CK-CCA}, -]}(\mathcal{B}_6) + \text{Adv}_{\text{dKEM}}^{[-, \text{IND-CCA}]}(\mathcal{B}_7) \right. \\ &\quad \left. + 2 \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{B}_8) \right), \\ \text{Adv}^{\text{PFS}} &\leq \text{Adv}^w + \beta + 2Nl^2 \left(2 \text{Adv}_{\text{dKEM}}^{[\text{IND-CK-CCA}, -]}(\mathcal{B}_9) + 2 \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{B}_{10}) \right. \\ &\quad \left. + \text{Adv}_{\text{MAC}}^{\text{sUnforg}}(\mathcal{B}_{11}) \right), \end{aligned}$$

where $\alpha = \text{Adv}_{\text{MAC}}^{\text{MacColl}}(\mathcal{B}_1) + N \text{Adv}_{\text{KEM}}^{\text{Key-Recov}}(\mathcal{B}_2) + N^2l \left(2 \text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{B}_3) + l\delta' + \text{Adv}_{\text{MAC}}^{\text{sUnforg}}(\mathcal{B}_4) \right)$ and $\beta = Nl \text{Adv}_{\text{dKEM}}^{[-, \text{Key-Recov}]}(\mathcal{B}_5) + 2Nl^2 (\delta + 2^{-\gamma})$.

Moreover, (1) $\mathcal{B}_3$ queries $\mathcal{O}_{\text{CCA}}$ at most l times, (2) $\mathcal{B}_4$ queries $\mathcal{O}_{\text{Mac}}$ at most l time. (3) $\mathcal{B}_6$ and $\mathcal{B}_9$ each query $\mathcal{O}_{\text{leak}_R}$ and $\mathcal{O}_{\text{CCA}_L}$ at most l times, (4) $\mathcal{B}_7$ does not query $\mathcal{O}_{\text{leak}_L}$ and queries $\mathcal{O}_{\text{CCA}_R}$ at most one time, and (5) $\mathcal{B}_{11}$ queries $\mathcal{O}_{\text{Mac}}$ at most one time.

Proof. The complete proof is given in Appendix E, structured in a modular way so as to simultaneously cover DAKE and DAKE $^\sigma$. In particular, the attacks to be considered are regrouped into a different set of scenarios than those of Table 4, leveraging the structural similarity between DAKE and DAKE $^\sigma$. As explained in Appendix E.1, this approach allows us to precisely isolate where the analyses of these protocols diverge. We now provide only an informal explanation of the core ideas used to prove the key secrecy of DAKE $_3$.

Key secrecy for Scenarios 1–4 (Table 4) is proven via a game sequence that progressively replaces the test session key s^{Test} with a uniformly random value, thereby ensuring that no adversary can win the real-or-random game.

The crucial game of this sequence is the one that replaces the key k of the test session, as well as the key of the unique session that also encapsulates or decapsulates this key (if such a session exists), by a uniform key.

The change in the adversary's advantage introduced by this modification is bounded using one of the security properties of the 2K-KEM dKEM, depending on the type of attack that we define below:

- *Attack of type 1: s^{Test} is an initiator and eph-key is not used on it.* We rely on $[-, \text{IND-CPA}]$, which informally guarantees that k is indistinguishable from uniform, provided that sk_R is unknown and that $(\text{pk}_L, \text{sk}_L)$ has been honestly generated.
- *Attack of type 2: s^{Test} is an initiator and corrupt is not used on it.* We rely on $[\text{IND-CCA}, -]$, which informally ensures that k is indistinguishable from uniform, provided that sk_L is unknown and that $(\text{pk}_R, \text{sk}_R)$ has been honestly generated. Note that, unlike the previous case, CCA security is required here, since multiple sessions distinct from s^{Test} may share the same long-term key $(\text{pk}_L, \text{sk}_L)$. The decapsulation oracle is therefore needed to simulate decapsulations for these sessions without knowing sk_L .

- *Attack of type 3: s^{Test} is a responder and corrupt is not used on its peer.* We need to rely on $[\text{IND-CK-CCA}, -]$, which informally guarantees that k is indistinguishable from uniform provided that sk_L is unknown, even if pk_R is adversarially chosen. The chosen-key property is crucial here, since the ephemeral key pk_R received by s^{Test} may be chosen by the adversary. Moreover, the CCA security is required for the same reason as the previous item.
- *Attack of type 4: s^{Test} is a responder and corrupt is used on its peer.* The situation corresponds to either scenario 3 or 4. Consequently, there exists an origin session s for s^{Test} , and eph-key has not been used on s . In particular, the ephemeral key pk_R received by s^{Test} was honestly generated by s . In this case, we rely on the $[- , \text{IND-CCA}]$ property, which informally guarantees that k is indistinguishable from a uniform key as long as sk_R remains unrevealed and (pk_L, sk_L) has been honestly generated. The CCA security is required in situations where the ciphertext ct received by s differs from the ciphertext created by s^{Test} ; in such cases, the decapsulation oracle must be used on ct to compute the key k of s .

Key secrecy for scenarios 5 and 6 (Table 4) is proven differently. Indeed, we show that these scenarios occur only with negligible probability. More precisely, we first consider the execution only up to the moment when s^{Test} accepts, which implies that we can assume its peer is not corrupted. We then prove that, except with negligible probability, there exists an origin session for s^{Test} , thereby contradicting both scenarios 5 and 6. We provide more details below:

- *If s^{Test} is an initiator*, we use the security of the MAC MAC to show that tag^R must have been honestly computed by a responder session s . From the content of the message authenticated by this MAC, we can then deduce that s is in fact an origin session of s^{Test} . Note that, to invoke the MAC security, we first rely on the IND-CCA security of the KEM KEM to ensure that the key $\tilde{k}$ generated by s^{Test} is indistinguishable from a uniform key. The CCA security is necessary to compute the decapsulations performed by the responder sessions whose actor is $s_{\text{Peer}}^{\text{Test}}$.
- *If s^{Test} is a responder*, we use the security of the MAC to show that tag^I must have been honestly computed by an initiator session s . As in the previous case, the content of the authenticated message allows us to deduce that s is an origin session of s^{Test} . Note that to apply the MAC security, we first rely on the $[\text{IND-CK-CCA}, -]$ security of the 2K-KEM $d\text{KEM}$ and then on the security of PRG to ensure that the key k^{MAC} created by s^{Test} is indistinguishable from uniform. The use of $[\text{IND-CK-CCA}, -]$ is justified in the same way as in the analysis of attacks of type 3 in the Key Secrecy proof for scenarios 1 to 4.

Match Soundness. It is achieved for DAKE, DAKE^σ , and DUAKE. Indeed, the first condition is a direct consequence of the correctness of the used primitives. Concerning the second condition, note that:

- Two distinct initiator sessions can only derive the same key pk_R with negligible probability. The contrary would directly lead to an attack against the right-key-recovery game $[\cdot, \text{Key-Recov}]$ of the 2K-KEM.

- Spreadness of the 2K-KEM implies that two responder sessions will compute the same encapsulation ct with negligible probability.

Inverse-Match Soundness. It is not guaranteed by default in DAKE , DAKE^σ , or DUAKE , but it can be enforced in a straightforward manner by deriving the session key through a collision-resistant key derivation function (KDF). Specifically, each protocol sets the session key as $\text{KDF}(k^{\text{SESSION}}, \tau)$, where k^{SESSION} is the initial session key, and τ is the concatenation of the message transcript and the identities of the actor and its peer.

The advantage of the collision-resistance experiment of a KDF against an adversary $\mathcal{A}$ is defined as¹⁵

$$\Pr[(k, m) \neq (k', m') \wedge \text{KDF}(k, m) = \text{KDF}(k', m') : (k, k', m, m') \leftarrow \mathcal{A}(\text{pp})] .$$

We chose not to include this KDF natively, both for simplicity of the protocol description and because it is only required for this specific property.

Explicit Authentication. It is fully achieved by DAKE_3 and DAKE_3^σ , and partially achieved by our other protocols. More precisely:

- Init-explicit authentication is achieved by DAKE_3 , DAKE_3^σ and DUAKE_3 .
- Resp-explicit authentication holds for DAKE_2 , DAKE_3 , DAKE_2^σ , and DAKE_3^σ .

Theorem 2 provides precise security bounds for explicit authentication in DAKE . All remaining bounds, along with their corresponding proofs, are deferred to Appendix F.

The arguments establishing explicit authentication for our protocols closely follow those used to prove key secrecy in attack scenarios 5 and 6 of Table 4.¹⁶ The core idea is that the security of the MAC (and of the signature in the case of DAKE^σ) guarantees that the tag (or signature) received by a session s was honestly generated by another session s' . The elements authenticated by this tag (or signature) then allow concluding that s' satisfies all the requirements of the explicit authentication property. In particular, to ensure Init-explicit authentication, the message used to compute tag^1 in all our three-message AKEs is defined to include both the transcript of exchanged messages and the identities of the actor and its peer. If Init-explicit authentication is not required, this message can simply be set to any fixed string.

Theorem 2 (Explicit Authentication for DAKE). *Let $\mathcal{A}$ be a PPT adversary against Init-ExplicitAuth for DAKE_3 . Suppose it activates at most N parties and, for each party P , at most l sessions with actor or peer P . Assume that the underlying 2K-KEM is γ -spread and δ -correct. Then, in the standard model, there exist PPT adversaries $\mathcal{B}, \mathcal{C}, \mathcal{D}$ and $\mathcal{E}$ such that:*

$$\begin{aligned} \text{Adv}_{\text{DAKE}_3}^{\text{Init-ExplicitAuth}}(\mathcal{A}) &\leq Nl \text{Adv}_{\text{dKEM}}^{[\cdot, \text{Key-Recov}]}(\mathcal{B}) + 2Nl^2 (\delta + 2^{-\gamma}) \\ &\quad + 2Nl^2 \left(2 \text{Adv}_{\text{dKEM}}^{[\text{IND-CK-CCA}, -]}(\mathcal{C}) + 2 \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{D}) + \text{Adv}_{\text{MAC}}^{\text{sUnforg}}(\mathcal{E}) \right) . \end{aligned}$$

¹⁵ Note that further discussions about this notion are given in Appendix A.

¹⁶ They are described in the proof sketch of Theorem 1 for the case of DAKE_3 .

Moreover, $\mathcal{C}$ queries $\mathcal{O}_{\text{leak}_R}$ and $\mathcal{O}_{\text{CCA}_L}$ at most l times, and $\mathcal{E}$ queries $\mathcal{O}_{\text{Mac}}$ at most one time.

Now let $\Pi \in \{\text{DAKE}_2, \text{DAKE}_3\}$ and suppose that $\mathcal{A}$ is a PPT adversary against Resp-ExplicitAuth for Π . Assume the same conditions on $\mathcal{A}$ as above, and that the standard KEM is δ' -correct. Then, in the standard model, there exist PPT adversaries $\mathcal{B}, \mathcal{C}, \mathcal{D}, \mathcal{E}$ such that $\text{Adv}_{\Pi}^{\text{Resp-ExplicitAuth}}(\mathcal{A})$ is upper-bounded by:

$$\text{Adv}_{\text{MAC}}^{\text{MacColl}}(\mathcal{B}) + N \text{Adv}_{\text{KEM}}^{\text{Key-Recov}}(\mathcal{C}) + N^2 l \left(2 \text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{D}) + l \delta' + \text{Adv}_{\text{MAC}}^{\text{sUnforg}}(\mathcal{E}) \right).$$

Moreover, $\mathcal{D}$ queries $\mathcal{O}_{\text{CCA}}$ at most l times and $\mathcal{E}$ queries $\mathcal{O}_{\text{Mac}}$ at most l times.

4 Chosen-key Fujisaki–Okamoto Transform

In this section, we introduce and analyze a new *chosen-key* Fujisaki–Okamoto transform (CK-FO) which upgrades a double-key, double-message IND-CPA PKE into an IND-CCA double-KEM that additionally provides left chosen-key security. Concretely, in the corresponding security game the adversary may freely choose the (possibly malformed) right public key. Section 4.1 defines the class of 2K2M-PKE schemes compatible with our CK-FO, obtained by combining two PKE instances, while Section 4.2 presents the transform and its security proof.

4.1 Combining Two PKE to Obtain a 2K2M-PKE

We describe a generic method for combining two PKE schemes that share common randomness into a 2K2M-PKE. This construction produces shorter ciphertexts than running the original PKEs independently, while yielding exactly the input format required by our CK-FO transform.

Concretely, let two PKE schemes $\text{PKE}_L = (\text{Setup}, \text{Keygen}_L, \text{Encr}_L, \text{Decr}_L)$ and $\text{PKE}_R = (\text{Setup}, \text{Keygen}_R, \text{Encr}_R, \text{Decr}_R)$ share the same setup algorithm and satisfy the following conditions:

- The randomness spaces are structured as $\text{Rand}_C \times \text{Rand}_L$ for Encr_L and $\text{Rand}_C \times \text{Rand}_R$ for Encr_R , with Rand_C the common part.
- There exists a deterministic algorithm E_{base} that, given (pp, r_C) with public parameters pp and $r_C \in \text{Rand}_C$, outputs c_C such that both Encr_L and Encr_R produce ciphertexts whose first component equals $c_C = E_{\text{base}}(\text{pp}, r_C)$.

Under these conditions, encryptions using the same r_C share the component c_C , which can be factored out to improve compactness. This yields the combined scheme $\text{DPKE} = (\text{dSetup} = \text{Setup}, \text{dKeygen}_L = \text{Keygen}_L, \text{dKeygen}_R = \text{Keygen}_R, \text{dEnc}, \text{dDec})$, with algorithms dEnc and dDec given in Figure 5.

It is important to note that IND-CPA security of PKE_L and PKE_R does not automatically imply any security for DPKE, since the two encryptions are no longer independent. The security of DPKE must therefore be established separately, on a case-by-case basis. In Section 5, we present Maul_{PKE} as an example of this transformation, whose security can be proven under lattice assumptions.

$\text{dEnc}(\text{pp}, \text{pk}_L, \text{pk}_R, m = (m_L, m_R))$	$\text{dDec}(\text{pp}, \text{sk}_L, \text{sk}_R, \text{ct} = (c_L, c_R))$
$r_C \leftarrow \$ \text{Rand}_C$	$m_L = \text{Decr}_L(\text{pp}, \text{sk}_L, c_L)$
$r_L \leftarrow \$ \text{Rand}_L$	$m_R = \text{Decr}_R(\text{pp}, \text{sk}_R, c_R)$
$r_R \leftarrow \$ \text{Rand}_R$	if $m_L = \perp \vee m_R = \perp$ then
$\quad // \text{ Here, } c_C = E_{\text{base}}(\text{pp}, r_C)$	return $\perp$
$(c_C, c_L) = \text{Encr}_L(\text{pp}, \text{pk}_L, m_L; (r_C, r_L))$	return $m = (m_L, m_R)$
$(c_C, c_R) = \text{Encr}_R(\text{pp}, \text{pk}_R, m_R; (r_C, r_R))$	
return (c_C, c_L, c_R)	

Fig. 5: Algorithms dEnc and dDec of the transformation described in Section 4.1.

4.2 The CK-FO Transform and Its Security

Our CK-FO transform takes as input a family of hash functions $(H_{\text{rej}}, H_{\text{key}}, H_{\text{rand}}, H_{\text{OTP}})$ together with a 2K2M-PKE DPKE constructed from two PKE schemes PKE_L and PKE_R , as detailed in Section 4.1. The transform outputs a 2K-KEM, denoted dKEM^{FO} , described in Figure 6.

The CK-FO transform upgrades the $[\text{IND-CPA}, \text{IND-CPA}]$ security of DPKE into the $[\text{IND-CK-CCA}, \text{IND-CCA}]$ security of the resulting 2K-KEM.

A central difficulty in proving $[\text{IND-CK-CCA}, -]$ security is that the adversary may choose the right public key pk_R , which need not be well-formed. Since the 2K2M-PKE is only $[\text{IND-CPA}, -]$ -secure, we can guarantee the security of c_R only when pk_R is generated by Keygen_R . Otherwise, c_R might leak information about both its encapsulated message m_R and the common randomness r_C , potentially revealing the message m_L encapsulated in c_L .

Our transform resolves this by concealing c_R using m_L (line 6 of $\text{dEncaps}^{\text{FO}}$ in Figure 6).¹⁷ This ensures c_R remains hidden until m_L is recovered, allowing us to reduce the $[\text{IND-CK-CCA}, -]$ security of dKEM^{FO} to the IND-CPA security of PKE_L . The proof therefore relies fundamentally on the structure of the input 2K2M-PKE and, in particular, on the security of its left constituent PKE PKE_L .

To obtain security in the QROM rather than only in the ROM, we must carefully select which elements are hashed. In particular, the encapsulation randomness is not derived solely from the messages m_L and m_R , but also incorporates the two public keys, as shown in the algorithm $\text{dEncaps}^{\text{FO}}$ in Figure 6. This choice is essential for the QROM proof, which uses semi-classical oracles [3], following the approach of [24].

The following theorem summarizes the security guarantees of our CK-FO transform in both the ROM and the QROM. The corresponding proofs are provided in Appendix B.

¹⁷ Note that the xor is well-defined: we assume that the set of right ciphertexts is contained in the binary space $\{0, 1\}^\tau$ and choose H_{OTP} to have range $\{0, 1\}^\tau$.

$\text{dSetup}^{\text{FO}}()$	$\text{dKeygen}_L^{\text{FO}}(\text{pp})$	$\text{dKeygen}_R^{\text{FO}}(\text{pp})$
$\text{pp} \leftarrow \text{Setup}()$	$\text{s}_L \leftarrow_{\$} \{0, 1\}^\nu$	$\text{s}_R \leftarrow_{\$} \{0, 1\}^\nu$
return pp	$(\text{pk}_L, \text{sk}_L) \leftarrow \text{Keygen}_L(\text{pp})$	$(\text{pk}_R, \text{sk}_R) \leftarrow \text{Keygen}_R(\text{pp})$
	$\text{sk}_L^{\text{FO}} = (\text{sk}_L, \text{s}_L)$	$\text{sk}_R^{\text{FO}} = (\text{sk}_R, \text{s}_R)$
	return $(\text{pk}_L, \text{sk}_L^{\text{FO}})$	return $(\text{pk}_R, \text{sk}_R^{\text{FO}})$
$\text{dEncaps}^{\text{FO}}(\text{pp}, \text{pk}_L, \text{pk}_R)$	$\text{dDecaps}^{\text{FO}}(\text{pk}_L, \text{pk}_R, \text{sk}_L^{\text{FO}} = (\text{sk}_L, \text{s}_L), \text{sk}_R^{\text{FO}} = (\text{sk}_R, \text{s}_R), (\text{c}_C, \text{c}_L, \text{c}_R^{\text{OTP}}))$	
$m_L \leftarrow_{\$} \mathcal{M}_L$	$k' \leftarrow H_{\text{rej}}(\text{pk}_L, \text{pk}_R, \text{s}_L, \text{s}_R, \text{c}_C, \text{c}_L, \text{c}_R^{\text{OTP}})$	
$m_R \leftarrow_{\$} \mathcal{M}_R$	$m'_L \leftarrow \text{Decr}_L(\text{pp}, \text{sk}_L, (\text{c}_C, \text{c}_L))$	
$(r_C, r'_L, r'_R) \leftarrow H_{\text{rand}}(\text{pk}_L, \text{pk}_R, m_L, m_R)$	if $m'_L = \perp$ then return k'	
$(\text{c}_C, \text{c}_L) = \text{Encr}_L(\text{pp}, \text{pk}_L, m_L; r_C, r'_L)$	$\text{c}_R \leftarrow \text{c}_R^{\text{OTP}} \oplus H_{\text{OTP}}(m'_L)$	
$(\text{c}_C, \text{c}_R) = \text{Encr}_R(\text{pp}, \text{pk}_R, m_R; r_C, r'_R)$	$m'_R \leftarrow \text{Decr}_R(\text{pp}, \text{sk}_R, (\text{c}_C, \text{c}_R))$	
$\text{c}_R^{\text{OTP}} \leftarrow \text{c}_R \oplus H_{\text{OTP}}(m_L)$	if $m'_R = \perp$ then return k'	
$k \leftarrow H_{\text{key}}(\text{pk}_L, \text{pk}_R, m_L, m_R, \text{c}_C, \text{c}_L, \text{c}_R^{\text{OTP}})$	$(r'_C, r'_L, r'_R) \leftarrow H_{\text{rand}}(\text{pk}_L, \text{pk}_R, m'_L, m'_R)$	
return $(k, (\text{c}_C, \text{c}_L, \text{c}_R^{\text{OTP}}))$	if $\text{Encr}_L(\text{pk}_L, m'_L; r'_C, r'_L) \neq (\text{c}_C, \text{c}_L)$ then return k'	
	if $\text{Encr}_R(\text{pk}_R, m'_R; r'_C, r'_R) \neq (\text{c}_C, \text{c}_R)$ then return k'	
	return $H_{\text{key}}(\text{pk}_L, \text{pk}_R, m'_L, m'_R, \text{c}_C, \text{c}_L, \text{c}_R^{\text{OTP}})$	

Fig. 6: Our CK-FO Transform. $(\text{dSetup}, \text{dKeygen}_L, \text{dKeygen}_R, \text{dEnc}, \text{dDec})$ is a 2K2M-PKE built from two PKEs PKE_L and PKE_R , as described in Section 4.1.

Theorem 3 (CK-FO transform in ROM and QROM). *Consider random oracles H_{key} , H_{rej} , H_{OTP} , and H_{rand} , and let DPKE constructed from PKE_L and PKE_R be δ -correct. Let $\mathcal{A}$ be an adversary against the $[\text{IND-CK-CCA}, -]$ (resp. $[-, \text{IND-CCA}]$) security of dKEM^{FO} , making at most q_{leak} , q_{CCA} , q_{key} , q_{rej} , q_{OTP} and q_{rand} queries to the oracles $\mathcal{O}_{\text{leak}_R}$ (resp. $\mathcal{O}_{\text{leak}_L}$), $\mathcal{O}_{\text{CCA}_L}$ (resp. $\mathcal{O}_{\text{CCA}_R}$), H_{key} , H_{rej} , H_{OTP} and H_{rand} . Then there exists an adversary $\mathcal{B}$ such that:*

$$\begin{aligned} \text{Adv}_{\text{dKEM}^{\text{FO}}}^{[\text{IND-CK-CCA}, -]}(\mathcal{A}) &\leq 2 \text{Adv}_{\text{PKE}_L}^{\text{IND-CPA}}(\mathcal{B}) + \frac{q_{\text{rej}}}{2^\nu} + q_{\text{rand}}\delta + \frac{q_{\text{rand}} + q_{\text{key}} + q_{\text{OTP}}}{|\mathcal{M}_L|}, \\ \text{Adv}_{\text{dKEM}^{\text{FO}}}^{[-, \text{IND-CCA}]}(\mathcal{A}) &\leq 2 \text{Adv}_{\text{DPKE}}^{[-, \text{IND-CPA}]}(\mathcal{B}) + \frac{q_{\text{rej}}}{2^\nu} + q_{\text{rand}}\delta + \frac{q_{\text{key}}}{|\mathcal{M}_R|}. \end{aligned}$$

If we operate in the quantum-accessible random oracle model with quantum-accessible oracles H_{key} , H_{rej} , H_{OTP} , and H_{rand} , and q_{key} , q_{rej} , q_{OTP} , q_{rand} denote

the respective query bounds, then there exists an adversary $\mathcal{B}$ such that:

$$\begin{aligned} \text{Adv}_{\text{dKEM}^{\text{FO}}}^{[\text{IND-CK-CCA}, -]}(\mathcal{A}) &\leq \beta + 2\sqrt{(\hat{q} + 1) \left(2 \text{Adv}_{\text{PKE}_L}^{\text{IND-CPA}}(\mathcal{B}) + 4\hat{q}/|\mathcal{M}_L| \right)}, \\ \text{Adv}_{\text{dKEM}^{\text{FO}}}^{[-, \text{IND-CCA}]}(\mathcal{A}) &\leq \tilde{\beta} + 2\sqrt{(\tilde{q} + 1) \left(2 \text{Adv}_{\text{DPKE}}^{[-, \text{IND-CPA}]}(\mathcal{B}) + 4\tilde{q}/|\mathcal{M}_R| \right)}, \end{aligned}$$

where $\beta = 2 q_{\text{rej}} 2^{-\nu/2} + 16 q_{\text{leak}} \delta (q_{\text{rand}} + q_{\text{CCA}} + 1)^2$, $\tilde{\beta} = 2 q_{\text{rej}} 2^{-\nu/2} + 16 (q_{\text{leak}} + 1) \delta (q_{\text{rand}} + q_{\text{CCA}} + 1)^2$, $\hat{q} = q_{\text{rand}} + q_{\text{key}} + q_{\text{OTP}} + q_{\text{CCA}}$ and $\tilde{q} = q_{\text{rand}} + q_{\text{key}} + q_{\text{CCA}}$.

In summary, $[-, \text{IND-CCA}]$ security reduces to the $[-, \text{IND-CPA}]$ security of DPKE, and $[\text{IND-CK-CCA}, -]$ security reduces to the IND-CPA security of PKE_L .

5 Compact Lattice-based Double-KEM

In this section we introduce *Maul*, a compact 2K-KEM inspired by Kyber [50] and achieving $[\text{IND-CK-CCA}, \text{IND-CCA}]$ security. Its security is proven in both the ROM and QROM under the MLWE and Hint-MLWE assumptions.

Maul is obtained by applying our CK-FO transform (Section 4) to Maul_{PKE} , a $[\text{IND-CPA}, \text{IND-CPA}]$ -secure 2K2M-PKE.

After introducing the necessary preliminaries in Section 5.1, we present the 2K2M-PKE Maul_{PKE} and establish its security in Section 5.2. Finally, the concrete parameters of *Maul* are provided in Section 5.3 and justified in Sections 5.4 and 5.5.

5.1 Preliminaries on Lattices, MLWE, and Hint-MLWE Problems

Linear Algebra and Modular Operations. Let A be a ring. Vectors $\mathbf{u} \in A^n$ are assumed to be column vectors, and we denote by $\mathbf{u}^\top$ the transpose of $\mathbf{u}$. For $x \in \mathbb{R}$, we denote by $\lfloor x \rfloor$ the nearest integer to x , with ties rounded up. We define the polynomial ring $\mathcal{R} := \mathbb{Z}[X]/(X^n + 1)$, where $n \in \mathbb{N}$ is called the *dimension*, to be specified as needed. For a modulus $q \in \mathbb{N}$, we define the quotient ring $\mathcal{R}_q := \mathbb{Z}_q[X]/(X^n + 1)$. For any $q > 1$, we identify $\mathcal{R}_2$ with the subset of polynomials in $\mathcal{R}_q$ whose coefficients lie in $\{0, 1\}$. For a real matrix $\mathbf{H} \in \mathbb{R}^{d \times \ell}$, we define the spectral norm $s_1(\mathbf{H})$ as $s_1(\mathbf{H}) = \max_{\mathbf{x} \neq 0} \frac{\|\mathbf{H}\mathbf{x}\|_2}{\|\mathbf{x}\|_2}$, where $\|\cdot\|_2$ denotes the euclidean norm. For $\mathbf{H} \in \mathcal{R}^{d \times \ell}$, we identify it with an associated negacyclic matrix $\tilde{\mathbf{H}} \in \mathbb{Z}^{nd \times n\ell}$, and define the spectral norm of $\mathbf{H}$ as that of $\tilde{\mathbf{H}}$.

Throughout this work, unless otherwise stated, all operations are performed modulo a positive integer $q \in \mathbb{N}$, which will be specified in context.

MLWE and Hint-MLWE Problems. We rely on the *module learning with errors* (MLWE) problem [42], as well as a generalized variant known as the Hint-MLWE problem [38], in which the adversary is additionally given partial information (or *hints*) about the MLWE secret and error vectors.

Definition 1 (MLWE). Let k, m, q be positive integers, and let χ be a distribution over $\mathcal{R}^{k+m}$. The advantage of an adversary $\mathcal{A}$ against the module learning with errors problem $\text{MLWE}_{\mathcal{R},k,m,q}^\chi$ is defined as: $|\Pr[\mathcal{A}(\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e}) = 1] - \Pr[\mathcal{A}(\mathbf{A}, \mathbf{u}) = 1]|$, where $\mathbf{A} \leftarrow \mathcal{U}(\mathcal{R}_q^{m \times k})$, $(\begin{smallmatrix} \mathbf{s} \\ \mathbf{e} \end{smallmatrix}) \leftarrow \chi$ and $\mathbf{u} \leftarrow \mathcal{U}(\mathcal{R}_q^m)$.

In the original Hint-MLWE formulation [38, Definition 7], the adversary is given noisy linear hints of the form $\gamma_i \cdot \mathbf{r} + \mathbf{y}_i$, where $\mathbf{r} = (\begin{smallmatrix} \mathbf{s} \\ \mathbf{e} \end{smallmatrix})$ is the concatenated secret-error vector, $\mathbf{y}_i$ are noise terms, and γ_i are public polynomials. This was later generalized in [25] to hints of the form $\mathbf{H}\mathbf{r} + \mathbf{y}$, where $\mathbf{H} \in \mathcal{R}_q^{\ell \times 2k}$ and $\mathbf{A} \in \mathcal{R}_q^{k \times k}$. We consider a slight extension of this generalization, where $\mathbf{A} \in \mathcal{R}_q^{m \times k}$ and $\mathbf{H} \in \mathcal{R}_q^{\ell \times d}$ for some $d \leq k + m$. This allows the hints to depend only on a subset of the secret-error vector.

Definition 2 (Hint-MLWE, adapted from [25]).

Let k, m, ℓ, d, q be positive integers with $d \leq k + m$, and let $J \subseteq \mathbb{Z} \cap [0, k + m - 1]$ be an index set of size d . Let χ be a distribution over $\mathcal{R}^{k+m}$, χ_h a distribution over $\mathcal{R}^\ell$, and $\mathcal{F}$ a distribution over $\mathcal{R}_q^{\ell \times d}$.

The advantage of an adversary $\mathcal{A}$ against the hint module learning with errors problem $\text{Hint-MLWE}_{\mathcal{R},k,m,q,\ell,d,J}^{\chi, \chi_h, \mathcal{F}}$ is defined as $|\Pr[\mathcal{A}(\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e}, \mathbf{H}, \mathbf{H}\mathbf{r} + \mathbf{y}) = 1] - \Pr[\mathcal{A}(\mathbf{A}, \mathbf{b}, \mathbf{H}, \mathbf{H}\mathbf{r} + \mathbf{y}) = 1]|$, where $(\mathbf{A}, \mathbf{b}, \mathbf{r}, \mathbf{H}, \mathbf{y}) \leftarrow \mathcal{U}(\mathcal{R}_q^{m \times k}) \times \mathcal{U}(\mathcal{R}_q^m) \times \chi \times \mathcal{F} \times \chi_h$, with $\mathbf{r} := (\begin{smallmatrix} \mathbf{s} \\ \mathbf{e} \end{smallmatrix}) \in \mathcal{R}^{k+m}$, and where $\mathbf{r} = (\mathbf{r}_i)_{i \in J} \in \mathcal{R}^d$ consists of the components of $\mathbf{r}$ indexed by J .

In [38], it is shown that the Hint-MLWE problem is at least as hard as the MLWE problem when the secret and error are sampled from discrete Gaussian distributions, assuming certain constraints on the width parameters. For completeness, we present an adaptation of this reduction that applies to the generalized Hint-MLWE formulation in Definition 2, under the same parameter regime. It is proven in Appendix H.1.

Theorem 4 (Hardness of Hint-MLWE, adapted from [38]).

Let k, m, ℓ, d, q, J be as in Definition 2, and let $\mathcal{F}$ be a distribution over $\mathcal{S}_B := \{\mathbf{H} \in \mathcal{R}_q^{\ell \times d} \mid s_1(\mathbf{H})^2 \leq B\}$, for $B > 0$. Let $\sigma_1, \sigma_2 > 0$, and define $\sigma > 0$ via $\frac{1}{\sigma^2} = 2 \left(\frac{1}{\sigma_1^2} + \frac{B}{\sigma_2^2} \right)$. Let $\chi_h := D_{\mathbb{Z}^{\ell \cdot n}, \sigma_2}$, and let χ be a distribution over $\mathcal{R}^{k+m-d}$. Define χ_σ over $\mathcal{R}^{k+m}$ as the distribution where coordinates in J are drawn from $D_{\mathbb{Z}^{d \cdot n}, \sigma}$ and the rest from χ .

If $\sigma \geq \sqrt{2} \cdot \eta_\varepsilon(\mathbb{Z}^{d \cdot n})$ for some $0 < \varepsilon \leq 1/2$, then there exists an efficient reduction from $\text{MLWE}_{\mathcal{R},k,m,q}^{\chi_\sigma}$ to $\text{Hint-MLWE}_{\mathcal{R},k,m,q,\ell,d,J}^{\chi_{\sigma_1}, \chi_h, \mathcal{F}}$ that reduces the advantage by at most 2ε .

5.2 Maul_{PKE}: A [IND-CPA, IND-CPA]-Secure 2K2M-PKE

Let $k \in \mathbb{N}$, two distributions $\mathcal{B}, \mathcal{D}$ over $\mathcal{R}$, and compression parameters $d_u, d_v \leq \log_2(q)$. For $x \in \mathbb{Q}$, we define the compression $\text{Comp}_{(q, d_u)}(x) = \lfloor (2^{d_u} x) / q \rfloor \bmod 2^{d_u}$, and the decompression $\text{Decomp}_{(q, d_v)}(x) = \lfloor (qx) / 2^{d_v} \rfloor$.

Consider the Kyber-like PKE DualRegev defined in Figure 7. Its encryption algorithm uses three sources of randomness (s_C, e_C, f) , producing ciphertexts (c_C, c_{scal}) , where c_C depends only on (s_C, e_C) . By applying the transformation from Section 4.1, we combine two instances of DualRegev into a 2K2M-PKE that reuses the randomness pair (s_C, e_C) . The resulting scheme, Maul_{PKE} , is detailed in Figure 8.

DualRegev.Setup ()	DualRegev.Keygen (pp = A)	DualRegev.Decr (pp = A, sk = s, ct = (c_C, c_scal))
$A \leftarrow \mathcal{U}(\mathcal{R}_q^{k \times k})$	$(s, e) \leftarrow \mathcal{B}^{2k}$	$m' \leftarrow \text{Decomp}_{(q, d_v)}(c_{scal})$
return A(= pp)	$t \leftarrow As + e$	$- \text{Decomp}_{(q, d_u)}(c_C)^T s$
	return (t(= pk), s(= sk))	return $\lfloor 2m'/q \rfloor$
DualRegev.Encr (pp = A, pk = t, m $\in \mathcal{R}_2$)		
$(s_C, e_C) \leftarrow \mathcal{B}^{2k}$	$c_C \leftarrow \text{Comp}_{(q, d_u)}(A^T s_C + e_C)$	
$f \leftarrow \mathcal{D}$	$c_{scal} \leftarrow \text{Comp}_{(q, d_v)}(s_C^T t + f + \lceil q/2 \rceil m)$	
	return ct = (c_C, c_scal)	

Fig. 7: Algorithms for DualRegev.

Maul _{PKE} .Setup ()	Maul _{PKE} .KeygenL (pp = A)	Maul _{PKE} .KeygenR (pp = A)
$A \leftarrow \mathcal{U}(\mathcal{R}_q^{k \times k})$	$(s_L, e_L) \leftarrow \mathcal{B}^{2k}$	$(s_R, e_R) \leftarrow \mathcal{B}^{2k}$
return A(= pp)	$t_L \leftarrow As_L + e_L$	$t_R \leftarrow As_R + e_R$
	return (pk _L = t _L , sk _L = s _L)	return (pk _R = t _R , sk _R = s _R)
Maul _{PKE} .Encr (pp = A, pk _L = t _L , pk _R = t _R , (m _L , m _R) $\in \mathcal{R}_2^2$)	Maul _{PKE} .Decr (pp = A, sk _L = s _L , sk _R = s _R , ct = (c_C, c_L, c_R))	
$(s_C, e_C) \leftarrow \mathcal{B}^{2k}$	$m'_L \leftarrow \text{Decomp}_{(q, d_v)}(c_L)$	
$(f_L, f_R) \leftarrow \mathcal{D}^{2k}$	$- \text{Decomp}_{(q, d_u)}(c_C)^T s_L$	
$c_C \leftarrow \text{Comp}_{(q, d_u)}(A^T s_C + e_C)$	$m'_R \leftarrow \text{Decomp}_{(q, d_v)}(c_R)$	
$c_L \leftarrow \text{Comp}_{(q, d_v)}(s_C^T t_L + f_L + \lceil q/2 \rceil m_L)$	$- \text{Decomp}_{(q, d_u)}(c_C)^T s_R$	
$c_R \leftarrow \text{Comp}_{(q, d_v)}(s_C^T t_R + f_R + \lceil q/2 \rceil m_R)$	return ($\lfloor 2m'_L/q \rfloor, \lfloor 2m'_R/q \rfloor$)	
return ct = (c_C, c_L, c_R)		

Fig. 8: Algorithms for Maul_{PKE}.

Theorem 5 analyses the [IND-CPA, -] and [- , IND-CPA] security of Maul_{PKE}. Its correctness is analyzed in the following lemma, proven in Appendix H.2.

Lemma 1 (Maul_{PKE} correctness). *Using the notation from Figure 8, the scheme Maul_{PKE} is δ -correct provided that*

$$\Pr \left[\left\| \begin{pmatrix} f_L + e_{c_L} + s_C^\top e_L - (e_C + e_{c_C})^\top s_L \\ f_R + e_{c_R} + s_C^\top e_R - (e_C + e_{c_C})^\top s_R \end{pmatrix} \right\|_\infty \leq \frac{q}{4} \right] > 1 - \delta,$$

where, $e_{c_C} = \text{Decomp}_{(q,d_u)}(c_C) - \mathbf{A}^\top s_C + e_C$, $e_{c_L} = \text{Decomp}_{(q,d_v)}(c_L) - s_C^\top t_L + f_L + m_L$, and $e_{c_R} = \text{Decomp}_{(q,d_v)}(c_R) - s_C^\top t_R + f_R + m_R$.

Theorem 5 ([IND-CPA, IND-CPA] Security of Maul_{PKE}). *Let consider $X \in \{[\text{IND-CPA}, -], [-, \text{IND-CPA}]\}$. Let $\mathcal{A}$ be a PPT adversary against the X security of Maul_{PKE}, there exist PPT adversaries $\mathcal{B}, \mathcal{C}$ with same time complexity as $\mathcal{A}$ such that:*

$$\text{Adv}_{\text{MaulPKE}}^X(\mathcal{A}) \leq \text{Adv}_{k,k,\mathcal{B}^k,\mathcal{B}^k}^{\text{MLWE}}(\mathcal{B}) + \text{Adv}_{\mathcal{R},k,k+1,q,1,2k,J}^{\text{Hint-MLWE}^{X,\chi_h,\mathcal{F}}}(\mathcal{C}).$$

Where $\mathcal{F} = \mathcal{B}^k \times (-\mathcal{B}^k)$, $\chi_h = \mathcal{D}$, $\chi = \mathcal{B}^{2k} \times \mathcal{D}$ and the subset J being the $2k$ first elements.

Proof. The complete proof is provided in Appendix H.2. We briefly outline here the argument for [IND-CPA, -] security; the proof for [-, IND-CPA] follows symmetrically. Moreover, it suffices to consider the variant of the scheme in which the functions **Compress** and **Decompress** are replaced by the identity.

Our goal is to show that the left ciphertext $c_{L,b}$ given to the adversary $\mathcal{A}$ is indistinguishable from uniform. Consequently, $\mathcal{A}$ cannot determine which left message was encrypted.

To establish this, we first introduce a game G_1 in which the vector t_L is sampled uniformly at random, and the vectors e_L and s_L are no longer used. The difference in the adversary's advantage between the two games is then bounded by the MLWE advantage specified in the theorem statement.

Using the notation $\mathbf{B}^\top = (\mathbf{A} \mid t_L)$, we can express the pair $(c_C, \tilde{c}_L)$ as $(c_C^\top, \tilde{c}_L^\top)^\top = \mathbf{B} s_C + (e_C^\top, f_L)^\top$. In G_1 , we know that $\mathbf{B}$ is uniform. We would thus like to invoke the MLWE assumption to argue that $(c_C^\top, \tilde{c}_L^\top)^\top$ is indistinguishable from uniform. However, the secret s_C also appears in another term: $c_R = s_C^\top t_R + f_R + \lceil q/2 \rceil m_R$, which prevents a direct application of the MLWE hypothesis. To address this, we instead rely on the Hint-MLWE hypothesis.

5.3 Parameter Selection

In this section, we propose concrete parameters for Maul, our 2K-KEM construction, targeting λ -bit [IND-CK-CCA, IND-CCA] security for $\lambda \in \{128, 192, 256\}$, corresponding respectively to NIST security levels 1, 3, and 5. These parameters are selected with consideration of the security implications of the Hint-MLWE assumption, on which the scheme ultimately relies.

Table 5 summarizes the parameter sets for each target security level. As shown in Appendix H.3, instantiating Maul with Kyber's original parameters

Table 5: Parameter sets for Maul, including sizes (in bytes).

	bikz	n	k	q	$\mathcal{B}$	$\mathcal{D}$	(d_u, d_v)	δ	ν	$ \text{pk} $	$ \text{ct} $	$ \text{pk} + \text{ct} $
Maul512	337	256	2	7681	$\mathcal{B}_4$	$\mathcal{D}_{64}$	(10, 4)	2^{-150}	384	826	896	1722
Maul768	557	256	3	7681	$\mathcal{B}_4$	$\mathcal{D}_{64}$	(11, 6)	2^{-196}	512	1240	1440	2680
Maul1024	766	256	4	9473	$\mathcal{B}_4$	$\mathcal{D}_{64}$	(12, 5)	2^{-257}	640	1691	1856	3547

yielded a decapsulation failure probability δ above our design target, mainly due to the large variance of the distribution $\mathcal{D}$. To restore the intended failure probability, we slightly adjusted the parameters. For each set, we report the estimated BKZ block size (bikz) for solving the underlying Hint-MLWE problem, the ring dimension n , key dimension k , modulus q (NTT-friendly), compression parameters (d_u, d_v) , and the distributions $(\mathcal{B}, \mathcal{D})$ used in key generation and encapsulation. Here, $\mathcal{B}_\eta$ denotes the centered binomial distribution with parameter η as defined in [54], and $\mathcal{D}_\sigma$ a *Gaussian-like* distribution of standard deviation σ , encompassing centered binomials, sums of uniforms, and discrete Gaussians. We also provide the public key and ciphertext sizes (in bytes). Estimates for bikz and δ were obtained using a modified version of the scripts¹⁸.

5.4 Security Estimation

Our security analysis is grounded in the assumed hardness of the MLWE problem, assessed via the *core-SVP* cost model [1]. This model estimates security levels against both classical and quantum attacks by relating them to the required BKZ block size (bikz) needed to solve the underlying MLWE instance. A conservative estimate is then derived by applying the complexity of the best known sieving algorithms—both classical and quantum [6, 15]—to this block size, deliberately omitting polynomial factors in the full BKZ complexity. These omitted factors typically account for around 30 bits of security.

Transitioning from Hint-MLWE to standard MLWE for parameter estimation involves subtleties, as the distributions and parameter regimes fall outside Theorem 4 reduction. To address this, we follow the heuristic approach of Dodis et al. [22], which guides our parameter selection and hardness estimation for Hint-MLWE. Specifically, we rely on the following three heuristics:

Heuristic 1: Replacing Gaussian Distributions. Although Theorem 4 formally applies to discrete Gaussians with widths above the smoothing parameter, we assume similar hardness for non-Gaussian distributions of equal variance. This is justified by their *Gaussian-like* behavior for large parameters and consistent performance in Rényi-divergence analyses.

¹⁸ <https://github.com/pq-crystals/security-estimates>

Heuristic 2: Removing the Factor 2. We omit the factor 2 in the expression $1/\sigma^2 = 2 \cdot (1/\sigma_1^2 + s_1(\mathbf{H})^2/\sigma_2^2)$ originally introduced in the Hint-MLWE reduction. As noted in [22], this factor arises from smoothing arguments specific to discrete Gaussians and is irrelevant for non-Gaussian distributions. Under this simplification, the relation between the equivalent Gaussian parameters becomes $1/\sigma^2 = 1/\sigma_1^2 + s_1(\mathbf{H})^2/\sigma_2^2$.

Heuristic 3: Approximate Spectral Norm. The security reduction involves the spectral norm $s_1(\mathbf{H})$ of the hint-related matrix $\mathbf{H} = (\mathbf{e}_L^\top, -\mathbf{s}_L^\top)$. Instead of bounding this norm in the worst-case, we adopt the average-case approximation, yielding that $s_1(\mathbf{H})^2$ is replaced by $n \cdot \sigma_1^2$. This estimation gives us a baseline to assess the hardness of Hint-MLWE, with $\sigma_2 = \sqrt{n} \sigma_1^2$, as that of the MLWE problem, with $\sigma = \sigma_1/\sqrt{2}$.

5.5 Concrete Security Loss

We emphasize that the heuristic approach described above does not formally capture potential security loss arising from the reduction. In particular, the reduction from MLWE to Hint-MLWE assumes that the standard deviation of the involved distributions exceeds the smoothing parameter—an assumption that ensures a tight reduction. Since the distributions used in **Maul** fall below this threshold, some degree of security degradation may be expected.

Nevertheless, we conjecture that even under these conditions, the security of Hint-MLWE instantiated with parameters σ_1 and $\sigma_2 = \sqrt{n} \sigma_1^2$ remains higher than that of MLWE instantiated with parameter $\sigma_1/\sqrt{2}$. To support this conjecture, we conduct empirical analyses and propose a concrete attack against **Maul** that explicitly exploits the hint. Further details are provided in Appendix H.3.

LWE with Side Information [19]. The most effective known attack against Hint-MLWE in the case of **Maul** parameters is based on the framework of Ducas et al. [19], which extends the primal attack to incorporate auxiliary information. The standard primal attack on MLWE (typically ignoring the module structure) consists in interpreting—via appropriate embedding—the LWE instance as a *bounded distance decoding* (BDD) instance, which is then tackled using lattice reduction techniques. The framework of [19] generalizes the BDD problem by accounting for non-spherical distributions and modeling the hint as side information that modifies the target distribution.

Specifically, *approximate hints* are particularly relevant in the context of **Maul** when an adversary, given $(\mathbf{e}_L, \mathbf{s}_L)$, attempts to recover the encapsulated message m_R without access to $(\mathbf{e}_R, \mathbf{s}_R)$.¹⁹ These hints take the form of noisy linear relations $\mathbf{z} = \mathbf{H} \begin{pmatrix} \mathbf{s} \\ \mathbf{e} \end{pmatrix} + \mathbf{y}$, which match the structure of the Hint-MLWE hint in **Maul**, with $\mathbf{H} = (\mathbf{e}_L^\top, -\mathbf{s}_L^\top)$, $(\mathbf{s}, \mathbf{e}) = (\mathbf{s}_C, \mathbf{e}_C)$, and $\mathbf{y} = \mathbf{f}_L$. The framework then

¹⁹ This is the only attack scenario in which the adversary faces a Hint-MLWE instance; in the symmetric left-side case, the one-time pad introduced by our CK-FO transform precludes any such attack.

applies a suitable linear transformation that maps the distorted instance back to a spherical BDD instance, which is more amenable to attack.

To estimate the residual security under this model, we assume that the vectors $\mathbf{s}$, $\mathbf{e}$, and $\mathbf{y}$ follow continuous Gaussian distributions with variances matching those of their actual distributions. This approximation enables the use of the leaky-LWE estimator from [19]²⁰ to evaluate the impact of approximate hints.

Table 6: Estimated security ($\text{PrimalLWE}_{\Sigma_{\text{Hints}}}$) in bikz for **Maul** parameters, including upper bound ($\text{PrimalLWE}_{\sigma_1}$), and lower bound ($\text{PrimalLWE}_{\sigma_1/\sqrt{2}}$).

	$\text{PrimalLWE}_{\sigma_1}$	$\text{PrimalLWE}_{\Sigma_{\text{Hints}}}$	$\text{PrimalLWE}_{\sigma_1/\sqrt{2}}$
Maul512	373	353	338
Maul768	613	589	560
Maul1024	839	811	772

Table 6 reports the leaky-LWE estimated security (bikz) for **Maul**, including an upper bound, and a lower bound, under the following models:

- $\text{PrimalLWE}_{\sigma_1}$: the estimated security of a standard (M)LWE instance with secret and error distributions of standard deviation σ_1 ; this serves as an upper bound on the security of Hint-MLWE with parameters $(\sigma_1, \sigma_2 = \sqrt{n}\sigma_1^2)$,
- $\text{PrimalLWE}_{\Sigma_{\text{Hints}}}$: the residual security after incorporating approximate hints to (M)LWE with parameter σ_1 , using the leaky-LWE estimator,
- $\text{PrimalLWE}_{\sigma_1/\sqrt{2}}$: the estimated security of a standard (M)LWE instance with standard deviation $\sigma_1/\sqrt{2}$; this models an instance that is at least as hard as the corresponding Hint-MLWE instance under the Heuristics 2-3.

Figure 9 shows that approximate hints weaken security, though less than Theorem 4 reduction suggests, indicating that more competitive Hint-MLWE parameters may be possible. Nevertheless, heuristic estimates alone are insufficient, as other potential attack vectors remain underexplored. Assessing security under non-Gaussian distributions remains an open problem. Recent work by Cao, Jia, and Chen [14] introduces a new security estimator for LWE with approximate hints. However, their model assumes that the entries of the matrix $\mathbf{H}$ are sampled uniformly over $\mathbb{Z}_q$, rather than according to Gaussian-like distributions as in our case. Moreover, their framework considers hints only on the MLWE secret, whereas our setting involves hints on both the secret and error vectors. Consequently, their results are not directly applicable to our scenario. That said, nothing suggests that their framework would lead to a better assessment of the security loss in our setting, as the approach of [14] is primarily tailored to the *too-many-hints* regime [44]—which is not the case in our work.

²⁰ <https://github.com/lucas/leaky-LWE-Estimator>

²¹ For $k = 2$, $n = 256$, and $q = 7681$, as in the **Maul512** parameter set.

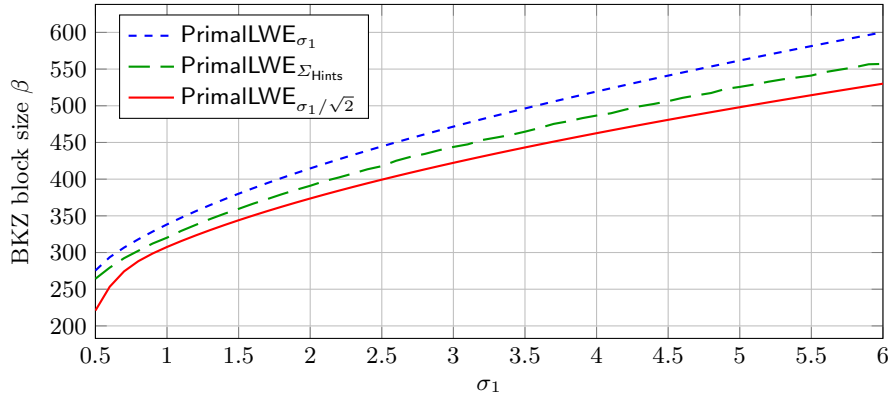


Fig. 9: Estimated security²¹ ($\text{PrimalLWE}_{\Sigma_{Hints}}$) in bikz as a function of σ_1 , including upper bound ($\text{PrimalLWE}_{\sigma_1}$), and lower bound ($\text{PrimalLWE}_{\sigma_1/\sqrt{2}}$).

References

1. Alkim, E., Ducas, L., Pöppelmann, T., Schwabe, P.: Post-quantum key exchange - A new hope. In: Holz, T., Savage, S. (eds.) *USENIX Security 2016: 25th USENIX Security Symposium*. pp. 327–343. USENIX Association, Austin, TX, USA (Aug 10–12, 2016), <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/alkim>
2. Alwen, J., Coretti, S., Dodis, Y.: The double ratchet: Security notions, proofs, and modularization for the Signal protocol. In: Ishai, Y., Rijmen, V. (eds.) *Advances in Cryptology – EUROCRYPT 2019, Part I. Lecture Notes in Computer Science*, vol. 11476, pp. 129–158. Springer, Cham, Switzerland, Darmstadt, Germany (May 19–23, 2019). https://doi.org/10.1007/978-3-030-17653-2_5
3. Ambainis, A., Hamburg, M., Unruh, D.: Quantum security proofs using semi-classical oracles. In: Boldyreva, A., Micciancio, D. (eds.) *Advances in Cryptology – CRYPTO 2019, Part II. Lecture Notes in Computer Science*, vol. 11693, pp. 269–295. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 18–22, 2019). https://doi.org/10.1007/978-3-030-26951-7_10
4. Apple Security Engineering and Architecture (SEAR): iMessage with PQ3: The new state of the art in quantum-secure messaging at scale (February 2024), <https://security.apple.com/blog/imessage-pq3>
5. Barnes, R., Bhargavan, K., Lipp, B., Wood, C.A.: Hybrid Public Key Encryption. RFC 9180 (Feb 2022). <https://doi.org/10.17487/RFC9180>, <https://www.rfc-editor.org/info/rfc9180>
6. Becker, A., Ducas, L., Gama, N., Laarhoven, T.: New directions in nearest neighbor searching with applications to lattice sieving. In: Krauthgamer, R. (ed.) *27th Annual ACM-SIAM Symposium on Discrete Algorithms*. pp. 10–24. ACM-SIAM, Arlington, VA, USA (Jan 10–12, 2016). <https://doi.org/10.1137/1.9781611974331.ch2>
7. Bellare, M., Pointcheval, D., Rogaway, P.: Authenticated key exchange secure against dictionary attacks. In: Preneel, B. (ed.) *Advances in Cryptology – EUROCRYPT 2000. Lecture Notes in Computer Science*, vol. 1807, pp. 139–155.

- Springer Berlin Heidelberg, Germany, Bruges, Belgium (May 14–18, 2000). https://doi.org/10.1007/3-540-45539-6_11
8. Bindel, N., Hamburg, M., Hövelmanns, K., Hülsing, A., Persichetti, E.: Tighter proofs of CCA security in the quantum random oracle model. In: Hofheinz, D., Rosen, A. (eds.) TCC 2019: 17th Theory of Cryptography Conference, Part II. Lecture Notes in Computer Science, vol. 11892, pp. 61–90. Springer, Cham, Switzerland, Nuremberg, Germany (Dec 1–5, 2019). https://doi.org/10.1007/978-3-030-36033-7_3
 9. Blake-Wilson, S., Menezes, A.: Unknown key-share attacks on the station-to-station (STS) protocol. In: Imai, H., Zheng, Y. (eds.) PKC'99: 2nd International Workshop on Theory and Practice in Public Key Cryptography. Lecture Notes in Computer Science, vol. 1560, pp. 154–170. Springer Berlin Heidelberg, Germany, Kamakura, Japan (Mar 1–3, 1999). https://doi.org/10.1007/3-540-49162-7_12
 10. Bos, J.W., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS - Kyber: A CCA-secure module-lattice-based KEM. In: 2018 IEEE European Symposium on Security and Privacy. pp. 353–367. IEEE Computer Society Press, London, United Kingdom (Apr 24–26, 2018). <https://doi.org/10.1109/EuroSP.2018.00032>
 11. Boyd, C., Cliff, Y., González Nieto, J., Paterson, K.G.: Efficient one-round key exchange in the standard model. In: Mu, Y., Susilo, W., Seberry, J. (eds.) ACISP 08: 13th Australasian Conference on Information Security and Privacy. Lecture Notes in Computer Science, vol. 5107, pp. 69–83. Springer Berlin Heidelberg, Germany, Wollongong, Australia (Jul 7–9, 2008). https://doi.org/10.1007/978-3-540-70500-0_6
 12. Boyd, C., de Kock, B., Millerjord, L.: Modular design of KEM-based authenticated key exchange. In: Simpson, L., Bae, M.A.R. (eds.) ACISP 23: 28th Australasian Conference on Information Security and Privacy. Lecture Notes in Computer Science, vol. 13915, pp. 553–579. Springer, Cham, Switzerland, Brisbane, QLD, Australia (Jul 5–7, 2023). https://doi.org/10.1007/978-3-031-35486-1_24
 13. Canetti, R., Krawczyk, H.: Analysis of key-exchange protocols and their use for building secure channels. In: Pfitzmann, B. (ed.) Advances in Cryptology – EUROCRYPT 2001. Lecture Notes in Computer Science, vol. 2045, pp. 453–474. Springer Berlin Heidelberg, Germany, Innsbruck, Austria (May 6–10, 2001). https://doi.org/10.1007/3-540-44987-6_28
 14. Cao, J., Jiang, H., Cheng, Q.: Refined attack on LWE with hints: Constructing lattice via gaussian elimination. In: Kalai, Y.T., Kamara, S.F. (eds.) Advances in Cryptology – CRYPTO 2025, Part I. Lecture Notes in Computer Science, vol. 16000, pp. 385–416. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 17–21, 2025). https://doi.org/10.1007/978-3-032-01855-7_13
 15. Chailloux, A., Loyer, J.: Lattice sieving via quantum random walks. In: Tibouchi, M., Wang, H. (eds.) Advances in Cryptology – ASIACRYPT 2021, Part IV. Lecture Notes in Computer Science, vol. 13093, pp. 63–91. Springer, Cham, Switzerland, Singapore (Dec 6–10, 2021). https://doi.org/10.1007/978-3-030-92068-5_3
 16. Cremers, C., Düzl , S., Fiedler, R., Fischlin, M., Janson, C.: BUFFing signature schemes beyond unforgeability and the case of post-quantum signatures. In: 2021 IEEE Symposium on Security and Privacy. pp. 1696–1714. IEEE Computer Society Press, San Francisco, CA, USA (May 24–27, 2021). <https://doi.org/10.1109/SP40001.2021.00093>

17. Cremers, C.J.F.: Session-state reveal is stronger than ephemeral key reveal: Attacking the NAXOS authenticated key exchange protocol. In: Abdalla, M., Pointcheval, D., Fouque, P.A., Vergnaud, D. (eds.) ACNS 2009: 7th International Conference on Applied Cryptography and Network Security. Lecture Notes in Computer Science, vol. 5536, pp. 20–33. Springer Berlin Heidelberg, Germany, Paris-Rocquencourt, France (Jun 2–5, 2009). https://doi.org/10.1007/978-3-642-01957-9_2
18. Cremers, C.J.F., Feltz, M.: Beyond eCK: Perfect forward secrecy under actor compromise and ephemeral-key reveal. In: Foresti, S., Yung, M., Martinelli, F. (eds.) ESORICS 2012: 17th European Symposium on Research in Computer Security. Lecture Notes in Computer Science, vol. 7459, pp. 734–751. Springer Berlin Heidelberg, Germany, Pisa, Italy (Sep 10–12, 2012). https://doi.org/10.1007/978-3-642-33167-1_42
19. Dachman-Soled, D., Ducas, L., Gong, H., Rossi, M.: LWE with side information: Attacks and concrete security estimation. In: Micciancio, D., Ristenpart, T. (eds.) Advances in Cryptology – CRYPTO 2020, Part II. Lecture Notes in Computer Science, vol. 12171, pp. 329–358. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 17–21, 2020). https://doi.org/10.1007/978-3-030-56880-1_12
20. D’Anvers, J.P., Batsleer, S.: Multitarget decryption failure attacks and their application to Saber and Kyber. In: Hanaoka, G., Shikata, J., Watanabe, Y. (eds.) PKC 2022: 25th International Conference on Theory and Practice of Public Key Cryptography, Part I. Lecture Notes in Computer Science, vol. 13177, pp. 3–33. Springer, Cham, Switzerland, Virtual Event (Mar 8–11, 2022). https://doi.org/10.1007/978-3-030-97121-2_1
21. Dodis, Y., Fiore, D.: Unilaterally-authenticated key exchange. In: Kiayias, A. (ed.) FC 2017: 21st International Conference on Financial Cryptography and Data Security. Lecture Notes in Computer Science, vol. 10322, pp. 542–560. Springer, Cham, Switzerland, Sliema, Malta (Apr 3–7, 2017). https://doi.org/10.1007/978-3-319-70972-7_31
22. Dodis, Y., Jost, D., Katsumata, S., Prest, T., Schmidt, R.: Triple ratchet: A bandwidth efficient hybrid-secure signal protocol. In: Fehr, S., Fouque, P.A. (eds.) Advances in Cryptology – EUROCRYPT 2025, Part VIII. Lecture Notes in Computer Science, vol. 15608, pp. 302–331. Springer, Cham, Switzerland, Madrid, Spain (May 4–8, 2025). https://doi.org/10.1007/978-3-031-91101-9_11
23. Dowling, B., Hale, B.: Authenticated continuous key agreement: Active MitM detection and prevention. Cryptology ePrint Archive, Report 2023/228 (2023), <https://eprint.iacr.org/2023/228>
24. Duman, J., Hövelmanns, K., Kiltz, E., Lyubashevsky, V., Seiler, G.: Faster lattice-based KEMs via a generic Fujisaki-Okamoto transform using prefix hashing. In: Vigna, G., Shi, E. (eds.) ACM CCS 2021: 28th Conference on Computer and Communications Security. pp. 2722–2737. ACM Press, Virtual Event, Republic of Korea (Nov 15–19, 2021). <https://doi.org/10.1145/3460120.3484819>
25. Esgin, M.F., Espitau, T., Niot, G., Prest, T., Sakzad, A., Steinfeld, R.: Plover: Masking-friendly hash-and-sign lattice signatures. In: Joye, M., Leander, G. (eds.) Advances in Cryptology – EUROCRYPT 2024, Part VII. Lecture Notes in Computer Science, vol. 14657, pp. 316–345. Springer, Cham, Switzerland, Zurich, Switzerland (May 26–30, 2024). https://doi.org/10.1007/978-3-031-58754-2_12
26. Fiore, D., Gennaro, R., Smart, N.P.: Constructing certificateless encryption and ID-based encryption from ID-based key agreement. In: Joye, M., Miyaji, A., Otsuka, A. (eds.) PAIRING 2010: 4th International Conference on Pairing-based Cryptography. Lecture Notes in Computer Science, vol. 6072, pp. 1–15. Springer, Berlin Heidelberg, Germany, Paris-Rocquencourt, France (Jun 1–3, 2010). https://doi.org/10.1007/978-3-642-12813-2_1

- tography. Lecture Notes in Computer Science, vol. 6487, pp. 167–186. Springer Berlin Heidelberg, Germany, Yamanaka Hot Spring, Japan (Dec 13–15, 2010). https://doi.org/10.1007/978-3-642-17455-1_11
27. Fouque, P.A., Hoffstein, J., Kirchner, P., Lyubashevsky, V., Pornin, T., Prest, T., Ricosset, T., Seiler, G., Whyte, W., Zhang, Z.: Falcon: Fast-Fourier lattice-based compact signatures over NTRU. Submission to the NIST’s post-quantum cryptography standardization process (2020)
 28. Fujioka, A., Suzuki, K., Xagawa, K., Yoneyama, K.: Strongly secure authenticated key exchange from factoring, codes, and lattices. In: Fischlin, M., Buchmann, J., Manulis, M. (eds.) PKC 2012: 15th International Conference on Theory and Practice of Public Key Cryptography. Lecture Notes in Computer Science, vol. 7293, pp. 467–484. Springer Berlin Heidelberg, Germany, Darmstadt, Germany (May 21–23, 2012). https://doi.org/10.1007/978-3-642-30057-8_28
 29. Fujioka, A., Suzuki, K., Xagawa, K., Yoneyama, K.: Strongly secure authenticated key exchange from factoring, codes, and lattices. *Designs, Codes and Cryptography* **76**(3), 469–504 (2015). <https://doi.org/10.1007/s10623-014-9972-2>
 30. Gaborit, P., Aguilar-Melchor, C., Aragon, N., Bettaieb, S., Bidoux, L., Blazy, O., Deneuville, J.C., Persichetti, E., Zémor, G., Bos, J., Dion, A., Lacan, J., Robert, J.M., Véron, P., Barreto, P.L., Ghosh, S., Gueron, S., Güneysu, T., Misoczki, R., Richter-Brokmann, J., Sendrier, N., Tillich, J.P., Vasseur, V.: Hamming Quasi-Cyclic (HQC). Submission to the NIST’s post-quantum cryptography standardization process (2025)
 31. Goldberg, I., Stebila, D., Ustaoglu, B.: Anonymity and one-way authentication in key exchange protocols. *Designs, Codes and Cryptography* **67**(2), 245–269 (2013). <https://doi.org/10.1007/s10623-011-9604-z>
 32. Hashimoto, K., Katsumata, S., Niot, G., Wiggers, T.: Revisiting PQ WireGuard: A comprehensive security analysis with a new design using Reinforced KEMs p. To Appear (2026), <https://eprint.iacr.org/2025/1758>
 33. Hövelmanns, K., Kiltz, E., Schäge, S., Unruh, D.: Generic authenticated key exchange in the quantum random oracle model. In: Kiayias, A., Kohlweiss, M., Wallden, P., Zikas, V. (eds.) PKC 2020: 23rd International Conference on Theory and Practice of Public Key Cryptography, Part II. Lecture Notes in Computer Science, vol. 12111, pp. 389–422. Springer, Cham, Switzerland, Edinburgh, UK (May 4–7, 2020). https://doi.org/10.1007/978-3-030-45388-6_14
 34. Hülsing, A., Ning, K.C., Schwabe, P., Weber, F.J., Zimmermann, P.R.: Post-quantum WireGuard. In: 2021 IEEE Symposium on Security and Privacy. pp. 304–321. IEEE Computer Society Press, San Francisco, CA, USA (May 24–27, 2021). <https://doi.org/10.1109/SP40001.2021.00030>
 35. Jacobs, F.: Invited talk: Designing imessage pq3: Quantum-secure messaging at scale. Talk given at RWC 2024 (2024), video at <https://www.youtube.com/watch?v=RVbHElGe518>
 36. Jager, T., Kohlar, F., Schäge, S., Schwenk, J.: On the security of TLS-DHE in the standard model. *Cryptology ePrint Archive*, Report 2011/219 (2011), <https://eprint.iacr.org/2011/219>
 37. Katsumata, S., Kwiakowski, K., Pintore, F., Prest, T.: Scalable ciphertext compression techniques for post-quantum KEMs and their applications. In: Moriai, S., Wang, H. (eds.) *Advances in Cryptology – ASIACRYPT 2020*, Part I. Lecture Notes in Computer Science, vol. 12491, pp. 289–320. Springer, Cham, Switzerland, Daejeon, South Korea (Dec 7–11, 2020). https://doi.org/10.1007/978-3-030-64837-4_10

38. Kim, D., Lee, D., Seo, J., Song, Y.: Toward practical lattice-based proof of knowledge from hint-MLWE. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology – CRYPTO 2023, Part V. Lecture Notes in Computer Science*, vol. 14085, pp. 549–580. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 20–24, 2023). https://doi.org/10.1007/978-3-031-38554-4_18
39. Krawczyk, H.: HMQV: A high-performance secure Diffie-Hellman protocol. In: Shoup, V. (ed.) *Advances in Cryptology – CRYPTO 2005. Lecture Notes in Computer Science*, vol. 3621, pp. 546–566. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 14–18, 2005). https://doi.org/10.1007/11535218_33
40. Krawczyk, H., Paterson, K.G., Wee, H.: On the security of the TLS protocol: A systematic analysis. In: Canetti, R., Garay, J.A. (eds.) *Advances in Cryptology – CRYPTO 2013, Part I. Lecture Notes in Computer Science*, vol. 8042, pp. 429–448. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 18–22, 2013). https://doi.org/10.1007/978-3-642-40041-4_24
41. LaMacchia, B.A., Lauter, K., Mityagin, A.: Stronger security of authenticated key exchange. In: Susilo, W., Liu, J.K., Mu, Y. (eds.) *ProvSec 2007: 1st International Conference on Provable Security. Lecture Notes in Computer Science*, vol. 4784, pp. 1–16. Springer Berlin Heidelberg, Germany, Wollongong, Australia (Nov 1–2, 2007). https://doi.org/10.1007/978-3-540-75670-5_1
42. Langlois, A., Stehlé, D.: Worst-case to average-case reductions for module lattices. *Des. Codes Cryptogr.* **75**(3), 565–599 (2015). <https://doi.org/10.1007/S10623-014-9938-4>, <https://doi.org/10.1007/s10623-014-9938-4>
43. Lyu, Y., Liu, S.: Two-message authenticated key exchange from public-key encryption. In: Tsudik, G., Conti, M., Liang, K., Smaragdakis, G. (eds.) *ESORICS 2023: 28th European Symposium on Research in Computer Security, Part I. Lecture Notes in Computer Science*, vol. 14344, pp. 414–434. Springer, Cham, Switzerland, The Hague, The Netherlands (Sep 25–29, 2023). https://doi.org/10.1007/978-3-031-50594-2_21
44. May, A., Nowakowski, J.: Too many hints - when LLL breaks LWE. In: Guo, J., Steinfeld, R. (eds.) *Advances in Cryptology – ASIACRYPT 2023, Part IV. Lecture Notes in Computer Science*, vol. 14441, pp. 106–137. Springer, Singapore, Singapore, Guangzhou, China (Dec 4–8, 2023). https://doi.org/10.1007/978-981-99-8730-6_4
45. Naehrig, M., Alkim, E., Bos, J., Ducas, L., Easterbrook, K., LaMacchia, B., Longa, P., Mironov, I., Nikolaenko, V., Peikert, C., Raghunathan, A., Stebila, D.: FrodoKEM. Tech. rep., National Institute of Standards and Technology (2020), available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>
46. Peikert, C.: An efficient and parallel Gaussian sampler for lattices. In: Rabin, T. (ed.) *Advances in Cryptology – CRYPTO 2010. Lecture Notes in Computer Science*, vol. 6223, pp. 80–97. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 15–19, 2010). https://doi.org/10.1007/978-3-642-14623-7_5
47. Prest, T.: Sharper bounds in lattice-based cryptography using the Rényi divergence. In: Takagi, T., Peyrin, T. (eds.) *Advances in Cryptology – ASIACRYPT 2017, Part I. Lecture Notes in Computer Science*, vol. 10624, pp. 347–374. Springer, Cham, Switzerland, Hong Kong, China (Dec 3–7, 2017). https://doi.org/10.1007/978-3-319-70694-8_13
48. Rescorla, E.: The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446 (Aug 2018). <https://doi.org/10.17487/RFC8446>, <https://www.rfc-editor.org/info/rfc8446>

49. Rogaway, P.: Formalizing human ignorance. In: Nguyen, P.Q. (ed.) Progress in Cryptology - VIETCRYPT 06: 1st International Conference on Cryptology in Vietnam. Lecture Notes in Computer Science, vol. 4341, pp. 211–228. Springer Berlin Heidelberg, Germany, Hanoi, Vietnam (Sep 25–28, 2006). https://doi.org/10.1007/11958239_14
50. Schwabe, P., Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Seiler, G., Stehlé, D., Ding, J.: CRYSTALS-KYBER. Tech. rep., National Institute of Standards and Technology (2022), available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>
51. Schwabe, P., Stebila, D., Wiggers, T.: Post-quantum TLS without handshake signatures. In: Ligatti, J., Ou, X., Katz, J., Vigna, G. (eds.) ACM CCS 2020: 27th Conference on Computer and Communications Security. pp. 1461–1480. ACM Press, Virtual Event, USA (Nov 9–13, 2020). <https://doi.org/10.1145/3372297.3423350>
52. Schwabe, P., Stebila, D., Wiggers, T.: More efficient post-quantum KEMTLS with pre-distributed public keys. In: Bertino, E., Shulman, H., Waidner, M. (eds.) ESORICS 2021: 26th European Symposium on Research in Computer Security, Part I. Lecture Notes in Computer Science, vol. 12972, pp. 3–22. Springer, Cham, Switzerland, Darmstadt, Germany (Oct 4–8, 2021). https://doi.org/10.1007/978-3-030-88418-5_1
53. Shoup, V.: On formal models for secure key exchange. Cryptology ePrint Archive, Report 1999/012 (1999), <https://eprint.iacr.org/1999/012>
54. of Standards, N.I., Technology: Fips 203: Module-lattice-based key-encapsulation mechanism standard, <https://csrc.nist.gov/pubs/fips/203/final>
55. of Standards, N.I., Technology: Fips 204: Module-lattice-based digital signature standard, <https://csrc.nist.gov/pubs/fips/204/final>
56. of Standards, N.I., Technology: Fips 205: Stateless hash-based digital signature standard, <https://csrc.nist.gov/pubs/fips/205/final>
57. Wiggers, T., Celi, S., Schwabe, P., Stebila, D., Sullivan, N.: KEM-based Authentication for TLS 1.3. Internet-Draft draft-celi-wiggers-tls-authkem-05, Internet Engineering Task Force (Apr 2025), <https://datatracker.ietf.org/doc/draft-celi-wiggers-tls-authkem/05/>, work in Progress
58. Wiggers, T., Celi, S., Schwabe, P., Stebila, D., Sullivan, N.: KEM-based Authentication for TLS 1.3. Internet-Draft draft-celi-wiggers-tls-authkem-05, Internet Engineering Task Force (Apr 2025), <https://datatracker.ietf.org/doc/draft-celi-wiggers-tls-authkem/05/>, work in Progress
59. Xue, H., Lu, X., Li, B., Liang, B., He, J.: Understanding and constructing AKE via double-key key encapsulation mechanism. In: Peyrin, T., Galbraith, S. (eds.) Advances in Cryptology – ASIACRYPT 2018, Part II. Lecture Notes in Computer Science, vol. 11273, pp. 158–189. Springer, Cham, Switzerland, Brisbane, Queensland, Australia (Dec 2–6, 2018). https://doi.org/10.1007/978-3-030-03329-3_6

A Additional Preliminaries

Additional Notations We denote by ϵ the empty string in $\{0,1\}^*$. For any non-zero integer $i \in \mathbb{N}$, we define $\text{Range}(i) := [0, i-1] \cap \mathbb{Z}$.

Public Key Encryption. A public key encryption (PKE) scheme (**Setup**, **Keygen**, **Enc**, **Dec**) consists of four probabilistic polynomial-time (PPT) algorithms.

The $\text{Setup}()$ algorithm generates public parameters pp . Using these parameters, $\text{Keygen}(\text{pp})$ produces a pair of public and secret keys (pk, sk) . The $\text{Enc}(\text{pp}, \text{pk}, m)$ algorithm encrypts a message m from the message space $\mathcal{M}$ into a ciphertext ct , while $\text{Dec}(\text{pp}, \text{sk}, \text{ct})$ decrypts the ciphertext ct back into the message m . The IND-CPA security of a PKE scheme is defined through the following game, where the adversary's advantage is given by $|\Pr[b = b'] - 1/2|$.

IND-CPA
$\text{pp} \leftarrow \text{Setup}()$ $(\text{pk}, \text{sk}) \leftarrow \text{Keygen}(\text{pp})$ $(\text{state}, m_0, m_1) \leftarrow \mathcal{A}_1(\text{pp}, \text{pk})$ $\text{ct}_0 \leftarrow \text{Enc}(\text{pp}, \text{pk}, m_0)$ $\text{ct}_1 \leftarrow \text{Enc}(\text{pp}, \text{pk}, m_1)$ $b \leftarrow \{0, 1\}$ $b' \leftarrow \mathcal{A}_2(\text{state}, \text{ct}_b)$ return $b = b'$

Key Encapsulation Mechanism. A key encapsulation mechanism (KEM) scheme (Setup , Keygen , Encaps , Decaps) consists of four probabilistic polynomial-time (PPT) algorithms. The $\text{Setup}()$ algorithm generates public parameters pp . Using these parameters, $\text{Keygen}(\text{pp})$ produces a pair of public and secret keys (pk, sk) . The $\text{Encaps}(\text{pp}, \text{pk})$ algorithm creates a couple (ct, k) : k is a key belonging to a key space Keys and ct is its encapsulation. $\text{Dec}(\text{pp}, \text{sk}, \text{ct})$ decrypts ct back into key k . The IND-CCA security of a KEM scheme is defined through the following game, where the adversary's advantage is given by $|\Pr[b = b'] - 1/2|$.

IND-CCA($\mathcal{A}$)	$\mathcal{O}_{\text{CCA}}(\text{ct}')$
$b \leftarrow \{0, 1\}$ $\text{pp} \leftarrow \text{Setup}()$ $(\text{pk}, \text{sk}) \leftarrow \text{Keygen}(\text{pp})$ $k_0 \leftarrow \text{Keys}$ $(\text{ct}, k_1) \leftarrow \text{Encaps}(\text{pp}, \text{pk})$ $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{CCA}}}(\text{pp}, \text{pk}, \text{ct}, k_b)$ return $b = b'$	if $\text{ct}' \neq \text{ct}$ then $k' \leftarrow \text{Decaps}(\text{pp}, \text{sk}, \text{ct}')$ return k' return $\perp$

For $\delta \geq 0$, a KEM scheme is said δ -correct if for $\text{pp} \leftarrow \text{Setup}()$, $(\text{pk}, \text{sk}) \leftarrow \text{Keygen}(\text{pp})$, $(\text{ct}, k) \leftarrow \text{Encaps}(\text{pp}, \text{pk})$, the probability to have $\text{Decaps}(\text{pp}, \text{sk}, \text{ct}) = k$ is more than $1 - \delta$.

We also consider the *key-recovery game* Key-Recov . First, we suppose the existence of a function RecovPKL , such that, for any $\text{pp} \leftarrow \text{Setup}()$ and $(\text{pk}, \text{sk}) \leftarrow$

$\text{Keygen}()$, we have $\text{pk} = \text{RecovPK}(\text{pp}, \text{sk})$. The advantage of the game **Key-Recov** is defined as:

$$\Pr \left[\text{pk} = \text{RecovPK}(\text{pp}, \text{sk}') : \begin{array}{l} \text{pp} \leftarrow \text{Setup}(), (\text{pk}, \text{sk}) \leftarrow \text{Keygen}(\text{pp}), \\ \text{sk} \leftarrow \mathcal{A}(\text{pp}, \text{pk}) \end{array} \right].$$

Signature. A signature scheme $(\text{Setup}, \text{Keygen}, \text{Sign}, \text{SignVf})$ consists of four probabilistic polynomial-time (PPT) algorithms. The $\text{Setup}()$ algorithm generates public parameters pp . Using these parameters, $\text{Keygen}(\text{pp})$ produces a pair of public and secret keys (pk, sk) . The $\text{Sign}(\text{sk}, \text{m})$ create a signature σ of a message m belonging to the message space $\mathcal{M}$. $\text{SignVf}(\text{pp}, \text{pk}, \text{m}, \sigma)$ returns 1 (the signature is valid) or 0 (the signature is invalid).

The *strong unforgeability under chosen message attack* (sUF-CMA) security of a signature scheme is defined through the game sUF-CMA of Figure 10, where the adversary's advantage is given by the probability $\Pr[\text{sUF-CMA}(\mathcal{A}) = 1]$.

sUF-CMA($\mathcal{A}$)	MBS($\mathcal{A}$)
$Q \leftarrow \emptyset$	$\text{pp} \leftarrow \text{Setup}()$
$\text{pp} \leftarrow \text{Setup}()$	$(\text{m}_1, \text{m}_2, \sigma, \text{pk}) \leftarrow \mathcal{A}(\text{pp})$
$(\text{pk}, \text{sk}) \leftarrow \text{Keygen}(\text{pp})$	if $\text{m}_1 \neq \text{m}_2$
$(\text{m}', \sigma') \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sign}}}(\text{pp}, \text{pk})$	$\wedge \text{SignVf}(\text{pp}, \text{pk}, \text{m}_1, \sigma) = 1$
if $(\text{m}', \sigma') \notin Q$ then	$\wedge \text{SignVf}(\text{pp}, \text{pk}, \text{m}_2, \sigma) = 1$ then
return $\text{SignVf}(\text{pp}, \text{pk}, \text{m}, \sigma)$	return 1
return 0	return 0
$\mathcal{O}_{\text{Sign}}(\text{m})$	M-S-UEO($\mathcal{A}$)
$\text{m} \leftarrow \text{Sign}(\text{sk}, \text{m})$	$\text{pp} \leftarrow \text{Setup}()$
$Q \leftarrow Q \cup \{(\text{m}, \sigma)\}$	$(\text{m}_1, \text{m}_2, \sigma, \text{pk}_1, \text{pk}_2) \leftarrow \mathcal{A}(\text{pp})$
return σ	if $\text{pk}_1 \neq \text{pk}_2$
	$\wedge \text{SignVf}(\text{pp}, \text{pk}_1, \text{m}_1, \sigma) = 1$
	$\wedge \text{SignVf}(\text{pp}, \text{pk}_2, \text{m}_2, \sigma) = 1$ then
	return 1
	return 0

Fig. 10: Security games of a signature scheme.

The key-recovery game **Key-Recov** of a signature scheme is defined identically to the recovery game of a KEM. The *message-bound signature* (MBS) security of a signature scheme, explained in [16], is defined through the game MBS of Figure 10, where the adversary's advantage is given by the probability

$\Pr[\text{MBS}(\mathcal{A}) = 1]$. The *malicious-strong universal exclusive ownership* (M-S-UEO) security of a signature scheme, explained in [16], is defined through the game M-S-UEO of Figure 10, where the adversary's advantage is given by the probability $\Pr[\text{M-S-UEO}(\mathcal{A}) = 1]$.

Message Authentication Code. A message authentication code (MAC) scheme $(\text{Mac}, \text{MacVf})$ consists of two PPT algorithms. The $\text{Mac}(k, m)$ algorithm takes a key from a key space SetKeysMAC and a message $m \in \{0, 1\}^*$ to generate a tag tag . The algorithm $\text{MacVf}(k, \text{tag}, m)$ takes as input a key, a tag, and a message, returning a boolean value. For all messages m and keys k , we require that $\text{MacVf}(k, \text{Mac}(k, m), m)$ always evaluates to true.

The strong unforgeability security of a MAC scheme against an adversary $\mathcal{A}$ is evaluated by the probability that $\mathcal{A}$ wins the game sUnforg , defined in Figure 11.

$\text{sUnforg}(\mathcal{A})$	$\mathcal{O}_{\text{Mac}}(m)$
$Q \leftarrow \emptyset$	$\text{tag} \leftarrow \text{MAC}(k, m)$
$k \leftarrow \$ \text{Keys}$	$Q \leftarrow Q \cup \{(m, \text{tag})\}$
$(m', \text{tag}') \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Mac}}}()$	return tag
if $(m', \text{tag}') \notin Q$ then	
return $\text{MacVf}(k, \text{tag}', m')$	
return 0	

Fig. 11: Strong forgeability security game of a MAC scheme.

We also consider another security notion, *message-key bound* security—inspired by the signature security notions of [16]—that can be ensured when $\text{MAC}(k, m)$ is given by $H((k, m))$ for any collision-resistant hash function H .

The advantage of an adversary $\mathcal{A}$ against this game is defined as:

$$\Pr \left[\begin{array}{l} (k_1, m_1) \neq (k_2, m_2) \\ \wedge \text{MacVf}(k_1, \text{tag}, m_1) = 1 : (m_1, m_2, k_1, k_2, \text{tag}) \leftarrow \mathcal{A}() \\ \wedge \text{MacVf}(k_2, \text{tag}, m_2) = 1 \end{array} \right].$$

Note that, using an argument similar to the one given in [49] for KDFs, a MAC cannot claim MacColl security against all efficient adversaries, since the mere existence of a collision (even if the colliding values are not yet known) implies the existence of an efficient adversary.

To capture a stronger notion secure against all efficient adversaries, one may employ a prefixed variant of this property. This game relies on an external source

AuxInput and the advantage of an adversary $\mathcal{A}$ against it is given by:

$$\Pr \left[\begin{array}{l} (k_1, m_1) \neq (k_2, m_2) \\ \wedge \text{MacVf}(k_1, \text{tag}, (\text{in}, m_1)) = 1 : \text{in} \leftarrow \text{AuxInput} \\ \wedge \text{MacVf}(k_2, \text{tag}, (\text{in}, m_2)) = 1 : (m_1, m_2, k_1, k_2, \text{tag}) \leftarrow \mathcal{A}(\text{in}) \end{array} \right].$$

To apply the prefixed variant to DAKE and its variants, the external source can be taken as the public parameters of the AKE. In this case, the public parameters must be prefixed to the message used in the computation of m_{tag}^R .

Pseudo-Random Generator. A pseudorandom generator (PRG) is a function $\text{PRG} : \mathcal{K}_{\text{PRG}} \times \text{In}_{\text{PRG}} \rightarrow \text{Out}_{\text{PRG}}$, where the security against an adversary $\mathcal{A}$ is quantified by the advantage:

$$\text{Adv}^{\text{IND-PRG}}(\mathcal{A}) = \left| \Pr [b = b' \mid b' \leftarrow \mathcal{A}(\text{out}_b); \text{out}_1 = \text{PRG}(k); \right. \\ \left. (k, \text{out}_0) \leftarrow_{\$} \mathcal{K}_{\text{PRG}} \times \text{Out}_{\text{PRG}}; b \leftarrow_{\$} \{0, 1\}] - 1/2 \right|.$$

Key Derivation Function. A key derivation function (KDF) is a function $\text{KDF} : \mathcal{K}_{\text{KDF}} \times \{0, 1\}^* \rightarrow \text{Out}_{\text{KDF}}$. Its security is defined through the following game, where the adversary's advantage is given by $|\Pr[b = b'] - 1/2|$.

IND-KDF
$(m, \text{state}) \leftarrow \mathcal{A}_1()$ $(k, \text{out}_0) \leftarrow_{\$} \mathcal{K}_{\text{KDF}} \times \text{Out}_{\text{KDF}}$ $\text{out}_1 = \text{KDF}(k, m)$ $b \leftarrow_{\$} \{0, 1\}$ $b' \leftarrow \mathcal{A}_2(\text{state}, \text{out}_b)$ return $b = b'$

We also consider the collision-resistance security of a KDF. The advantage of an adversary $\mathcal{A}$ against the collision-resistance experiment of KDF is defined as

$$\Pr[(k, m) \neq (k', m') \wedge \text{KDF}(k, m) = \text{KDF}(k', m') : (k, k', m, m') \leftarrow \mathcal{A}()].$$

Note that, as with a MAC and for the same reason, a KDF cannot be secure against all efficient adversaries, since the mere existence of a collision already implies the existence of such an adversary.

Similarly, to capture a stronger notion that remains secure against all efficient adversaries, one may employ a prefixed variant of this property. This game relies on an external source **AuxInput** and its advantage against an adversary $\mathcal{A}$ is given by

$$\Pr \left[(k, m) \neq (k', m') \wedge \text{KDF}(k, (\text{in}, m)) = \text{KDF}(k', (\text{in}, m')) : \begin{array}{l} \text{in} \leftarrow \text{AuxInput}(), \\ (k, k', m, m') \leftarrow \mathcal{A}() \end{array} \right].$$

When applying the prefixed variant to DAKE and its variants, the external source may be taken as the public parameters of the AKE. In this case, ensuring inverse-match soundness requires prefixing the public parameters to the message transcript being added.

Hash Function. A hash function is a function $H : \{0, 1\}^* \rightarrow \text{SetHashs}$. The advantage of an adversary $\mathcal{A}$ against the collision-resistance experiment HashColl of H is defined as $\Pr[m \neq m' \wedge H(m) = H(m') : (m, m') \leftarrow \mathcal{A}()]$.

An analogous observation to the one above on collisions in KDF functions applies to collisions in hash functions, and a prefixed variant can likewise be defined.

B Proof of the CK-FO Transform

We first recall and introduce some notations that will be commonly used in the proofs for the CK-FO (Section 4.2) in both the ROM and QROM.

The PKE schemes PKE_L and PKE_R are subject to the conditions described in Section 4.1. In particular, they share the same setup algorithm and satisfy the following two properties:

- The randomness spaces are structured as $\text{Rand}_C \times \text{Rand}_L$ for Encr_L and $\text{Rand}_C \times \text{Rand}_R$ for Encr_R , where Rand_C denotes the common part. We define $\text{Rand}_{CLR} = \text{Rand}_C \times \text{Rand}_L \times \text{Rand}_R$.
- There exists a deterministic algorithm E_{base} that, given (pp, r_C) with $r_C \in \text{Rand}_C$, outputs c_C such that both Encr_L and Encr_R produce ciphertexts whose first component equals $c_C = E_{\text{base}}(\text{pp}, r_C)$. We denote by Ciphers_C the set of all such values c_C . Moreover, we denote by Ciphers_L and Ciphers_R the sets such that the ciphertext spaces of Encr_L and Encr_R are respectively structured as $\text{Ciphers}_C \times \text{Ciphers}_L$ and $\text{Ciphers}_C \times \text{Ciphers}_R$.

We denote Enc_{OTP} and Dec_{OTP} as the following algorithms:

$$\begin{array}{ll}
 \text{Enc}_{\text{OTP}}(\text{pp}, \text{pk}_L, \text{pk}_R, m_L, m_R) & \text{Dec}_{\text{OTP}}(\text{pp}, \text{sk}_L, \text{sk}_R, (c_C, c_L, c_R^{\text{OTP}})) \\
 (r_L, r_R, r_C) \leftarrow H_{\text{rand}}(\text{pk}_L, \text{pk}_R, m_L, m_R) & m_L \leftarrow \text{Decr}_L(\text{pp}, \text{sk}_L, (c_C, c_L)) \\
 (c_C, c_L) \leftarrow \text{Encr}_L(\text{pp}, \text{pk}_L, m_L; r_C, r_L) & \text{if } m_L = \perp \text{ then return } (\perp, \perp) \\
 (c_C, c_R) \leftarrow \text{Encr}_R(\text{pp}, \text{pk}_R, m_R; r_C, r_R) & c_R = c_R^{\text{OTP}} \oplus H_{\text{OTP}}(m_L) \\
 c_R^{\text{OTP}} = c_R \oplus H_{\text{OTP}}(m_L) & m_R \leftarrow \text{Decr}_R(\text{pp}, \text{sk}_R, (c_C, c_R)) \\
 \text{return } (c_C, c_L, c_R^{\text{OTP}}) & \text{if } m_R = \perp \text{ then return } (\perp, \perp) \\
 & \text{return } (m_L, m_R)
 \end{array}$$

Moreover, if we explicitly indicate some elements (r_C, r_L, r_R) as randomness, such as in $\text{Enc}_{\text{OTP}}(\text{pp}, \text{pk}_L, \text{pk}_R, m_L, m_R; r_C, r_L, r_R)$, the elements (r_C, r_L, r_R) will be used instead of $H_{\text{rand}}(\text{pk}_L, \text{pk}_R, m_L, m_R)$.

Lastly, we will also suppose that Ciphers_R is included in a binary set $\{0, 1\}^\tau$ and choose H_{OTP} to have range $\{0, 1\}^\tau$.

[IND-CK-CCA, -] and [IND-CCA, -]	$\mathcal{O}_{\text{CCA}_L}(\text{pk}_R, \text{ct} = (c_C, c_L, c_R^{\text{OTP}}))$
1 : $\text{pp} \leftarrow \text{Setup}()$ 2 : $(\text{pk}_{L,*}, \text{sk}_{L,*}^{\text{FO}} = (\text{sk}_{L,*}, \text{sl}_{L,*})) \leftarrow \text{dKeygen}_L^{\text{FO}}(\text{pp})$ 3 : $\text{LeakedR} = \emptyset$ 4 : $\text{Oracles} = \{\mathcal{O}_{\text{CCA}_L}, \mathcal{O}_{\text{leakR}}, \text{H}_{\text{rej}}, \text{H}_{\text{rand}}, \text{H}_{\text{key}}, \text{H}_{\text{OTP}}\}$ 5 : $(\text{state}, \text{pk}_{R,*}) \leftarrow \mathcal{A}_1^{\text{Oracles}}(\text{pk}_{L,*})$ 6 : $b \leftarrow \{0, 1\}$ 7 : $(\text{ct}_*, k_0) \leftarrow \text{dEncaps}^{\text{FO}}(\text{pp}, \text{pk}_{L,*}, \text{pk}_{R,*})$ 8 : We parse ct_* as $(c_{C,*}, c_{L,*}, c_{R,*})$ 9 : We denote by $(m_{L,*}, m_{R,*})$ and by 10 : $(r_{C,*}, r_{L,*}, r_{R,*})$ the messages and 11 : randomnesses used to produce 12 : this encapsulation. 13 : $k_1 \leftarrow \text{Keys}$ 14 : $b' \leftarrow \mathcal{A}_2^{\text{Oracles}}(\text{state}, \text{ct}_*, k_b)$ 15 : return $b = b'$	39 : // with $\text{ct}_*, \text{LeakedR}$ and $\text{sk}_{L,*}$ 40 : // taken from the security game 41 : if $(\exists \text{sk}_R^{\text{FO}} = (\text{sk}_R, \text{sr}_*), r : (\text{pk}_R, \text{sk}_R^{\text{FO}}, r) \in \text{LeakedR})$ 42 : $\wedge (\text{ct} \neq \text{ct}_* \vee \text{pk}_R \neq \text{pk}_{R,*})$ then 43 : $\tilde{k} \leftarrow \text{H}_{\text{rej}}(\text{pk}_L, \text{pk}_R, \text{sl}_L, \text{sr}_R, c_C, c_L, c_R^{\text{OTP}})$ / G_0 44 : $\tilde{k} \leftarrow \text{H}_{\text{rej}}(\text{pk}_L, \text{pk}_R, \text{sr}_R, c_C, c_L, c_R^{\text{OTP}})$ / $G_1 - G_3$ 45 : return $\tilde{k}$ / $G_2 - G_3$ 46 : $(m_L, m_R) \leftarrow \text{Dec}_{\text{OTP}}(\text{pp}, \text{sk}_L, \text{sk}_R, c_C, c_L, c_R^{\text{OTP}})$ / $G_0 - G_1$ 47 : if $(m_L, m_R) = (\perp, \perp)$ then / $G_0 - G_1$ 48 : return $\tilde{k}$ / $G_0 - G_1$ 49 : if $\text{Enc}_{\text{OTP}}(\text{pp}, \text{pk}_L, \text{pk}_R, m_L, m_R)$ 50 : $\neq (c_C, c_L, c_R^{\text{OTP}})$ then / $G_0 - G_1$ 51 : return $\tilde{k}$ / $G_0 - G_1$ 52 : return $\text{H}_{\text{key}}(\text{pk}_L, \text{pk}_R, m_L, m_R, c_C, c_L, c_R^{\text{OTP}})$ / $G_0 - G_1$
$\mathcal{O}_{\text{leakR}}()$ 51 : // with LeakedR taken from the security game 52 : $r \leftarrow \text{RandKeygenR}$ 53 : $(\text{pk}_R, \text{sk}_R^{\text{FO}} = (\text{sk}_R, \text{sr}_*)) \leftarrow \text{dKeygen}_R^{\text{FO}}(r)$ 54 : $\text{LeakedR} \leftarrow \text{LeakedR} \cup \{(\text{pk}_R, \text{sk}_R^{\text{FO}}, r)\}$ 55 : $\text{LeakedpkR} \leftarrow \text{LeakedpkR} \cup \{\text{pk}_R\}$ 56 : return $(\text{pk}_R, \text{sk}_R^{\text{FO}}, r)$	$\text{H}_{\text{key}}(\text{pk}_L, \text{pk}_R, m_L, m_R, c_C, c_L, c_R^{\text{OTP}})$ 59 : if $\exists k : (\text{pk}_L, \text{pk}_R, m_L, m_R, c_C, c_L, c_R^{\text{OTP}}, k) \in \mathcal{L}_{\text{Hkey}}$ then 60 : return k 61 : $k \leftarrow \text{Keys}$ 62 : if $(\exists \text{sk}_R^{\text{FO}} = (\text{sk}_R, \text{sr}_*), r : (\text{pk}_R, \text{sk}_R^{\text{FO}}, r) \in \text{LeakedR})$ 63 : $\wedge \text{pk}_L = \text{pk}_{L,*}$ then / $G_2 - G_3$ 64 : $(c'_C, c'_L, c'_R) = \text{Enc}_{\text{OTP}}(\text{pp}, \text{pk}_L, \text{pk}_R, m_L, m_R)$ / $G_2 - G_3$ 65 : $(m'_L, m'_R) = \text{Dec}_{\text{OTP}}(\text{pp}, \text{sk}_L, \text{sk}_R, c_C, c_L, c_R^{\text{OTP}})$ / G_2 66 : if $(c'_C, c'_L, c'_R, m'_L, m'_R) = (c_C, c_L, c_R^{\text{OTP}}, m_L, m_R)$ then / G_2 67 : $k \leftarrow \text{H}_{\text{rej}}(\text{pk}_{L,*}, \text{pk}_R, c_C, c_L, c_R^{\text{OTP}})$ / G_2 68 : if $(c'_C, c'_L, c'_R) = (c_C, c_L, c_R^{\text{OTP}})$ then / G_3 69 : $k \leftarrow \text{H}_{\text{rej}}(\text{pk}_{L,*}, \text{pk}_R, c_C, c_L, c_R^{\text{OTP}})$ / G_3 70 : $\mathcal{L}_{\text{Hkey}} = \{(\text{pk}_L, \text{pk}_R, m_L, m_R, c_C, c_L, c_R^{\text{OTP}}, k)\} \cup \mathcal{L}_{\text{Hkey}}$ 71 : return k
$\text{H}_{\text{OTP}}, \text{H}_{\text{rej}}, \text{H}_{\text{rand}}, \text{H}_{\text{rej}}$ 74 : Random oracles	

Fig. 12: Games G_0 to G_3 for the proof of [IND-CK-CCA, -] security in the ROM (Theorem 3).

B.1 Proof in the Random Oracle Model

Proof. Security games for [IND-CK-CCA, -] security.

They are detailed below, the games G_0 to G_3 being in addition illustrated in Figure 12 (page 42).

Game G_0 . The initial [IND-CK-CCA, -] game. We clarify some of the notations used in this game. $(\text{pk}_{L,*}, \text{sk}_{L,*}^{\text{FO}} = (\text{sk}_{L,*}, \text{sl}_{L,*}))$ are the left challenge public and secret keys. $\text{pk}_{R,*}$ is the right challenge public key, and, only in the [IND-CK-CCA, -] game, $\text{sk}_{R,*}^{\text{FO}} = (\text{sk}_R, \text{sr}_*)$ is the right challenge secret key. The ciphertext and key computed with $\text{dEncaps}^{\text{FO}}(\text{pp}, \text{pk}_{L,*}, \text{pk}_{R,*})$ by the challenger form the pair $(\text{ct}_* = (c_{C,*}, c_{L,*}, c_{R,*}), k_0)$, whereas k_1 is the key uniformly sam-

pled from **Keys** by the challenger. We note $(m_{L,*}, m_{R,*})$ the messages sampled to compute the challenge ciphertexts and $(r_{C,*}, r_{L,*}, r_{R,*}) = H_{\text{rand}}(\text{pk}_{L,*}, \text{pk}_{R,*}, m_{L,*}, m_{R,*})$. b is the challenge bit, and b' is the response provided by the adversary. We also recall that **LeakedR** is the set of triplets $(\text{pk}_R, \text{sk}_R^{\text{fo}} = (\text{sk}_R, \text{s}_R), r)$ outputted by the oracle $\mathcal{O}_{\text{leak}_R}$.

Game G_1 . The oracle H_{rej} used by $\mathcal{O}_{\text{CCA}_L}$ is replaced by a new internal oracle $\underline{H}_{\text{rej}}(\text{pk}_{L,*}, \text{pk}_R, \text{s}_R, c_C, c_L, c_R^{\text{otp}})$ that no longer takes the secret s_L as input. Moreover, $\underline{H}_{\text{rej}}$ is not accessible by the adversary.²²

As s_L is sampled uniformly from $\{0, 1\}^\nu$ and is not given to the adversary, we have $\left| \text{Adv}^{G_1}(\mathcal{A}) - \text{Adv}^{G_0}(\mathcal{A}) \right| \leq \frac{q_{\text{rej}}}{2^\nu}$.

Game G_2 . In G_2 , we modify the construction of $H_{\text{key}}(\text{pk}_L, \text{pk}_R, m_L, m_R, c_C, c_L, c_R^{\text{otp}})$. When $\text{pk}_L = \text{pk}_{L,*}$, pk_R belongs to a triplet $(\text{pk}_R, \text{sk}_R^{\text{fo}} = (\text{sk}_R, \text{s}_R), r) \in \text{LeakedR}$, and $(\text{pk}_L, \text{sk}_{L,*}, \text{pk}_R, \text{sk}_R, m_L, m_R, c_C, c_L, c_R^{\text{otp}})$ verifies the predicate P defined by

$$\begin{aligned} & ((c_C, c_L, c_R^{\text{otp}}) = \text{Enc}_{\text{OTP}}(\text{pp}, \text{pk}_{L,*}, \text{pk}_R, m_L, m_R)) \\ & \wedge ((m_L, m_R) = \text{Dec}_{\text{OTP}}(\text{pp}, \text{sk}_{L,*}, \text{sk}_R, (c_C, c_L, c_R^{\text{otp}}))) , \end{aligned}$$

we define $H_{\text{key}}(\text{pk}_L, \text{pk}_R, m_L, m_R, c_C, c_L, c_R^{\text{otp}})$ as $\underline{H}_{\text{rej}}(\text{pk}_{L,*}, \text{pk}_R, \text{s}_R, c_C, c_L, c_R^{\text{otp}})$. Otherwise, $H_{\text{key}}(\text{pk}_L, \text{pk}_R, m_L, m_R, c_C, c_L, c_R^{\text{otp}})$ is equal to $H_{\text{key}}(\text{pk}_L, \text{pk}_R, m_L, m_R, c_C, c_L, c_R^{\text{otp}})$ as in G_1 . Thanks to this new definition of H_{key} , we can rewrite the oracle $\mathcal{O}_{\text{CCA}_L}$ using only H_{rej} .

Note that for such tuples, the function that adds s_R while removing (m_L, m_R) :

$$(\text{pk}_{L,*}, \text{pk}_R, m_L, m_R, c_C, c_L, c_R^{\text{otp}}) \mapsto (\text{pk}_{L,*}, \text{pk}_R, \text{s}_R, c_C, c_L, c_R^{\text{otp}})$$

is injective. Moreover, the value of $\underline{H}_{\text{rej}}$ on such tuples $(\text{pk}_{L,*}, \text{pk}_R, c_C, c_L, c_R^{\text{otp}})$ was never used in the previous game. As a result, the advantage does not change between G_1 and G_2 .

Game G_3 . In G_3 , the predicate $P(\text{pk}_L, \text{pk}_R, m_L, m_R, c_C, c_L, c_R^{\text{otp}})$ described in G_2 is simplified to only be:

$$(c_C, c_L, c_R^{\text{otp}}) = \text{Enc}_{\text{OTP}}(\text{pp}, \text{pk}_{L,*}, \text{pk}_R, m_L, m_R).$$

The only situation where an adversary can distinguish G_2 from G_3 is if $\mathcal{A}$ queries H_{key} on an input $(\text{pk}_{L,*}, \text{pk}_R, m_L, m_R, c_C, c_L, c_R^{\text{otp}})$ for a pk_R that corresponds to a triplet $(\text{pk}_R, \text{sk}_R^{\text{fo}}, r) \in \text{LeakedR}$, and where:

$$\begin{aligned} & ((c_C, c_L, c_R^{\text{otp}}) = \text{Enc}_{\text{OTP}}(\text{pp}, \text{pk}_L, \text{pk}_R, m_L, m_R)) \\ & \wedge ((m_L, m_R) \neq \text{Dec}_{\text{OTP}}(\text{pp}, \text{sk}_{L,*}, \text{sk}_R, (c_C, c_L, c_R^{\text{otp}}))) . \end{aligned}$$

Because DPKE is δ -correct, we see that the probability that such a query is made is less than $q_{\text{rand}}\delta$. We thus have $\left| \text{Adv}^{G_2}(\mathcal{A}) - \text{Adv}^{G_3}(\mathcal{A}) \right| \leq q_{\text{rand}}\delta$.

Moreover, we can see that, by definition, G_3 no longer uses the secret key $\text{sk}_{L,*}^{\text{fo}} = (\text{sk}_{L,*}, \text{s}_{L,*})$ and can thus be completely simulated without its knowledge.

²² We underline all oracles that are not (directly) accessible to the adversary.

Game G_4 . We denote by `EventAskedLeftMess` the event that, at any moment in G_3 , the adversary asks $m_{L,*}$ to H_{OTP} or asks an input containing left message $m_{L,*}$ to H_{rand} or H_{key} . We define G_4 as G_3 except that:

- $c_{R,*}^{\text{OTP}}$ is uniformly sampled in Ciphers_R instead of being computed as $c_{R,*} \oplus H_{\text{OTP}}(m_{L,*})$.
- The randomness $(r_{L,*}, r_{R,*}, r_{C,*})$, previously computed as $H_{\text{rand}}(\text{pk}_{L,*}, \text{pk}_{R,*}, m_{L,*}, m_{R,*})$, is now independently and uniformly sampled.
- The challenge key, previously computed as $H_{\text{key}}(\text{pk}_{L,*}, \text{pk}_{R,*}, m_{L,*}, m_{R,*}, c_{C,*}, c_{L,*}, c_{R,*}^{\text{OTP}})$, is now independently and uniformly sampled.
- The game returns a random bit if `EventAskedLeftMess` happens.

We now study the difference of advantage between G_3 and G_4 . Note that while `EventAskedLeftMess` does not happen, the distribution of $c_{R,*}^{\text{OTP}}$, (r_L, r_R, r_C) and k_1 is indistinguishable from independent uniform distributions by $\mathcal{A}$. Thus, we have

$$\left| \text{Adv}^{G_3}(\mathcal{A}) - \text{Adv}^{G_4}(\mathcal{A}) \right| \leq \Pr[\text{EventAskedLeftMess}]. \quad (1)$$

For the same reason, the event `EventAskedLeftMess` has the same probability to appear in G_3 than in G_4 . Thus, we study the probability of `EventAskedLeftMess` happening in G_4 .

To this end, we build an adversary $\mathcal{B} = (\mathcal{B}_1, \mathcal{B}_2)$ against the IND-CPA security of $\text{PKE}_L = (\text{Keygen}_L, \text{Encr}_L, \text{Decr}_L)$, using in black-box the adversary $\mathcal{A}$ of G_4 .

$\mathcal{B}_1$ is given $(\text{pp}, \text{pk}_{L,*})$ and returns two messages $(m_{L,\text{Fixed}}, m_{L,*})$ to the challenger where $m_{L,*}$ is uniformly sampled and $m_{L,\text{Fixed}}$ is any fixed left message. It then receives a challenge ciphertext $(c_{C,*}, c_{L,*})$ that is either an encryption of $m_{L,\text{Fixed}}$ (if some bit u sampled by the IND-CPA challenger is 0) or $m_{L,*}$ (if u is 1).

$\mathcal{B}_2$ uses $(\text{pp}, \text{pk}_{L,*}, c_{C,*}, c_{L,*})$ to make a simulation of G_4 to the adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ with left challenge public key $\text{pk}_{L,*}$. First, $\mathcal{B}$ gives $(\text{pp}, \text{pk}_{L,*})$ to $\mathcal{A}_1$ that returns the right challenge public key $\text{pk}_{R,*}$. Then, $\mathcal{B}$ uniformly samples $c_{R,*}^{\text{OTP}} \leftarrow \text{Ciphers}_R$ and simulates the rest of G_4 with challenge ciphertext $(c_{C,*}, c_{L,*}, c_{R,*}^{\text{OTP}})$.

Moreover, each time an input is requested to H_{OTP} , H_{rand} or H_{key} , $\mathcal{B}_2$ verifies if $m_{L,*}$ is part of the input. If it is true, it returns 1 (*i.e.* $\mathcal{B}_2$ thinks that $(c_{C,*}, c_{L,*})$ is an encryption of $m_{L,*}$). If this never happens, at the end of the simulation, it returns 0 (*i.e.*, $\mathcal{B}_2$ thinks that $(c_{C,*}, c_{L,*})$ is an encryption of $m_{L,\text{Fixed}}$).

We will show the following inequality:

$$\Pr[\text{EventAskedLeftMess}] \leq 2\text{Adv}_{\text{PKE}_L}^{\text{IND-CPA}}(\mathcal{B}) + \frac{q_{\text{rand}} + q_{\text{key}} + q_{\text{OTP}}}{|\mathcal{M}_L|}. \quad (2)$$

It is trivially verified if $\Pr[\text{EventAskedLeftMess}] \leq \frac{(q_{\text{rand}} + q_{\text{key}} + q_{\text{OTP}})}{|\mathcal{M}_L|}$, we now suppose that $\Pr[\text{EventAskedLeftMess}] > \frac{(q_{\text{rand}} + q_{\text{key}} + q_{\text{OTP}})}{|\mathcal{M}_L|}$.

A direct analysis of $\mathcal{B}$ shows that:

- If the IND-CPA challenger sampled $u = 0$, any value seen by $\mathcal{A}$ is independent of $m_{L,*}$, and thus, $\Pr[\mathcal{B} \text{ returns } 1 \mid u = 0] \leq \frac{(q_{\text{rand}} + q_{\text{key}} + q_{\text{OTP}})}{|\mathcal{M}_L|}$.

- If the IND-CPA challenger sampled $u = 1$, $\mathcal{B}$ perfectly simulates G_4 to $\mathcal{A}$ and $\mathcal{B}_2$ returns 1 exactly when, in this simulation of G_4 , `EventAskedLeftMess` happens. Thus $\Pr[\mathcal{B} \text{ returns } 1 \mid u = 1] = \Pr[\text{EventAskedLeftMess}]$.

It follows that:

$$\begin{aligned} 2\text{Adv}_{\text{PKE}_L}^{\text{IND-CPA}}(\mathcal{B}) &= |\Pr[\mathcal{B} \text{ returns } 1 \mid u = 1] - \Pr[\mathcal{B} \text{ returns } 1 \mid u = 0]| \\ &> \Pr[\text{EventAskedLeftMess}] - \frac{(q_{\text{rand}} + q_{\text{key}} + q_{\text{OTP}})}{|\mathcal{M}_L|}, \end{aligned}$$

which indeed implies Equation (2). Using Equations (1) and (2), we find:

$$\left| \text{Adv}^{G_3}(\mathcal{A}) - \text{Adv}^{G_4}(\mathcal{A}) \right| \leq 2\text{Adv}_{\text{PKE}_L}^{\text{IND-CPA}}(\mathcal{B}) + \frac{(q_{\text{rand}} + q_{\text{key}} + q_{\text{OTP}})}{|\mathcal{M}_L|}.$$

Finally, we see that in G_4 , k_0 and k_1 are both uniformly sampled and independent of any other elements given to $\mathcal{A}$. Thus, $\text{Adv}^{G_4}(\mathcal{A}) = 0$. This completes the proof.

Security games for $[-, \text{IND-CCA}]$ security.

Game G_0 . The initial game $[-, \text{IND-CCA}]$. We use similar notations as for the proof of the $[\text{IND-CK-CCA}, -]$ security. $(\text{pk}_{L,*}, \text{sk}_{L,*})$ are the left challenge public and secret keys, computed with randomness $r_{L,*}$, while $(\text{pk}_{R,*}, \text{sk}_{R,*})$ are the right challenge public and secret keys. The ciphertext and key computed with $\text{dEncaps}^{\text{FO}}(\text{pp}, \text{pk}_{L,*}, \text{pk}_{R,*})$ by the challenger form the pair $(\text{ct}_* = (c_{C,*}, c_{L,*}, c_{R,*}), k_0)$, whereas k_1 is the key uniformly sampled from Keys by the challenger. We note $(m_{L,*}, m_{R,*})$ the messages sampled to compute the challenge ciphertexts and $(r_{C,*}, r_{L,*}, r_{R,*}) = \text{H}_{\text{rand}}(\text{pk}_{L,*}, \text{pk}_{R,*}, m_{L,*}, m_{R,*})$. b is the challenge bit, and b' is the response provided by the adversary. We also recall that LeakedR is the set of triplets $(\text{pk}_L, \text{sk}_L^{\text{FO}} = (\text{sk}_L, s_L), r)$ outputted by the oracle $\mathcal{O}_{\text{leak}_L}$.

We can construct $G_1, \dots, G_3$ by applying the same modifications as those used in $[\text{IND-CK-CCA}, -]$ security, with only two differences:

- We invert the left and right elements,
- In G_3, G_4 and G_5 , we do not only apply the changes when pk_L belongs to a triplet $(\text{pk}_L, \text{sk}_L^{\text{FO}} = (\text{sk}_L, s_L), r) \in \text{LeakedL}$, but also include $\text{pk}_L = \text{pk}_{L,*}$, even if $\text{pk}_{L,*}$ does not belong to such a triplet.

Using the same reasoning as with the $[\text{IND-CK-CCA}, -]$ security, we obtain:

$$\left| \text{Adv}^{G_1}(\mathcal{A}) - \text{Adv}^{G_3}(\mathcal{A}) \right| \leq \frac{q_{\text{rej}}}{2^\nu} + q_{\text{rand}}\delta.$$

Game G_4 . We denote by `EventAskHashKey` the event that, at any moment in G_3 , the adversary asks $(\text{pk}_{L,*}, \text{pk}_{R,*}, m_{L,*}, m_{R,*}, c_{C,*}, c_{L,*}, c_{R,*})$ to H_{key} .

We define G_4 as G_3 except that the game returns a random bit if the event `EventAskHashKey` happens. By definition, we have:

$$\left| \text{Adv}^{G_3}(\mathcal{A}) - \text{Adv}^{G_4}(\mathcal{A}) \right| \leq \Pr[\text{EventAskHashKey}]. \quad (3)$$

In order to bound $\Pr[\text{EventAskHashKey}]$, we build an adversary $\mathcal{B} = (\mathcal{B}_1, \mathcal{B}_2)$ against the $[-, \text{IND-CPA}]$ security of the 2K2M-PKE DPKE, using in black-box the adversary $\mathcal{A}$ of $\mathbf{G}_3$.

First, $\mathcal{B}_1$ obtains public parameters, randomness and keys $(\text{pp}, \text{pk}_{L,*}, \text{sk}_{L,*}, \text{r}_{L,*}, \text{pk}_{R,*})$, and then gives $(m_{R,\text{Fixed}}, m_R, m_L)$ to the challenger, m_R and m_L being uniformly sampled and $m_{R,\text{Fixed}}$ being any fixed right message. It then receives from the challenger a challenge ciphertext $(c_{C,*}, c_{L,*}, c_{R,*})$ which is either an encryption of the two messages $(m_{L,*}, m_{R,\text{Fixed}})$ (if some bit u sampled by the $[-, \text{IND-CPA}]$ challenger is 0) or an encryption of the two messages $(m_{L,*}, m_{R,*})$ (if u is 1).

$\mathcal{B}_2$ uses these elements to make a perfect simulation of $\mathbf{G}_3$ to $\mathcal{A}$ with challenge ciphertext $(c_{C,*}, c_{L,*}, c_{R,*}^{\text{OTP}} = c_{R,*} \oplus \text{H}_{\text{OTP}}(m_{L,*}))$. This is possible because knowledge of $\text{sk}_{R,*}$ is not required to simulate $\mathbf{G}_3$.

Moreover, each time H_{key} is queried by $\mathcal{A}$ on an input of the form $(\text{pk}_{L,*}, \text{pk}_{R,*}, m_{L,*}, m_R, c_{C,*}, c_{L,*}, c_{R,*})$, $\mathcal{B}_2$ checks if $m_R = m_{R,*}$. If it is the case, it stops and returns 1 (*i.e.* $\mathcal{B}_2$ thinks that $(c_{C,*}, c_{L,*}, c_{R,*})$ is an encryption of $(m_{L,*}, m_{R,*})$). If this never happens during the simulation, $\mathcal{B}_2$ returns 0 (*i.e.* $\mathcal{B}_2$ thinks that $(c_{C,*}, c_{L,*}, c_{R,*})$ is an encryption of $(m_{L,*}, m_{R,\text{Fixed}})$).

We will show the following inequality:

$$\Pr[\text{EventAskHashKey}] \leq 2 \text{Adv}_{\text{DPKE}}^{[-, \text{IND-CPA}]}(\mathcal{B}) + \frac{q_{\text{key}}}{|\mathcal{M}_R|}. \quad (4)$$

It is trivially verified if $\Pr[\text{EventAskHashKey}] \leq \frac{q_{\text{key}}}{|\mathcal{M}_R|}$, we now suppose that $\Pr[\text{EventAskHashKey}] > \frac{q_{\text{key}}}{|\mathcal{M}_R|}$.

A direct analysis of $\mathcal{B}$ shows that:

- If the $[-, \text{IND-CPA}]$ challenger sampled $u = 0$, any value seen by $\mathcal{A}$ is independent of $m_{R,*}$, and thus, $\Pr[\mathcal{B} \text{ returns } 1 \mid u = 0] \leq \frac{q_{\text{key}}}{|\mathcal{M}_R|}$.
- If the $[-, \text{IND-CPA}]$ challenger sampled $u = 1$, $\mathcal{B}$ makes a perfect simulation of $\mathbf{G}_3$ to $\mathcal{A}$ and $\mathcal{B}_2$ returns 1 exactly when EventAskHashKey happens in this simulation of $\mathbf{G}_3$. Thus $\Pr[\mathcal{B} \text{ returns } 1 \mid u = 1] = \Pr[\text{EventAskHashKey}]$.

Thus, we have:

$$\begin{aligned} 2 \text{Adv}_{\text{DPKE}}^{[-, \text{IND-CPA}]}(\mathcal{B}) &= |\Pr[\mathcal{B} \text{ returns } 1 \mid u = 1] - \Pr[\mathcal{B} \text{ returns } 1 \mid u = 0]| \\ &> \Pr[\text{EventAskHashKey}] - \frac{q_{\text{key}}}{|\mathcal{M}_R|}, \end{aligned}$$

which indeed implies Equation (4).

Using Equations (3) and (4), we obtain

$$\left| \text{Adv}^{\mathbf{G}_3}(\mathcal{A}) - \text{Adv}^{\mathbf{G}_4}(\mathcal{A}) \right| \leq 2 \text{Adv}_{\text{DPKE}}^{[-, \text{IND-CPA}]}(\mathcal{B}) + \frac{q_{\text{key}}}{|\mathcal{M}_R|}. \quad (5)$$

Finally, we see that in $\mathbf{G}_4$, whenever EventAskHashKey does not happen, k_0 and k_1 are both uniformly sampled and independent of any other element given to $\mathcal{A}$. Thus, $\text{Adv}^{\mathbf{G}_4}(\mathcal{A}) = 0$ and we can conclude.

B.2 Proof in the Quantum-Accessible Random Oracle Model

The proof of this theorem uses the One-Way to Hide Lemma with Semi-Classical Oracles introduced in [3] and follows the same ideas than the proof of [24, Theorem 3.2].

We begin by introducing some necessary results in two subsections before doing the proof, using the quantum notations defined in [24, Section 6].

Generic Distinguishing Problem. For $\lambda \in [0, 1]$, we denote by B_λ the Bernoulli distribution with parameter λ .

Lemma 2 (Generic Distinguishing Problem with Bounded Probabilities [33]). *Let X be a finite set, and let $\lambda \in [0, 1]$. For any (unbounded, quantum) algorithm $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ issuing at most q_0 quantum queries to the oracle $\mathcal{O}$ given by the games $\text{GDPB}_{\lambda,0}$ and $\text{GDPB}_{\lambda,1}$ defined in Figure 13, we have:*

$$\left| \Pr[\text{GDPB}_{\lambda,0}^{\mathcal{A}} \Rightarrow 1] - \Pr[\text{GDPB}_{\lambda,1}^{\mathcal{A}} \Rightarrow 1] \right| \leq 8 \lambda (q_0 + 1)^2.$$

Game $\text{GDPB}_{\lambda,b}^{\mathcal{A}}$
$(\lambda_x)_{x \in X} \leftarrow \mathcal{A}_1$ if $\exists x \in X$ such that $\lambda_x > \lambda$ then return 0 if $b = 1$ then for $x \in X$: $\mathcal{O}(x) \leftarrow B_{\lambda_x}$ else for $x \in X$: $\mathcal{O}(x) \leftarrow 0$ $b' \leftarrow \mathcal{A}_2^{ \mathcal{O}\rangle}$ return b'

Fig. 13: Generic distinguishing games $\text{GDPB}_{\lambda,b}$ with bounded maximal Bernoulli parameter $\lambda \in [0, 1]$.

Semi-Classical Oracles and One-Way to Hide Lemma. We follow the presentation of semi-classical oracles made in [24], using the quantum notations defined in [24, Section 6]. In particular, for a finite set X , we define $\mathcal{H}_X$ the Hilbert space with canonical orthonormal basis $\{|x\rangle, x \in X\}$. We define the X register as the space of $\sum_{x \in X} \alpha_x |x\rangle$ such that $\sum_{x \in X} |\alpha_x|^2 = 1$.

From any finite sets X, Y and subset $S \subset X$, one can define a *semi-classical* oracle $\mathcal{O}_S^{\text{SC}}$ that we describe now. Intuitively, the oracle $\mathcal{O}_S^{\text{SC}}$ collapses a state $\sum_{(x,y) \in X \times Y} \alpha_{x,y} |x\rangle \otimes |y\rangle \in \mathcal{H}_X \otimes \mathcal{H}_Y$ to a state whose X -register contains only

elements of either S or $X \setminus S$. More precisely, $\mathcal{O}_S^{\text{SC}}$ takes as input a quantum state $|\psi, 0\rangle$ such that $|\psi\rangle \in_X \otimes \mathcal{H}_Y$. $\mathcal{O}_S^{\text{SC}}$ first measures the X -register with respect to the projectors $M_1 := \sum_{x \in S} |x\rangle \langle x|$ and $M_0 := \sum_{x \notin S} |x\rangle \langle x|$ and then initializes the last register to $|b\rangle$ for the measured bit b . Consequently, $|\psi, 0\rangle$ collapses to either a state $|\psi', 1\rangle$ such that the X -register of $|\psi'\rangle$ only contains elements of S , or a state $|\psi', 0\rangle$ such that the X -register of $|\psi'\rangle$ only contains elements of $X \setminus S$.

To a quantum-accessible oracle $\mathcal{O}$ and a subset $S \subset X$, one can furthermore associate oracle $\mathcal{O} \setminus S$ which first queries $\mathcal{O}_S^{\text{SC}}$ and then $\mathcal{O}$. Let **FIND** denote the event that $\mathcal{O}_S^{\text{SC}}$ ever returns a state $|\psi', 1\rangle$. Unless **FIND** occurs, the outcome of $\mathcal{O} \setminus S$ is independent of the values $\mathcal{O}(x)$ for all $x \in S$, which is why $\mathcal{O} \setminus S$ is also called a “punctured” oracle.

Lemma 3 (One Way To Hiding Lemma of [24, Lemmas 5.2 and 5.3]).

Let $S \subset X$ be random and $\mathcal{O}_1, \mathcal{O}_2 \in Y^X$ be random functions such that $\mathcal{O}_1(x) = \mathcal{O}_2(x)$ for all $x \in X \setminus S$. Let also z be a random bitstring, where $(S, \mathcal{O}_1, \mathcal{O}_2, z)$ may have an arbitrary joint distribution. Let $q \in \mathbb{N}$ and $\mathcal{A}$ a quantum algorithm with binary output and making less than q queries to a random oracle $\mathcal{O}_1$. Moreover, for $i \in \{1, 2\}$, let

$$p_i := \Pr_{\mathcal{O}_i, z} [1 \leftarrow \mathcal{A}^{|\mathcal{O}_i\rangle}(z)],$$

$$p_{\text{Find}} = \Pr_{\mathcal{O}_2, S, z} [b \leftarrow \mathcal{A}^{|\mathcal{O}_2 \setminus S\rangle}(z) : \text{FIND}].^{23}$$

Then:

$$|p_1 - p_2| \leq 2\sqrt{(q+1)p_{\text{Find}}}.$$

Lemma 4 (Corollary 1 of [3]). Using the notations of Lemma 3, suppose that S and z are independent and that $\mathcal{C}$ is a q -query algorithm. Let $p_{\max} := \max_{x \in X} \Pr[x \in S]$. Then, we have $\Pr_{\mathcal{O}_2, S, z} [\text{FIND} : \mathcal{A}^{|\mathcal{O}_2 \setminus S\rangle}] \leq 4q \cdot p_{\max}$.

Lemma 5 (Using a hash function as a PRF in QROM, Adaptation of Corollary 1 of [8]). Let $H : (K \times X) \rightarrow Y$ be a quantum-accessible random oracle. This function may be used as a quantum-accessible PRF $F_k(x) := H(k, x)$ with a key $k \leftarrow \$K$. Suppose a PRF-adversary $\mathcal{A}$ makes q queries to H and any number of queries to F_k . Then $\text{Adv}^{\text{IND-PRF}}(\mathcal{A}) \leq 2q/\sqrt{|K|}$, where:

$$\text{Adv}^{\text{IND-PRF}}(\mathcal{A}) = \Pr [b = b' \mid b' \leftarrow \mathcal{A}^{|\mathcal{O}_b\rangle}()];$$

$$\mathcal{O}_0 = H(k, \cdot), (k, \mathcal{O}_1) \leftarrow \$K \times Y^X; b \leftarrow \$\{0, 1\}.$$

QROM Proof of [IND-CK-CCA, -] Security

²³ Hence, p_{Find} denotes the probability that the event **FIND** occurs during the execution of $\mathcal{A}^{|\mathcal{O}_2 \setminus S\rangle}(z)$, where S , z , and $\mathcal{O}_2$ are sampled according to their respective distributions.

Proof. The detailed descriptions of the games are given below, with an overview of Games G_0 to G_5 provided in Figure 14 (page 50).

Game G_0 . This is the initial game [IND-CK-CCA, -].

We clarify some of the notations used in this game. $(pk_{L,*}, sk_{L,*})$ are the left challenge public and secret keys, while $pk_{R,*}$ is the right challenge public key. The ciphertext and key computed with $dEncaps^{FO}(pp, pk_{L,*}, pk_{R,*})$ by the challenger form the pair $(ct_* = (c_{C,*}, c_{L,*}, c_{R,*}), k_0)$, whereas k_1 is the key uniformly sampled from $Keys$ by the challenger. We note $(m_{L,*}, m_{R,*})$ the messages sampled to compute the challenge ciphertexts and $(r_{C,*}, r_{L,*}, r_{R,*}) = H_{rand}(pk_{L,*}, pk_{R,*}, m_{L,*}, m_{R,*})$. b is the challenge bit and b' is the response provided by the adversary. We also recall that $LeakedR$ is the set of triplets $(pk_R, sk_R^{FO} = (sk_R, s_R), r)$ outputted by the oracle $\mathcal{O}_{leak_R}$.

Game G_1 . As we know there are at most q_{leak} requests to $\mathcal{O}_{leak_R}$ so we can precompute at the beginning of the game the elements (pk_R, sk_R^{FO}, r) that can be outputted by $\mathcal{O}_{leak_R}$. We denote them by $(pk_{R,i}, sk_{R,i}^{FO} = (sk_{R,i}, s_{R,i}), r_i)$ for $0 \leq i \leq q_{leak} - 1$. Moreover, we modify the construction of H_{rand} . When the input is of the form $(pk_{L,*}, pk_{R,i}, m_L, m_R)$, the computation is made by an internal oracle $H_{Rand}^i(m_L, m_R)$ with range $Rand_L \times Rand_R \times Rand_C$, not directly accessible by the adversary.²⁴ The advantage remains unchanged.

Game G_2 . The oracle H_{rej} used by $\mathcal{O}_{CCA_L}$ is replaced by a new oracle $\underline{H}_{Rej}(pk_{L,*}, pk_R, s_R, c_C, c_L, c_R^{OTP})$ that no longer takes the secret s_L as input. Moreover, $\underline{H}_{Rej}$ is not accessible by the adversary. Using Lemma 5, we obtain:

$$\left| Adv^{G_1}(\mathcal{A}) - Adv^{G_2}(\mathcal{A}) \right| \leq 2q_{rej}2^{-\nu/2}.$$

Game G_3 . We replace the random oracle H_{rand} by an oracle which samples only “good” randomness for any couple of public keys $(pk_{L,*}, pk_{R,i})$, for $0 \leq i \leq q_{leak} - 1$, meaning that no decryption failures can occur when using $Dec_{OTP}(pp, sk_{L,*}, sk_{R,i}, \cdot)$ on proper ciphertexts of the form $Enc_{OTP}(pp, pk_{L,*}, pk_{R,i}, \cdot)$.

More precisely, for each $0 \leq i \leq q_{leak} - 1$ and each m_L, m_R we define $\mathcal{R}_{bad}^i(m_L, m_R)$ as the set:

$$\left\{ r = (r_L, r_R, r_C) \in Rand_L \times Rand_R \times Rand_C : \right. \\ \left. Dec_{OTP}\left(pp, sk_L, sk_{R,i}, Enc_{OTP}(pp, pk_{L,*}, pk_{R,i}, m_L, m_R; r)\right) \neq (m_L, m_R) \right\}.$$

In G_3 , for each $0 \leq i \leq q_{leak} - 1$, we replace the internal random oracle H_{Rand}^i by an oracle that, on query (m_L, m_R) , uniformly samples an output in $Rand_L \times Rand_R \times Rand_C \setminus \mathcal{R}_{bad}^i(m_L, m_R)$.

Using Lemma 2 with exactly the same reasoning as in Game 4 of the proof of [24, Theorem 3.2], we deduce that:

$$\left| Adv^{G_2}(\mathcal{A}) - Adv^{G_3}(\mathcal{A}) \right| \leq 8q_{leak}\delta(q_{rand} + q_{CCA} + 1)^2.$$

²⁴ We underline all oracles that are not directly accessible by the adversary.

[IND-CK-CCA, -] and [IND-CCA, -]	$\mathcal{O}_{\text{CCA}_L}(\text{pk}_R, \text{ct} = (c_C, c_L, c_R^{\text{OTP}}))$
1 : $\text{pp} \leftarrow \text{Setup}()$ 2 : $(\text{pk}_{L,*}, \text{sk}_{L,*}^{\text{FO}} = (\text{sk}_{L,*}, \text{sl}_{L,*})) \leftarrow \text{dKeygen}_L^{\text{FO}}(\text{pp})$ 3 : for $i \in \text{Range}(\text{eph-key})$: 4 : $r_i \leftarrow \text{RandKeygenR}$ 5 : $(\text{pk}_{R,i}, \text{sk}_{R,i}^{\text{FO}} = (\text{sk}_{R,i}, \text{sr}_{R,i})) \leftarrow \text{dKeygen}_R^{\text{FO}}(r)$ 6 : $\text{cpt}_{\text{leak}} \leftarrow 0$ 7 : $\text{LeakedR} \leftarrow \emptyset$ 8 : $\text{Oracles} = \{\mathcal{O}_{\text{CCA}_L}, \mathcal{O}_{\text{leakR}}, \text{H}_{\text{rej}}, \text{H}_{\text{rand}}, \text{H}_{\text{key}}, \text{H}_{\text{OTP}}\}$ 9 : $(\text{state}, \text{pk}_{R,*}) \leftarrow \mathcal{A}_1^{\text{Oracles}}(\text{pk}_{L,*})$ 10 : $b \leftarrow \{0, 1\}$ 11 : $(\text{ct}_*, k_0) \leftarrow \text{dEncaps}^{\text{FO}}(\text{pp}, \text{pk}_{L,*}, \text{pk}_{R,*})$ 12 : We parse ct_* as $(c_{C,*}, c_{L,*}, c_{R,*})$ 13 : We denote by $(m_{L,*}, m_{R,*})$ and by 14 : $(r_{C,*}, r_{L,*}, r_{R,*})$ the messages and 15 : randomnesses used to produce 16 : this encapsulation. 17 : $k_1 \leftarrow \text{Keys}$ 18 : $b' \leftarrow \mathcal{A}_2^{\text{Oracles}}(\text{state}, \text{ct}_*, k_b)$ 19 : return $b = b'$	39 : // with $\text{ct}_*, \text{LeakedR}$ and $\text{sk}_{L,*}$ 40 : // taken from the security game 41 : if $(\exists \text{sk}_R^{\text{FO}} = (\text{sk}_R, \text{sr}), r : (\text{pk}_R, \text{sk}_R^{\text{FO}}, r) \in \text{LeakedR})$ 42 : $\wedge (\text{ct} \neq \text{ct}_* \vee \text{pk}_R \neq \text{pk}_{R,*})$ then 43 : $\tilde{k} \leftarrow \text{H}_{\text{rej}}(\text{pk}_L, \text{pk}_R, \text{sl}_R, \text{cc}, c_L, c_R^{\text{OTP}})$ 44 : $\tilde{k} \leftarrow \text{H}_{\text{rej}}(\text{pk}_L, \text{pk}_R, \text{sr}, \text{cc}, c_L, c_R^{\text{OTP}})$ 45 : return $\tilde{k}$ 46 : $(m_L, m_R) \leftarrow \text{Dec}_{\text{OTP}}(\text{pp}, \text{sk}_L, \text{sk}_R, \text{cc}, c_L, c_R^{\text{OTP}})$ 47 : if $(m_L, m_R) = (\perp, \perp)$ then 48 : return $\tilde{k}$ 49 : if $\text{Enc}_{\text{OTP}}(\text{pp}, \text{pk}_L, \text{pk}_R, m_L, m_R)$ 50 : $\neq (c_C, c_L, c_R^{\text{OTP}})$ then 51 : return $\tilde{k}$ 52 : return $\text{H}_{\text{key}}(\text{pk}_L, \text{pk}_R, m_L, m_R, \text{cc}, c_L, c_R^{\text{OTP}})$
$\mathcal{O}_{\text{leakR}}()$ 51 : $r \leftarrow \text{RandKeygenR}$ 52 : $(\text{pk}_R, \text{sk}_R^{\text{FO}} = (\text{sk}_R, \text{sr})) \leftarrow \text{dKeygen}_R^{\text{FO}}(r)$ 53 : $\text{LeakedR} \leftarrow \text{LeakedR} \cup \{(\text{pk}_R, \text{sk}_R^{\text{FO}}, r)\}$ 54 : $\text{LeakedpkR} \leftarrow \text{LeakedpkR} \cup \{\text{pk}_R\}$ 55 : return $(\text{pk}_R, \text{sk}_R^{\text{FO}}, r)$ 56 : $\text{cpt}_{\text{leak}} \leftarrow \text{cpt}_{\text{leak}} + 1$ 57 : return $(\text{pk}_{R, \text{cpt}_{\text{leak}}-1}, \text{sk}_{R, \text{cpt}_{\text{leak}}-1}, r_{\text{cpt}_{\text{leak}}-1})$	$\text{H}_{\text{key}}(\text{pk}_L, \text{pk}_R, m_L, m_R, \text{cc}, c_L, c_R^{\text{OTP}})$ 59 : if $\exists k : (\text{pk}_L, \text{pk}_R, m_L, m_R, \text{cc}, c_L, c_R^{\text{OTP}}, k) \in \mathcal{L}_{\text{Hkey}}$ then 60 : return k 61 : $k \leftarrow \text{Keys}$ 62 : if $(\text{pk}_L = \text{pk}_{L,*}) \wedge (\exists i : \text{pk}_R = \text{pk}_{R,i})$ then 63 : $(c'_C, c'_L, c'_R) = \text{Enc}_{\text{OTP}}(\text{pp}, \text{pk}_L, \text{pk}_{R,i}, m_L, m_R)$ 64 : if $(c'_C, c'_L, c'_R) = (c_C, c_L, c_R^{\text{OTP}})$ then 65 : $k = \text{H}_{\text{rej}}(\text{pk}_{L,*}, \text{pk}_{R,i}, \text{sr}_{R,i}, \text{cc}, c_L, c_R^{\text{OTP}})$ 66 : $\mathcal{L}_{\text{Hkey}} = \{(\text{pk}_L, \text{pk}_R, m_L, m_R, \text{cc}, c_L, c_R^{\text{OTP}}, k)\} \cup \mathcal{L}_{\text{Hkey}}$ 67 : return k
$\text{H}_{\text{OTP}}, \text{H}_{\text{rej}}, \text{H}_{\text{rand}}$ 74 : Random oracles	$\text{H}_{\text{rand}}^i(m_L, m_R)$ for $0 \leq i \leq \text{eph-key} - 1$ 74 : Computed as a random oracle / $\text{G}_1 - \text{G}_5$ 75 : if input already queried then / $\text{G}_3 - \text{G}_4$ 76 : return the same result / $\text{G}_3 - \text{G}_4$ 77 : else then / $\text{G}_3 - \text{G}_4$ 78 : $x \leftarrow \text{Rand}_{\text{CLR}} \setminus \mathcal{R}_{\text{bad}}^i(m_L, m_R)$ / $\text{G}_3 - \text{G}_4$ 79 : record and output x / $\text{G}_3 - \text{G}_4$
$\text{H}_{\text{rand}}(\text{pk}_L, \text{pk}_R, m_L, m_R)$ 74 : Computed as a random oracle / G_0 75 : if $\text{pk}_L = \text{pk}_{L,*} \wedge \exists i : \text{pk}_R = \text{pk}_{R,i}$ then / $\text{G}_1 - \text{G}_5$ 76 : return $\text{H}_{\text{rand}}^i(m_L, m_R)$ / $\text{G}_1 - \text{G}_5$ 77 : else return $\text{H}_{\text{rand}}(\text{pk}_L, \text{pk}_R, m_L, m_R)$ / $\text{G}_1 - \text{G}_5$	

Fig. 14: Games G_0 to G_5 for the proof of [IND-CK-CCA, -] security in the QROM (Theorem 3).

Game G_4 . In G_4 , we modify the construction of $\text{H}_{\text{key}}(\text{pk}_L, \text{pk}_R, m_L, m_R, \text{cc}, c_L, c_R^{\text{OTP}})$ when $\text{pk}_L = \text{pk}_{L,*}$ and $\text{pk}_R = \text{pk}_{R,i}$ for some $0 \leq i \leq q_{\text{leak}} - 1$. More precisely, for any tuple $(\text{pk}_{L,*}, \text{pk}_{R,i}, m_L, m_R, \text{cc}, c_L, c_R^{\text{OTP}})$ such that $\text{Enc}_{\text{OTP}}(\text{pp}, \text{pk}_{L,*}, \text{pk}_{R,i},$

$m_L, m_R) = (c_C, c_L, c_R^{\text{OTP}})$, we set $H_{\text{key}}(\text{pk}_L, \text{pk}_{R,i}, m_L, m_R, c_C, c_L, c_R^{\text{OTP}}) = H_{\text{Rej}}(\text{pk}_{L,*}, \text{pk}_{R,i}, s_{R,i}, c_C, c_L, c_R^{\text{OTP}})$.

Thanks to this new definition of H_{key} , we can rewrite the oracle $\mathcal{O}_{\text{CCA}_L}$ using only H_{Rej} .

Due to the modification brought by G_3 , for any tuple $(\text{pk}_{L,*}, \text{pk}_{R,i}, m_L, m_R, c_C, c_L, c_R^{\text{OTP}})$ such that $(c_C, c_L, c_R^{\text{OTP}}) = \text{Enc}_{\text{OTP}}(\text{pp}, \text{pk}_L, \text{pk}_{R,i}, m_L, m_R)$, we have $(m_L, m_R) = \text{Dec}_{\text{OTP}}(\text{pp}, \text{sk}_L, \text{sk}_{R,i}, (c_C, c_L, c_R^{\text{OTP}}))$. As a result, the function transforming tuples $(\text{pk}_{L,*}, \text{pk}_{R,i}, m_L, m_R, c_C, c_L, c_R^{\text{OTP}})$ to $(\text{pk}_{L,*}, \text{pk}_{R,i}, s_{R,i}, c_C, c_L, c_R^{\text{OTP}})$ is injective. Moreover, H_{Rej} was never used on such tuples. As a result, the advantage does not change between G_3 and G_4 .

Game G_5 . We make the reverse operation of game G_3 by returning to the initial oracle H_{rand} prior of game G_5 . Using the same argument as in G_3 , we obtain:

$$\left| \text{Adv}^{G_4}(\mathcal{A}) - \text{Adv}^{G_5}(\mathcal{A}) \right| \leq 8q_{\text{leak}}\delta(q_{\text{rand}} + q_{\text{CCA}} + 1)^2.$$

Game G_6 . The left challenge message $m_{L,*}$ is used by three random oracles H_{rand} , H_{OTP} and H_{key} . In order to use the *one-way to hiding* Lemma 3, we modify them in order that the message $m_{L,*}$ will be queried to only one random oracle.

We introduce an internal random oracle $\underline{H}_{\text{OWTH}}$ that, on query $(\text{pk}_L, \text{pk}_R, m_L, m_R, c_C, c_L, c_R^{\text{OTP}})$ (same input set than H_{key}), outputs a triplet $((r'_L, r'_R, r'_C), c'_C, k')$ where:

- (r'_L, r'_R, r'_C) only depends on $(\text{pk}_L, \text{pk}_R, m_L, m_R)$ (modifying the other arguments $(c_C, c_L, c_R^{\text{OTP}})$ does not change its value).
- c'_C only depends on m_L (modifying the other arguments $(\text{pk}_L, \text{pk}_R, m_R, c_C, c_L, c_R^{\text{OTP}})$ does not change its value).

Then, in G_6 , we use $\underline{H}_{\text{OWTH}}$ to simulate the three oracles H_{rand} , H_{OTP} and H_{key} . For example, if the adversary queries H_{rand} on input $(\text{pk}_L, \text{pk}_R, m_L, m_R)$, we add any three values $c_C, c_L, c_R^{\text{OTP}}$ to them, query $((r'_L, r'_R, r'_C), c'_C, k') = \underline{H}_{\text{OWTH}}(\text{pk}_L, \text{pk}_R, m_L, m_R, c_C, c_L, c_R^{\text{OTP}})$ and answer (r'_L, r'_R, r'_C) .

The distribution of any element is unchanged, thus the advantage remains unchanged.

Game G_7 . Informally, in G_7 , we decouple the oracle value returned by $\underline{H}_{\text{OWTH}}$ when queried with a message containing the challenge plaintext $m_{L,*}$ from the values that are used to compute the challenge ciphertext, the challenge keys and the OTP added to c_R to obtain c_R^{OTP} .

More formally, let us consider the set $\mathcal{X} = \text{PK}_L \times \text{PK}_R \times \mathcal{M}_L \times \mathcal{M}_R \times \text{Ciphers}_C \times \text{Ciphers}_L \times \text{Ciphers}_R$ and $\mathcal{S}$ the subset of elements of $\mathcal{X}$ where the left message is equal to the challenge left message $m_{L,*}$. In G_7 , we reprogram $\underline{H}_{\text{OWTH}}$ on any value of $\mathcal{S}$ with uniform values. As a result, in G_7 , $(r_{C,*}, r_{L,*}, r_{R,*})$, k_1 and $c_{R,*}^{\text{OTP}}$ follow a uniform distribution independent of the outputs of $\underline{H}_{\text{OWTH}}$, and thus independent of any value accessible by $\mathcal{A}$. Using [24, Lemma 5.2], we have:

$$\left| \text{Adv}^{G_6}(\mathcal{A}) - \text{Adv}^{G_7}(\mathcal{A}) \right| \leq 2\sqrt{(\hat{q} + 1) \cdot \Pr[b \leftarrow \mathcal{C}^{\underline{H}_{\text{OWTH}} \setminus \mathcal{S}}(m_{L,*}) : \text{FIND}]}, \quad (6)$$

where:

$$\hat{q} = q_{\text{rand}} + q_{\text{key}} + q_{\text{OTP}} + q_{\text{CCA}}.$$

- $\mathcal{C}$ runs the game G_7 with adversary $\mathcal{A}$ and the punctured oracle $|\mathcal{H}_{\text{OWTH}} \setminus \mathcal{S}\rangle$ instead of $|\mathcal{H}_{\text{OWTH}}\rangle$. Moreover, $m_{L,*}$ is considered as an input of $\mathcal{C}$, whose distribution is uniform over $\mathcal{M}_L$.

Because the distribution of $\mathcal{S}$ is not independent from $m_{L,*}$, we cannot use Lemma 4 to bound $\Pr[b \leftarrow \mathcal{C}^{|\mathcal{H}_{\text{OWTH}} \setminus \mathcal{S}\rangle}(m_{L,*}) : \text{FIND}]$.

In order to bound $\Pr[b \leftarrow \mathcal{C}^{\mathcal{O}_S^{\text{sc}}}(z) : \text{FIND}]$, we introduce game G_8 whose only difference with G_7 is that $c_{L,*}$ is computed as $\text{Encr}_L(\text{pk}_{L,*}, m_{L,\text{Fixed}}, r_{L,*})$ instead of $\text{Encr}_L(\text{pk}_{L,*}, m_{L,*}, r_{L,*})$, where $m_{L,\text{Fixed}}$ is any fixed left message.

We note $\mathcal{D}$ the algorithm with no input, that runs the game G_8 with adversary $\mathcal{A}$ and with the punctured oracle $|\mathcal{H}_{\text{OWTH}} \setminus \mathcal{S}\rangle$. There is clearly independence between the (empty) input of $\mathcal{D}$ and the set $\mathcal{S}$, so we can use Lemma 4 and obtain:

$$\Pr[\mathcal{D}^{|\mathcal{H}_{\text{OWTH}} \setminus \mathcal{S}\rangle}() : \text{FIND}] \leq 4\hat{q}|\mathcal{S}|/|\mathcal{X}| = 4\hat{q}/|\mathcal{M}_L|, \quad (7)$$

where $\hat{q} = q_{\text{rand}} + q_{\text{key}} + q_{\text{OTP}} + q_{\text{CCA}}$.

Now, we study the difference of advantages between G_7 and G_8 . We will bound it by the advantage of an adversary $\mathcal{E} = (\mathcal{E}_1, \mathcal{E}_2)$ against the IND-CPA game of PKE_L .

First, $\mathcal{E}_1$ obtains public parameters pp , public key $\text{pk}_{L,*}$ and then gives two messages $(m_{L,\text{Fixed}}, m_{L,*})$ to the challenger, $m_{L,*}$ being uniformly sampled and $m_{L,\text{Fixed}}$ being any fixed left message. We denote by $\tilde{b}$ the bit sampled by the challenger of the game IND-CPA of PKE_L before encrypting either $m_{L,\text{Fixed}}$ if $\tilde{b} = 0$ or $m_{L,*}$ if $\tilde{b} = 1$.

Then, $\mathcal{E}_2$ receives $(c_{C,*}, c_{L,*})$ and needs to decide if it is an encryption of $m_{L,\text{Fixed}}$ or $m_{L,*}$. $\mathcal{E}_2$ uses $(\text{pp}, \text{pk}_L, c_{C,*}, c_{L,*})$ to simulate G_7 to $\mathcal{A}$, this includes the simulation of each random oracle. However, $\mathcal{E}_2$ uses $|\mathcal{H}_{\text{OWTH}} \setminus \mathcal{S}\rangle$ instead of $|\mathcal{H}_{\text{OWTH}}\rangle$. Finally, $\mathcal{E}_2$ outputs 1 if FIND happens during the simulation, and 0 else.

By definition of $\mathcal{E}$, G_7 and G_8 we see that, until FIND happens:

- $\mathcal{E}$ makes a perfect simulation of $\mathcal{C}$ (with the punctured oracle) if $\tilde{b} = 1$.
- $\mathcal{E}$ makes a perfect simulation of $\mathcal{D}$ (with the punctured oracle) if $\tilde{b} = 0$.

Thus, we have

$$\Pr[\mathcal{C}^{|\mathcal{H}_{\text{OWTH}} \setminus \mathcal{S}\rangle}(m_{L,*}) : \text{FIND}] \leq 2\text{Adv}^{\text{IND-CPA}}(\mathcal{E}) + \Pr[\mathcal{D}^{|\mathcal{H}_{\text{OWTH}} \setminus \mathcal{S}\rangle}() : \text{FIND}]. \quad (8)$$

Using Equations (8) and (7), we obtain:

$$\Pr[\mathcal{C}^{|\mathcal{H}_{\text{OWTH}} \setminus \mathcal{S}\rangle}(m_{L,*}) : \text{FIND}] \leq 2\text{Adv}^{\text{IND-CPA}}(\mathcal{B}) + 4\hat{q}/|\mathcal{M}_L|, \quad (9)$$

where $\hat{q} = q_{\text{rand}} + q_{\text{key}} + q_{\text{OTP}} + q_{\text{CCA}}$.

Finally, we note that in G_8 , both k_0 and k_1 follow independent uniform distributions, and thus, $\text{Adv}^{G_7}(\mathcal{A}) = 0$. Using this as well as Equations (6) and (9), we have:

$$\text{Adv}^{G_7}(\mathcal{A}) \leq 2\sqrt{(\hat{q} + 1) \left(2\text{Adv}_{\text{PKE}_L}^{\text{IND-CPA}}(\mathcal{B}) + 4\hat{q}/|\mathcal{M}_L| \right)},$$

where $\hat{q} = q_{\text{rand}} + q_{\text{key}} + q_{\text{OTP}} + q_{\text{CCA}}$ and this concludes the proof.

Proof of the QROM case, $[-, \text{IND-CCA}]$ security

Proof. **Game G_0 .** This is the initial $[-, \text{IND-CCA}]$ game. We use similar notations as for the proof of the $[\text{IND-CCA}, -]$ security. $(\text{pk}_{L,*}, \text{sk}_{L,*})$ are the left challenge public and secret keys computed with randomness $r_{L,*}$, while $(\text{pk}_{R,*}, \text{sk}_{R,*})$ are the right ones. The ciphertext and key computed with $\text{dEncaps}^{\text{FO}}(\text{pp}, \text{pk}_{L,*}, \text{pk}_{R,*})$ by the challenger form the pair $(\text{ct}_* = (c_{C,*}, c_{L,*}, c_{R,*}), k_0)$, whereas k_1 is the key uniformly sampled from Keys by the challenger. We note $(m_{L,*}, m_{R,*})$ the messages sampled to compute the challenge ciphertexts and $(r_{C,*}, r_{L,*}, r_{R,*}) = \text{H}_{\text{rand}}(\text{pk}_{L,*}, \text{pk}_{R,*}, m_{L,*}, m_{R,*})$. b is the challenge bit and b' is the response provided by the adversary. We also recall that LeakedR is the set of triplets $(\text{pk}_L, \text{sk}_L^{\text{FO}} = (\text{sk}_L, s_L), r)$ outputted by the oracle $\mathcal{O}_{\text{leak}_L}$.

We introduce games $G_1, \dots, G_5$ that apply the same modifications as the ones with same name used for $[\text{IND-CK-CCA}, -]$ security, with only two differences:

- We invert the left and right elements.
- In G_3, G_4 and G_5 , we do not only apply the changes for any couple of public keys $(\text{pk}_{L,i}, \text{pk}_{R,*})$ for $0 \leq i \leq q_{\text{leak}} - 1$, but also include the couple $(\text{pk}_{L,*}, \text{pk}_{R,*})$,

and right elements.

Using the same reasoning as with $[\text{IND-CK-CCA}, -]$ security, we obtain that

$$\left| \text{Adv}^{G_0}(\mathcal{A}) - \text{Adv}^{G_5}(\mathcal{A}) \right| \leq 2q_{\text{rej}}2^{-\nu/2} + 16(q_{\text{leak}} + 1)\delta(q_{\text{rand}} + q_{\text{CCA}} + 1)^2.$$

Game G_6 . The right challenge message $m_{R,*}$ is used by the two random oracles H_{rand} and H_{key} . In order to use the One-Way to Hiding Lemma 3, we modify them so that the message $m_{R,*}$ will be asked to only one random oracle.

We introduce an internal random oracle $\underline{\text{H}}_{\text{OWTH}}$ that, on input $(\text{pk}_L, \text{pk}_R, m_L, m_R, c_C, c_L, c_R^{\text{OTP}})$ (same input set as H_{key}), outputs a triplet $((r'_L, r'_R, r'_C), k')$ where (r'_C, r'_L, r'_R) only depends on $(\text{pk}_L, \text{pk}_R, m_L, m_R)$ (changing the other arguments $(c_C, c_L, c_R^{\text{OTP}})$ does not change its value).

Then, in G_6 , we use $\underline{\text{H}}_{\text{OWTH}}$ to simulate the oracles H_{rand} and H_{key} . For example, if the adversary makes a query to H_{rand} with input $(\text{pk}_L, \text{pk}_R, m_L, m_R)$, we add any three values $c_C, c_L, c_R^{\text{OTP}}$ to them, query $((r'_C, r'_L, r'_R), c'_C, k') = \underline{\text{H}}_{\text{OWTH}}(\text{pk}_L, \text{pk}_R, m_L, m_R, c_C, c_L, c_R^{\text{OTP}})$ and output (r'_C, r'_L, r'_R) .

The distribution of any element is unchanged, thus the advantage remains unchanged.

Game G_7 . In this game, we decouple the values of k_0 and $(r_{C,*}, r_{L,*}, r_{R,*})$, usually computed as $k_0 = \text{H}_{\text{key}}(\text{pk}_{L,*}, \text{pk}_{R,*}, m_{L,*}, m_{R,*}, c_{C,*}, c_{L,*}, c_{R,*}^{\text{OTP}})$ and $\text{H}_{\text{rand}}(\text{pk}_{L,*}, \text{pk}_{R,*}, m_{L,*}, m_{R,*})$, from the oracles H_{key} and H_{rand} .

Let us consider the set $\mathcal{X} = \text{PK}_L \times \text{PK}_R \times \mathcal{M}_L \times \mathcal{M}_R \times \text{Ciphers}_C \times \text{Ciphers}_L \times \text{Ciphers}_R$ and $\mathcal{S}$ the subset of elements of $\mathcal{X}$ where the right message is equal to the challenge right message $m_{R,*}$. In G_7 , we reprogram $\underline{\text{H}}_{\text{OWTH}}$ on any value of $\mathcal{S}$ with uniform values.

As a result, in G_7 , $(r_{C,*}, r_{L,*}, r_{R,*})$, k_0 and $c_{R,*}^{\text{OTP}}$ have a uniform distribution, independent of the outputs of $\mathcal{H}_{\text{OWTH}}$, and thus independent of any value accessible by $\mathcal{A}$. Using [24, Lemma 5.2], we have:

$$\left| \text{Adv}^{G_6}(\mathcal{A}) - \text{Adv}^{G_7}(\mathcal{A}) \right| \leq 2\sqrt{(\hat{q} + 1) \Pr[b \leftarrow \mathcal{C}^{|\mathcal{H}_{\text{key}} \setminus \mathcal{S}\rangle}(m_{L,*}) : \text{FIND}]}, \quad (10)$$

where:

- $\hat{q} = q_{\text{rand}} + q_{\text{key}} + q_{\text{CCA}}$.
- $\mathcal{C}$ runs the game G_7 with adversary $\mathcal{A}$ and punctured oracle $|\mathcal{H}_{\text{OWTH}} \setminus \mathcal{S}\rangle$ instead of $|\mathcal{H}_{\text{OWTH}}\rangle$. Moreover, $m_{R,*}$ is uniformly sampled in $\mathcal{M}_R$.

Note that the distribution of $\mathcal{S}$ is dependent of the input $m_{R,*}$. Thus, we cannot use Lemma 4 to bound $\Pr[b \leftarrow \mathcal{C}^{|\mathcal{H}_{\text{OWTH}} \setminus \mathcal{S}\rangle}(m_{R,*}) : \text{FIND}]$.

In order to bound $\Pr[b \leftarrow \mathcal{C}^{|\mathcal{H}_{\text{OWTH}} \setminus \mathcal{S}\rangle}(z) : \text{FIND}]$, we introduce a new game G_8 whose only difference with G_7 is that $c_{R,*}$ is computed as $\text{Encr}_R(\text{pk}_{L,*}, m_{R,\text{Fixed}}, r_{L,*})$ instead of $\text{Encr}_R(\text{pk}_{L,*}, m_{R,*}, r_{L,*})$.

We note $\mathcal{D}$ the algorithm with no input, that runs the game G_8 with adversary $\mathcal{A}$ and with the punctured oracle $|\mathcal{H}_{\text{OWTH}} \setminus \mathcal{S}\rangle$.

There is clearly independence between the (empty) input of $\mathcal{D}$ and the set $\mathcal{S}$, thus we can use the Lemma 4 to obtain:

$$\Pr[\mathcal{D}^{|\mathcal{H}_{\text{OWTH}} \setminus \mathcal{S}\rangle}() : \text{FIND}] \leq 4\hat{q}|\mathcal{S}|/|X| = 4\hat{q}/|\mathcal{M}_R|, \quad (11)$$

where $\hat{q} = q_{\text{rand}} + q_{\text{key}} + q_{\text{CCA}}$.

Now, we study the difference of advantages between G_7 and G_8 . We will bound it by the advantage of an adversary $\mathcal{E} = (\mathcal{E}_1, \mathcal{E}_2)$ against the $[-, \text{IND-CPA}]$ game of DPKE.

First, $\mathcal{E}_1$ obtains public parameters and keys $\text{pp}, \text{pk}_{L,*}, \text{sk}_{L,*}, \text{pk}_{R,*}$, then gives $(m_{R,\text{Fixed}}, m_{R,*}, m_{L,*})$ to the challenger, $m_{R,*}$ and $m_{L,*}$ being uniformly sampled and $m_{R,\text{Fixed}}$ being any fixed right message. We denote by $\tilde{b}$ the bit sampled by the challenger of the $[-, \text{IND-CPA}]$ game of DPKE to choose to encrypt $(m_{L,*}, m_{R,\text{Fixed}})$ (if $\tilde{b} = 0$) or $(m_{L,*}, m_{R,*})$ (if $\tilde{b} = 1$).

Then, $\mathcal{E}_2$ receives $(c_{C,*}, c_{L,*}, c_{R,*})$ and needs to decide if it is an encryption of $(m_{L,*}, m_{R,\text{Fixed}})$ or $(m_{L,*}, m_{R,*})$. $\mathcal{E}_2$ uses $(\text{pp}, \text{pk}_{L,*}, \text{sk}_{L,*}, c_{C,*}, c_{L,*}, c_{R,*})$ to make a simulation of G_7 to $\mathcal{A}$, this simulation including the simulation of each the random oracles. However, $\mathcal{E}_2$ uses $|\mathcal{H}_{\text{OWTH}} \setminus \mathcal{S}\rangle$ instead of $|\mathcal{H}_{\text{OWTH}}\rangle$.

Finally, $\mathcal{E}_2$ outputs 1 if FIND happens during the simulation, and 0 else.

By definition of $\mathcal{E}$, G_7 and G_8 we see that, until FIND happens (if it happens):

- $\mathcal{E}$ make a perfect simulation of $\mathcal{C}$ (with the punctured oracle) if $\tilde{b} = 1$.
- $\mathcal{E}$ make a perfect simulation of $\mathcal{D}$ (with the punctured oracle) if $\tilde{b} = 0$.

We thus have:

$$\begin{aligned} \Pr[\mathcal{C}^{|\mathcal{H}_{\text{OWTH}} \setminus \mathcal{S}\rangle}(m_{L,*}) : \text{FIND}] &\leq 2 \text{Adv}^{[-, \text{IND-CPA}]}(\mathcal{E}) \\ &\quad + \Pr[\mathcal{D}^{|\mathcal{H}_{\text{OWTH}} \setminus \mathcal{S}\rangle}() : \text{FIND}]. \end{aligned} \quad (12)$$

Using Equations (12) and (11), we obtain:

$$\Pr\left[\mathcal{C}^{\mathcal{H}_{\text{OWTH}} \setminus \mathcal{S}}(m_{L,*}) : \text{FIND}\right] \leq 2\text{Adv}^{\text{IND-CPA}}(\mathcal{B}) + 4\hat{q}/|\mathcal{M}_R|, \quad (13)$$

where $\hat{q} = q_{\text{rand}} + q_{\text{key}} + q_{\text{CCA}}$.

Finally, we note that in $\mathcal{G}_8$, both k_0 and k_1 have an independent uniform distribution, and thus, $\text{Adv}^{\mathcal{G}_7}(\mathcal{A}) = 0$.

Using this as well as Equations (10) and (13), we have:

$$\text{Adv}^{\mathcal{G}_7}(\mathcal{A}) \leq 2\sqrt{(\hat{q} + 1) \left(2\text{Adv}^{[-, \text{IND-CPA}]}(\mathcal{B}) + 4\hat{q}/|\mathcal{M}_R| \right)}.$$

where $\hat{q} = q_{\text{rand}} + q_{\text{key}} + q_{\text{CCA}}$. This concludes the proof.

C The eCK Model and its Variants

Sections C.1 and C.2 introduce a generalization of the eCK model [41], which is then used in Section C.3 to redefine the eCK^w and eCK-PFS models from [18]. Finally, Section C.4 presents and justifies the unilateral variants of eCK^w and eCK-PFS.

C.1 Modelization of Protocol, Participants and Adversarial Queries

Preliminary Notations about Finite Sequences. For any set U , denote by $\text{FiniteSequences}(U)$ the set of finite sequences of elements from U , which includes the empty sequence ϵ_{seq} . The length of a finite sequence s —its number of elements—is denoted by $|s|$. For a set U , a finite sequence $s = (s_i)_{i \leq n} \in \text{FiniteSequences}(U)$, and an element $u \in U$, define $\text{AddToSequence}(s, u)$ as the sequence $(u_i)_{i \leq n+1}$ where $s_{n+1} = u$.

Parties and Sessions. In the eCK model, parties represent the participants of the protocol. Each party P has a role $P_{\text{Role}} \in \{\text{Init}, \text{Resp}\}$ and possesses a pair of static public and secret keys $(P_{\text{spk}}, P_{\text{ssk}})$. These keys are generated by the algorithm $\text{Keygen}_{\text{static}}(\text{pp}, \text{Role})$ upon the party's creation, where the static public parameters pp are initially created by an algorithm $\text{Setup}_{\text{static}}$. Each party also has a unique identity, ensuring that no two parties have the same identity.

Each party can be associated to multiple instances of a protocol. Formally, a session s is defined as a tuple

$$\begin{aligned} &(\text{InternId}, \text{Actor}, \text{Peer}, \text{SessionKey}, \text{Sent}, \text{Rec}, \text{Eph}, \text{ProtocolState}, \text{RunState}) \\ &\in \mathbb{N} \times \text{SetParties}^2 \times (\text{SetSessionKeys} \sqcup \{\perp\}) \times (\text{FiniteSequences}(\{0, 1\}^*))^3 \\ &\times \{0, 1\}^* \times \{\text{running}, \text{completed}, \text{fail}\}, \end{aligned}$$

where:

- **InternId** is an integer that uniquely identifies the session among other sessions, it is not related to the *sid* used in BPR model [7].
- **Actor** is a party called the actor of the session, it represents the party that is running the instance of the protocol.
- **Peer** is a party called the peer of the session whom the actor thinks, sometimes incorrectly, it is interacting with.
- **SessionKey** is the session key computed by the actor. It is either undefined and equal to $\perp$, or defined and belongs in the set **SetSessionKeys**.
- **Sent**, **Rec** respectively denote the finite sequence of timely ordered messages sent and received by the **Actor** in this session.
- **Eph** denotes the sequence of elements that may be leaked to the adversary via the **eph-key** query defined below.²⁵ For instance, **Eph** could contain all randomness used by the protocol that is not securely erased.
- **ProtocolState** is an internal state needed to run the protocol. It contains any elements that can be needed for further computations. The following subsection provides a detailed explanation of how the protocol is modeled.
- **RunState** is the running state of the session. It indicates if the session is running (**running**), complete (**completed**) or if some error occurred (**fail**).

We will note:

$$(\mathbf{SInternId}, \mathbf{SActor}, \mathbf{SPeer}, \mathbf{Ssent}, \mathbf{SRec}, \mathbf{SSessionKey}, \mathbf{SEph}, \mathbf{SProtocolState}, \mathbf{SRunState})$$

the attributes of a session s .

We call initiator (*resp.* responder) a party whose role is initiator (*resp.* responder). We call initiator session (*resp.* responder session) a session whose actor is an initiator (*resp.* responder).

As this will be shown by the definition of adversarial queries—more precisely, the **send** query—it is important to note that the elements received by a session will be decided by the adversary, whereas the elements created by a session will be computed according to the protocol’s definition.

Protocol. A protocol is modeled as the tuple $\Pi = (\pi, \text{Setup}_{\text{static}}, \text{Keygen}_{\text{static}})$, where $\text{Setup}_{\text{static}}$ and $\text{Keygen}_{\text{static}}$ are two probabilistic algorithms we have previously described, and π is a probabilistic algorithm that represents the computations performed by a party during a protocol execution. The algorithm π takes the following input:

$$(\text{ProtocolState}, m, \text{actor}, \text{peerId}, \text{peerSpk}, \text{Eph}) \in (\{0, 1\}^*)^2 \times \text{SetParties} \\ \times (\{0, 1\}^*)^2 \times \text{FiniteSequences}(\{0, 1\}^*)$$

²⁵ Our definition is more general than the original **eCK** model (and its variants **eCK^w** and **eCK-PFS**), since it allows the protocol designer to specify which elements may be leaked through **eph-key**. In contrast, the original model fixes this query to reveal exactly the ephemeral secret keys (i.e., the randomness coins) of session s . The choice of which elements can be leaked is significant and may have non-trivial consequences, as illustrated in [17].

and returns a tuple:

$$(\text{RunState}', \text{ProtocolState}', m', k^{\text{SESSION}}, \text{Eph}') \in \{\text{running}, \text{completed}, \text{fail}\} \\ \times \{0, 1\}^* \times \{0, 1\}^* \times (\text{SetSessionKeys} \sqcup \{\perp\}) \times \text{FiniteSequences}(\{0, 1\}^*),$$

where:

- ProtocolState and $\text{ProtocolState}'$ represent the internal state of the protocol.
- RunState indicates the resulting running state, *i.e.*, whether after this computation, the session will continue running, be complete, or be subject to an error²⁶.
- actor is the party which performs the computations—the actor. In particular, its static public and secret keys and its identity can be used in the computation.
- peerID and peerSpk represent the identity and static public key of the peer, *i.e.*, of the party the actor presumes it is interacting with.
- m is the received message.
- m' is the subsequent message to be sent.
- k^{SESSION} is the session key computed by the protocol. We suppose that $k^{\text{SESSION}} \neq \perp$ if, and only if, $\text{RunState}' = \text{completed}$.
- Eph' denotes the updated sequence of elements that may be leaked to the adversary through the eph -key query. For instance, Eph could contain all randomness used by the protocol that is not securely erased.

Adversarial Capabilities. The adversary can control the actions of any party through multiple queries. Additionally, it can obtain certain information and perform a real-or-random test with the session key of a session of its choice—this session is thus referred to as the test session.

Let $\text{cpt}_{\text{party}}$ and $\text{cpt}_{\text{session}}$ be global counters that track the number of created parties and session—initialized to 0. Let b_{qtest} and s^{Test} be variables both initialized to $\perp$. Here are the adversarial capabilities:

1. $\text{activate-party}(\text{Id}, \text{Role})$, with $\text{Id} \in \text{SetIds}$, $\text{Role} \in \{\text{Init}, \text{Resp}\}$, models the creation of a party. More precisely, if activate-party has already been used to create a party with identifier Id , it returns $\perp$. Else, it computes $(\text{spk}, \text{ssk}) \leftarrow \text{Keygen}_{\text{static}}(\text{pp})$, creates party $P = (\text{cpt}_{\text{party}}, \text{Id}, \text{Role}, \text{spk}, \text{ssk})$ and $\text{cpt}_{\text{party}}$ is incremented by one. Finally, it returns $(s_{\text{InternId}} (= \text{cpt}_{\text{session}} - 1), \text{spk})$ and spk . Party P is said to be activated.
2. $\text{activate-session}(\text{Id}, \text{Id}')$ models the creation of a session between the activated parties with identities Id —the actor—and Id' —the peer. More precisely, if one of these parties does not exist, it returns $\perp$. Else, it creates $s = (\text{cpt}_{\text{session}}, P[1], P[2], \epsilon, \epsilon_{\text{seq}}, \epsilon_{\text{seq}}, \epsilon, \text{running})$ where $P[1], P[2]$ are the parties with respective identities Id, Id' and $\text{cpt}_{\text{session}}$ is incremented by one. Finally, it returns $s_{\text{InternId}} (= \text{cpt}_{\text{session}} - 1)$. Session s is said to be activated.

²⁶ Note that we do not include a running state as input, because π will only be used for sessions whose running state is *running*. However, the output must indicate the resulting running state.

3. `send(InternId, m)` models the sending of message m to an activated session with internal identifier `InternId`. More precisely, if such session does not exist, it returns $\perp$. Else, s_{Rec} is set to `AddToSequence(m, Rec)`, and:
 - If $\text{RunState} \in \{\text{fail}, \text{completed}\}$, nothing is done, and it returns $\perp$.
 - Otherwise, it computes:

$$(\text{RunState}', \text{ProtocolState}', m', k^{\text{SESSION}}, \text{Eph}') = \pi(s_{\text{ProtocolState}}, m, s_{\text{Actor}}, (s_{\text{Actor}})_{\text{Id}}, (s_{\text{Peer}})_{\text{spk}}, s_{\text{Eph}}).$$

Then:

- If $\text{ProtocolState}' = \text{fail}$, s_{RunState} is set to `fail` and `send` returns `fail`.
- If $\text{ProtocolState}' = \text{running}$, $s_{\text{ProtocolState}}$ is set to state' , s_{Eph} to Eph' , s_{Sent} to `AddToSequence(sSent, m')` and `send` returns m' .
- If $\text{ProtocolState}' = \text{completed}$, RunState is set to `completed`, Eph to Eph' , SessionKey to k^{SESSION} and `send` returns `(completed, m')`.

Note that in an AKE, the initiator does not need an input message to compute its first message: this is modeled by using `send` with the empty message ϵ .

4. `corrupt(P)`. This query reveals the long-term secret keys of party P by returning P_{ssk} .
5. `eph-key(s)`. If $s_{\text{RunState}} = \text{fail}$, the query returns $\perp$. Otherwise, it outputs the sequence Eph of elements designated as leakable to the adversary (*e.g.*, all randomness used by the protocol that is not securely erased).
6. `session-key(s)`. This query returns $\perp$ if the session s is not complete—*i.e.* $s_{\text{RunState}} \neq \text{completed}$. Else, it returns its session key $s_{\text{SessionKey}}$.
7. `test-session(s)`. This query returns $\perp$ if the session s is not complete—*i.e.* $s_{\text{RunState}} \neq \text{completed}$ —or if $s^{\text{Test}} \neq \perp$. Else, s^{Test} is set to s and b_{qtest} is initialized to a uniform bit $b_{\text{qtest}} \leftarrow_{\$} \{0, 1\}$. Then:
 - If $b_{\text{qtest}} = 0$, $s_{\text{SessionKey}}^{\text{Test}}$ is returned.
 - If $b_{\text{qtest}} = 1$, a key $k \leftarrow_{\$} \text{SetSessionKeys}$ is computed and returned.

Matching Sessions and Origin Session. The notion of matching sessions specifies when two sessions are supposed to be intended communication partners. Here we formalize the matching session definition from the eCK model [41] which is based on matching conversations.

Two completed sessions s and s' are said to be matching if their respective actors have distinct roles and if, in addition: $s_{\text{Actor}} = s'_{\text{Peer}} \wedge s_{\text{Peer}} = s'_{\text{Actor}} \wedge s_{\text{Sent}} = s'_{\text{Rec}} \wedge s_{\text{Rec}} = s'_{\text{Sent}}$.

To relate a message received (and accepted) by some session to the session it originates from (if the later exists), [18] introduced the concept of origin sessions, a possibly not complete session s' is an origin session for a completed session s when $s'_{\text{Sent}} = s_{\text{Rec}}$. If an origin session s' for some session s exists, then the messages received by session s have not been modified or injected (as in Krawczyk's PFS attack [39]) by the adversary.

C.2 Correctness, Key Secrecy and Freshness predicates

Definition 3 (Correctness of the protocol). *Let $\delta > 0$. The protocol is considered δ -correct if the probability that two sessions executing the protocol together—without any modification of the messages by an adversary—either do not complete or fail to agree on the same session key is upper bounded by δ .*

In order to measure the Key Secrecy of the protocol, we will consider a real-or-random game where the adversary tries to distinguish whether the session key given by the query `test-session` is $s_{\text{SessionKey}}^{\text{Test}}$ or has been uniformly sampled.

Note that an adversary capable of performing the above queries can effortlessly reveal the session key of all sessions with `session-key`, including $s_{\text{SessionKey}}^{\text{Test}}$, thus easily winning this real-or-random game.

To obtain a sound notion of security, the powers of the adversary need to be limited. These limitations are defined by a predicate F , called freshness predicate, that takes as input the transcript of queries made by the adversary²⁷ and returns true or false. What defines a variant of eCK is the choice of freshness predicates. When, in an instance of the game, the limitations are not respected, we say that the instance of the game is invalid.

Note that for F and F' two freshness predicates, we can naturally define freshness predicates $F \wedge F'$, $F \vee F'$ and $\neg F$ (also denoted $\overline{F}$).

Let $\Pi = (\pi, \text{Keygen}_{\text{static}})$ a protocol, F a freshness predicate and $\mathcal{A}$ an adversary that can make queries to `send`, `corrupt`, `eph-key`, `session-key`, `test-session` and returns a bit $b \in \{0, 1\}$. The key secrecy game $F\text{-KeySec}$ for this protocol and adversary is defined in Figure 15. Its advantage is defined by:

$$\begin{aligned} \text{Adv}_{\Pi}^{F\text{-KeySec}}(\mathcal{A}) &= \left| \Pr[F\text{-KeySec}(\mathcal{A}) = 1] - \frac{1}{2} \right| \\ &= \frac{1}{2} \left| \Pr[1 \leftarrow \mathcal{A} \wedge F(\text{tr}) \mid b_{\text{qtest}} \text{ defined and } b_{\text{qtest}} = 0] \right. \\ &\quad \left. - \Pr[1 \leftarrow \mathcal{A} \wedge F(\text{tr}) \mid b_{\text{qtest}} \text{ defined and } b_{\text{qtest}} = 1] \right|. \end{aligned}$$

where b_{qtest} is defined if and only $s^{\text{Test}} \neq \perp$.

A game instance where $E_{\text{ValidInstance}}$ is true will be referred to as the valid instance, while all other instances will be termed invalid instances. A valid instance is thus an instance such that the adversary used `test-session` on a completed session and that $F(\text{tr})$ is true. Note that for an invalid instance, a random bit is returned instead of the adversary's result.

C.3 Definition of eCK^w and eCK-PFS Models

Simple Freshness Predicates. We define multiple freshness predicates that will be used to define the freshness predicates of eCK^w, eCK-PFS and their unilateral adaptations u-eCK^w, u-eCK-PFS.

²⁷ We include the arguments and outputs of the queries

```

 $F\text{-KeySec}(\mathcal{A})$ 


---


 $\mathcal{O} = \{\text{activate-party}, \text{activate-session}, \text{send}, \text{corrupt}, \text{eph-key}, \text{session-key}, \text{test-session}\}$ 
 $E_{\text{ValidInstance}} \leftarrow \text{true}$ 
 $b \leftarrow \mathcal{A}^{\mathcal{O}}()$ 
 $\text{tr} \leftarrow \text{transcript of requests made by } \mathcal{A}$ 
if  $s^{\text{Test}} = \perp$  then  $E_{\text{ValidInstance}} \leftarrow \text{false}$ 
   $\quad // s^{\text{Test}}$  is the test session, set by the query test-session,
   $\quad // s^{\text{Test}} = \perp$  happens when test-session has not be used to a completed session
if  $F(\text{tr}) = \text{false}$  then  $E_{\text{ValidInstance}} \leftarrow \text{false}$ 
if  $E_{\text{ValidInstance}} = \text{false}$  then
   $u \leftarrow_{\$} \{0, 1\}$ 
  return  $u$ 
return  $b = b_{\text{qtest}}$ 
   $\quad // b_{\text{qtest}}$  is the bit sampled by the query to test-session

```

Fig. 15: Security game $F\text{-KeySec}(\mathcal{A})$.

Before that, we introduce the following notations.

For a session s , s_{ActorId} (*resp.* s_{PeerId}) denotes the identity of the actor (*resp.* peer) of the session s and $s_{\text{ActorRole}}$ (*resp.* s_{PeerRole}) denotes its role. For a session s , s_{Init} (*resp.* s_{Resp}) denotes s_{Actor} if $s_{\text{ActorRole}} = \text{Init}$ (*resp.* $s_{\text{ActorRole}} = \text{Resp}$) and s_{Peer} otherwise.

For $X \in \mathbf{A}, \mathbf{P}, \mathbf{I}, \mathbf{R}$, we define $\text{party}(X)$ as $s_{\text{Actor}}^{\text{Test}}$, $s_{\text{Peer}}^{\text{Test}}$, $s_{\text{Init}}^{\text{Test}}$, and $s_{\text{Resp}}^{\text{Test}}$, respectively—where s^{Test} denotes the test session. For $Y \in \mathbf{Match}, \mathbf{Orig}$, we let $\text{Set}(Y)$ denote the set of sessions that are, respectively, matching or origin sessions of s^{Test} . Using these notations, we define the following freshness predicates:

- Y is true if, and only if, $\text{Set}(Y)$ is non-empty.
- $\text{Cor}X$ is true if, and only if, $\text{party}(X)$ is corrupted.
- $\text{EmptySsk}(X)$ is true if, and only if, the static secret-key of $\text{party}(X)$ is empty.
- AisInit is true if, and only if, the role of $\text{party}(A)$ is `Init`.
- AisResp is true if, and only if, the role of $\text{party}(A)$ is `Resp`.
- SkT is true if, and only if, the query `session-key`(s^{Test}) is asked by the adversary.
- SkM is true if, and only if, there exists a session s matching the test session such that `session-key`(s) is asked by the adversary.
- LeakRandT is true if, and only if, the query `eph-key`(s^{Test}) is asked by the adversary.
- LeakRandO is true if, and only if, the query `eph-key`(s) is asked by the adversary to an origin session s of s^{Test} .
- corXbeforeT (*resp.* corXafterT) is true if, and only if, $\text{party}(X)$ is corrupted before (*resp.* corrupted after) s^{Test} is complete.

Definitions of eCK^w and eCK-PFS. Using the freshness predicate defined in the previous section, we set:

$$\text{CorLeak} = (\overline{\text{CorP}} \vee \overline{\text{LeakRandO}}) \wedge (\overline{\text{CorA}} \vee \overline{\text{LeakRandT}}),$$

and

$$\begin{aligned} \text{eCK}^w &= \overline{\text{SkT}} \wedge \overline{\text{SkM}} \wedge \text{CorLeak} \wedge (\overline{\text{Orig}} \Rightarrow \overline{\text{CorP}}), \\ \text{eCK-PFS} &= \overline{\text{SkT}} \wedge \overline{\text{SkM}} \wedge \text{CorLeak} \wedge (\overline{\text{Orig}} \Rightarrow \overline{\text{corPbeforeT}}). \end{aligned}$$

These definitions correspond to the attack scenarios described in Section 3, Table 4. Indeed, if we define the five following predicates:

$$\begin{aligned} F_1 &= \overline{\text{CorA}} \wedge \overline{\text{CorP}} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}, \\ F_2 &= \text{CorA} \wedge \overline{\text{CorP}} \wedge \overline{\text{LeakRandT}} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}, \\ F_3 &= \overline{\text{CorA}} \wedge \text{CorP} \wedge \text{Orig} \wedge \overline{\text{LeakRandO}} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}, \\ F_4 &= \text{CorA} \wedge \text{CorP} \wedge \overline{\text{LeakRandT}} \wedge \text{Orig} \wedge \overline{\text{LeakRandO}} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}, \\ F_5 &= \text{CorA} \wedge \text{corPafterT} \wedge \overline{\text{LeakRandT}} \wedge \overline{\text{Orig}} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}, \\ F_6 &= \overline{\text{CorA}} \wedge \text{corPafterT} \wedge \overline{\text{Orig}} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}. \end{aligned} \tag{14}$$

We can see that they are mutually disjoint and that:

$$\text{eCK}^w = \bigvee_{i=1}^4 F_i, \quad \text{eCK-PFS} = \bigvee_{i=1}^6 F_i, \tag{15}$$

which implies:

$$\text{Adv}_{\Pi}^{\text{eCK}^w\text{-KeySec}} \leq \sum_{i=1}^4 \text{Adv}_{\Pi}^{F_i\text{-KeySec}}, \quad \text{Adv}_{\Pi}^{\text{eCK-PFS-KeySec}} \leq \sum_{i=1}^6 \text{Adv}_{\Pi}^{F_i\text{-KeySec}}.$$

C.4 Unilateral Versions of eCK^w (u-eCK^w) and eCK-PFS (u-eCK-PFS)

A protocol is called *unilateral* when the parties of a given role have no static secrets. More formally, for $\text{Role} \in \{\text{Init}, \text{Resp}\}$, we say that a protocol is Role -unilateral if, by denoting Role^* the other role and $\epsilon \in \{0, 1\}^*$ the empty string, we have,

$$\begin{aligned} \Pr[\text{ssk} \neq \epsilon \mid \text{pp} \leftarrow \text{Setup}_{\text{static}}(), (\text{spk}, \text{ssk}) \leftarrow \text{Keygen}_{\text{static}}(\text{pp}, \text{Role})] &= 1, \\ \Pr[\text{ssk} = \epsilon \mid \text{pp} \leftarrow \text{Setup}_{\text{static}}(), (\text{spk}, \text{ssk}) \leftarrow \text{Keygen}_{\text{static}}(\text{pp}, \text{Role}^*)] &= 1. \end{aligned}$$

A protocol is unilateral if there is a role Role such that it is Role -unilateral.

The models eCK^w, eCK-PFS we defined are not suited for unilateral protocols. Indeed, in a Role -unilateral protocol, the adversary can impersonate parties with role Role^* as they do not have secret static keys. Therefore, it is necessary

to impose additional restrictions on the eCK^w and eCK-PFS models to eliminate unavoidable attacks arising from their unilateral nature. This modification results in the development of the refined models u-eCK^w and u-eCK-PFS .

More precisely, we define variants of eCK^w and eCK-PFS as $\text{u-eCK}^w = \text{eCK}^w \wedge F_{\text{unilateral}}$ and $\text{u-eCK-PFS} = \text{eCK-PFS} \wedge F_{\text{unilateral}}$, where the predicate $F_{\text{unilateral}}$ is given by:

$$(\text{EmptySsk}(P) \Rightarrow (\text{PartialOrig} \wedge \overline{\text{LeakRandO}})) \wedge (\text{EmptySsk}(A) \Rightarrow \overline{\text{LeakRandT}}).$$

Here, PartialOrig is true if, and only if, there exists a partial origin session of s^{Test} . The notion of partial origin is a variant of the notion of origin session: a possibly not complete session s' is a partial origin session for a possibly not complete session s if one of the messages received by s is equal to one of the messages sent by s' .

We note that $F_{\text{unilateral}}$ is false if, and only if, at least one of the unavoidable attacks can occur. Specifically:

- If $\text{EmptySsk}(P)$ is true but PartialOrig is false, there is nothing to prevent the adversary from impersonating a session where P is the actor, allowing the adversary to conduct the protocol with s^{Test} .
- If $\text{EmptySsk}(P)$ and PartialOrig are true but $\overline{\text{LeakRandO}}$ is false, the adversary knows the randomness of the first origin session s' of s^{Test} . This randomness is the only secret protecting s' because its actor, the peer of s^{Test} , does not have a static secret.
- If $\text{EmptySsk}(A)$ is true but $\overline{\text{LeakRandT}}$ is false, the adversary knows the randomness of the test session s^{Test} , which is the sole secret protecting this session since its actor does not have a static secret.

u-eCK^w and u-eCK-PFS as the Disjunction of Simpler Cases. The following proposition shows how to simplify the analysis of Init-unilateral protocols for which the responder sends only one message per session by dividing the analysis in smaller cases.

Proposition 1. *Fix any Init-unilateral protocol such that any responder sends only one message per session. We consider the following predicates:*

$$\begin{aligned} F_1^U &= \text{AisInit} \wedge \overline{\text{Corl}} \wedge \overline{\text{CorR}} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}} \wedge \overline{\text{LeakRandO}} \wedge \text{Orig}, \\ F_2^U &= \text{AisResp} \wedge \overline{\text{Corl}} \wedge \overline{\text{CorR}} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}} \wedge \overline{\text{LeakRandT}}, \\ F_3^U &= \text{AisInit} \wedge \text{Corl} \wedge \overline{\text{CorR}} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}} \wedge \overline{\text{LeakRandT}} \wedge \overline{\text{LeakRandO}} \wedge \text{Orig}, \\ F_4^U &= \text{AisResp} \wedge \text{Corl} \wedge \overline{\text{CorR}} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}} \wedge \overline{\text{LeakRandT}} \wedge \overline{\text{LeakRandO}} \wedge \text{Orig}, \\ F_5^U &= \text{AisResp} \wedge \text{corlaterT} \wedge \overline{\text{CorR}} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}} \wedge \overline{\text{LeakRandT}} \wedge \overline{\text{Orig}}. \end{aligned}$$

They are disjoint and, for $X \in \{\text{u-eCK}^w, \text{u-eCK-PFS}\}$ and $\mathcal{A}$ an adversary against the X -KeySec security, there exists an adversary $\mathcal{B}$ running with same time as

$\mathcal{A}$ and using at most the same number of requests to each of the available oracles (activate-party, activate-session, eph-key, corrupt, test-session and random oracle) such that:

$$\begin{aligned} \text{Adv}^{\text{u-eCK}^w\text{-KeySec}}(\mathcal{A}) &\leq \sum_{i=1}^4 \text{Adv}^{F_i^U}(\mathcal{A}), \\ \text{Adv}^{\text{u-eCK-PFS-KeySec}}(\mathcal{A}) &\leq \sum_{i=1}^5 \text{Adv}^{F_i^U}(\mathcal{A}). \end{aligned}$$

Proof. The disjunction of these five predicates can be shown by direct comparison. We show the other part of the proposition.

Let $\mathcal{A}$ be an adversary and let $X \in \{\text{u-eCK}^w, \text{u-eCK-PFS}\}$. Since the protocol is Init-unilateral, each use of **corrupt** by $\mathcal{A}$ on a responder does not give more information to $\mathcal{A}$. We construct an adversary $\tilde{\mathcal{A}}$ that acts as $\mathcal{A}$ except for the following:

- each time $\mathcal{A}$ uses **corrupt** on a responder, $\tilde{\mathcal{A}}$ does not use **corrupt**²⁸.
- $\tilde{\mathcal{A}}$ memorizes all the necessary data to be able to tell at any time if the instance of the game is valid or not.
- when $\mathcal{A}$ outputs a bit, $\tilde{\mathcal{A}}$ checks if the instance of the game is valid or invalid:
 - if valid, $\tilde{\mathcal{A}}$ outputs the same bit that $\mathcal{A}$.
 - if invalid, $\tilde{\mathcal{A}}$ samples and outputs a uniform bit.

We can see that $\tilde{\mathcal{A}}$ never uses **corrupt** on a responder, uses the same number of oracle queries for any other cases and that $\text{Adv}^X(\mathcal{A}) = \text{Adv}^X(\tilde{\mathcal{A}})$.

Second step: We now suppose that $\mathcal{A}$ never uses **corrupt** on a responder. We can see that for this adversary, an instance of the game X is valid if, and only if, it is $\tilde{X}$ -valid, where $\tilde{X} = X \wedge \text{CorR}$.

Finally, a direct computation shows that $F_1^U \vee F_2^U \vee F_3^U \vee F_4^U = \widetilde{\text{u-eCK}^w}$ and $F_1^U \vee F_2^U \vee F_3^U \vee F_4^U \vee F_5^U = \widetilde{\text{u-eCK-PFS}}$.

D Detailed Presentation of DAKE^σ and DUAKE

In this section, we provide additional details on DAKE^σ and DUAKE, including their definitions, the elements revealed by **eph-key**, and their security properties.

D.1 DAKE^σ : A Variant of DAKE Using a Signature

DAKE^σ is a variant of DAKE that uses a signature, instead of a KEM and a MAC, to authenticate the responder. It is specified in Figure 16. It comes in two versions: a two-message variant, DAKE_2^σ , which provides weak Perfect Forward Secrecy (wPFS), and a three-message variant, DAKE_3^σ , which provides full PFS.

²⁸ Since the output of **corrupt** is ϵ , $\tilde{\mathcal{A}}$ does not lose any information and can therefore continue to behave as $\mathcal{A}$.

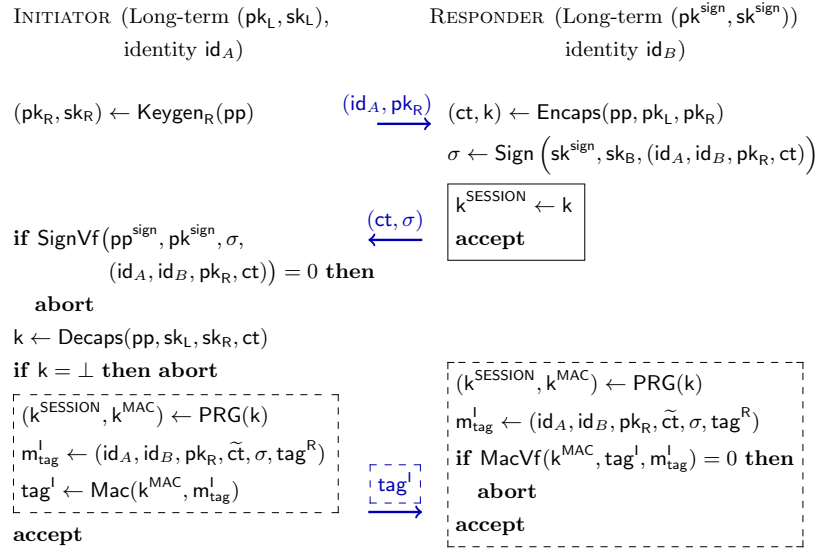


Fig. 16: The DAKE^σ protocol. Public parameters are (1) $(\text{pp}, \text{pp}^{\text{sign}})$ with $\text{pp} \leftarrow \text{Setup}()$ and $\text{pp}^{\text{sign}} \leftarrow \text{Setup}^{\text{sign}}()$, (2) all party identities, and (3) a dictionary mapping identities to public keys. The initiator chooses its peer before sending the first message. DAKE_2^σ (*resp.* DAKE_3^σ) is obtained by removing the dotted (*resp.* solid) boxes.

Leakage through eph-key. Concerning the elements leaked by eph-key:

- Applying `eph-key` to an initiator’s session reveals the randomness used in `KeygenR`, which implies knowledge of `skR`.
- For the responder, nothing is revealed: we assume that the randomness used in both the encapsulation and the signature is securely erased immediately after their computation.

Security properties. Key secrecy is given by the Theorem 7 of Appendix E. Match soundness and inverse-match soundness are studied in Section 3.3. Finally, explicit authentication is studied in Appendix F.

D.2 DUAKE: An Unilateral AKE Fitted for Maul

DUAKE is a variant of DAKE fitted for the unilateral setting, where only the initiator is authenticated. It is given in Figure 2 of Section 1. It has a two-message variant which provides weak Perfect Forward Secrecy (weak PFS) named DUAKE₂ and a three-message variant which provides PFS named DUAKE₃.

Leakage through eph-key. Concerning the elements leaked by eph-key:

- Applying eph-key to an initiator's session reveals the randomness used in Keygen_R , which implies knowledge of sk_R .
- For the responder, nothing is revealed: we assume that the randomness of the encapsulation is securely erased immediately after its computation.

Security properties. Key secrecy is given by the Theorem 8 in Appendix E. It is studied under the u-eCK^w model for the weak PFS and u-eCK-PFS for the PFS. These two models are respectively adaptations of the eCK^w and eCK-PFS models to the unilateral setting, their definitions being given and justified in Section C.4.

Match soundness and inverse-match soundness are studied in Section 3.3. Finally, explicit authentication is studied in Appendix F.

E Key Secrecy of DAKE, DAKE $^\sigma$, and DUAKE

We aim to prove the key secrecy of three protocols, each of which has both a two-message and a three-message variant:

- DAKE, with variants DAKE₂ and DAKE₃, presented in Figure 1 of Section 3. The key secrecy of DAKE₃ is given in Theorem 1, Section 3.3 and the key secrecy of DAKE₂ is given in Theorem 6 below.
- DAKE $^\sigma$, with variants DAKE₂ $^\sigma$ and DAKE₃ $^\sigma$, presented in Figure 16 of Section D.1. Its key secrecy is given in Theorem 7 below.
- DUAKE, with variants DUAKE₂ and DUAKE₃, presented in Figure 2 of Section D.2. Its key secrecy is given in Theorem 8 below.

The remainder of this section is organized as follows: Appendices E.1 and E.2 present the proof blueprint and required notations, while Appendices E.3, E.4, and E.5 contain the full security proof.

Theorem 6 (Key Secrecy of DAKE₂ in the Standard Model). *Let $\mathcal{A}$ be a PPT adversary against key secrecy of DAKE₂. Suppose it activates at most N parties and, for each party P , at most l sessions with actor or peer P . Assume that the underlying 2K-KEM is γ -spread and δ -correct while the standard KEM is δ' -correct. Let Adv^w denote the advantage of $\mathcal{A}$ against eCK^w -KeySec. Then, in the standard model, there exist PPT adversaries $\mathcal{B}_1, \dots, \mathcal{B}_7$ such that:*

$$\text{Adv}^w \leq \alpha + 4\beta + 6Nl^2 \left(\text{Adv}_{\text{dKEM}}^{\text{IND-CK-CCA}, -}(\mathcal{B}_6) + \text{Adv}_{\text{dKEM}}^{[-, \text{IND-CCA}]}(\mathcal{B}_7) \right),$$

where $\alpha = \text{Adv}_{\text{MAC}}^{\text{MacColl}}(\mathcal{B}_1) + N \text{Adv}_{\text{KEM}}^{\text{Key-Recov}}(\mathcal{B}_2) + N^2 l \left(2 \text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{B}_3) + l\delta' + \text{Adv}_{\text{MAC}}^{\text{sUnforg}}(\mathcal{B}_4) \right)$ and $\beta = Nl \text{Adv}_{\text{dKEM}}^{[-, \text{Key-Recov}]}(\mathcal{B}_5) + 2Nl^2 (\delta + 2^{-\gamma})$.

Moreover, (1) $\mathcal{B}_3$ queries $\mathcal{O}_{\text{CCA}}$ at most l times, (2) $\mathcal{B}_4$ queries $\mathcal{O}_{\text{Mac}}$ at most l time. (3) $\mathcal{B}_6$ query $\mathcal{O}_{\text{leak}_R}$ and $\mathcal{O}_{\text{CCA}_L}$ at most l times, (4) $\mathcal{B}_7$ does not query $\mathcal{O}_{\text{leak}_L}$ and queries $\mathcal{O}_{\text{CCA}_R}$ at most one time.

Theorem 7 (Key Secrecy of DAKE^σ in the Standard Model). *Let $\mathcal{A}$ be a PPT adversary against key secrecy of DAKE_3^σ . Suppose it activates at most N parties and, for each party P , at most l sessions with actor or peer P . Assume that the underlying 2K-KEM is γ -spread and δ -correct. Let Adv^w denote the advantage of $\mathcal{A}$ against eCK^w -KeySec and Adv^{PFS} its advantage against eCK -PFS-KeySec. Then, in the standard model, there exist PPT adversaries $\mathcal{B}_1, \dots, \mathcal{B}_{11}$ such that, denoting*

$$\alpha = \text{Adv}_{\text{Sign}}^{\text{MBS}}(\mathcal{B}_1) + \text{Adv}_{\text{Sign}}^{\text{M-S-UEO}}(\mathcal{B}_2) + N \text{Adv}_{\text{Sign}}^{\text{Key-Recov}}(\mathcal{B}_3) + N^2 l \text{Adv}_{\text{Sign}}^{\text{UF-CMA}}(\mathcal{B}_4),$$

and $\beta = Nl \text{Adv}_{\text{dKEM}}^{[\cdot, \text{Key-Recov}]}(\mathcal{B}_5) + 2Nl^2 (\delta + 2^{-\gamma})$, we have:

$$\begin{aligned} \text{Adv}^w &\leq \alpha + 4\beta + 6Nl^2 \left(\text{Adv}_{\text{dKEM}}^{[\text{IND-CK-CCA}, -]}(\mathcal{B}_6) + \text{Adv}_{\text{dKEM}}^{[-, \text{IND-CCA}]}(\mathcal{B}_7) \right. \\ &\quad \left. + 2 \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{B}_8) \right), \\ \text{Adv}^{\text{PFS}} &\leq \text{Adv}^w + \beta + 2Nl^2 \left(2 \text{Adv}_{\text{dKEM}}^{[\text{IND-CK-CCA}, -]}(\mathcal{B}_9) + 2 \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{B}_{10}) \right. \\ &\quad \left. + \text{Adv}_{\text{MAC}}^{\text{sUnforg}}(\mathcal{B}_{11}) \right). \end{aligned}$$

Moreover, (1) $\mathcal{B}_4$ queries $\mathcal{O}_{\text{Sign}}$ at most l times, (2) $\mathcal{B}_6, \mathcal{B}_9$ query $\mathcal{O}_{\text{leak}_R}$ and $\mathcal{O}_{\text{CCA}_L}$ at most l times, (3) $\mathcal{B}_7$ does not query $\mathcal{O}_{\text{leak}_L}$ and queries $\mathcal{O}_{\text{CCA}_R}$ at most one time, and (4) $\mathcal{B}_{11}$ queries $\mathcal{O}_{\text{Mac}}$ at most one time.

We now consider a PPT adversary $\mathcal{A}$ against the key secrecy of DAKE_2^σ . Under the same assumptions as above, and letting Adv_2^w denote its advantage against eCK^w -KeySec, Adv_2^w satisfies the same upper bound as Adv^w , except that the term $2 \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{B}_8)$ is omitted.

Theorem 8 (Key Secrecy of DUAKE in Standard Model). *Let $\mathcal{A}$ be a PPT adversary against key secrecy of DUAKE_3 . Suppose it activates at most N parties and, for each party P , at most l sessions with actor or peer P . Assume that the underlying 2K-KEM is γ -spread and δ -correct. Let Adv^w denote the advantage of $\mathcal{A}$ against eCK^w -KeySec and Adv^{PFS} its advantage against eCK -PFS-KeySec. Then, in the standard model, there exist PPT adversaries $\mathcal{B}_1, \dots, \mathcal{B}_8$ such that, denoting $\beta = Nl \text{Adv}_{\text{dKEM}}^{[\cdot, \text{Key-Recov}]}(\mathcal{B}_1) + 2Nl^2 (\delta + 2^{-\gamma})$, we have:*

$$\begin{aligned} \text{Adv}^w &\leq \text{Adv}_H^{\text{HashColl}}(\mathcal{B}_2) + 4\beta + 6Nl^2 \left(\text{Adv}_{\text{dKEM}}^{[\text{IND-CK-CCA}, -]}(\mathcal{B}_3) + \text{Adv}_{\text{dKEM}}^{[-, \text{IND-CCA}]}(\mathcal{B}_4) \right. \\ &\quad \left. + 2 \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{B}_5) \right), \\ \text{Adv}^{\text{PFS}} &\leq \text{Adv}^w + \beta + 2Nl^2 \left(2 \text{Adv}_{\text{dKEM}}^{[\text{IND-CK-CCA}, -]}(\mathcal{B}_6) + 2 \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{B}_7) \right. \\ &\quad \left. + \text{Adv}_{\text{MAC}}^{\text{sUnforg}}(\mathcal{B}_8) \right). \end{aligned}$$

Moreover, (2) $\mathcal{B}_3, \mathcal{B}_6$ query $\mathcal{O}_{\text{leak}_R}$ and $\mathcal{O}_{\text{CCA}_L}$ at most l times, (3) $\mathcal{B}_4$ does not query $\mathcal{O}_{\text{leak}_L}$ and queries $\mathcal{O}_{\text{CCA}_R}$ at most one time, and (4) $\mathcal{B}_8$ queries $\mathcal{O}_{\text{Mac}}$ at most one time.

We now consider a PPT adversary $\mathcal{A}$ against the key secrecy of DUAKE_2 . Under the same assumptions as above, and letting Adv_2^w denote its advantage against $\text{eCK}^w\text{-KeySec}$, Adv_2^w satisfies the same upper bound as Adv^w , except that the term $2\text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{B}_5)$ is omitted.

E.1 Context, Rationale of the Proof and Notations

In this section we outline the key secrecy proof for DAKE and DAKE^σ . The analysis of DUAKE , which reuses this approach, is given separately in Appendix E.5.

Note that DAKE and DAKE^σ differ only in how the responder is authenticated:

- In DAKE , identification is achieved through a KEM and a MAC.
- In DAKE^σ , identification is achieved through a digital signature.

If we remove the responder's identification step altogether, we obtain the common underlying AKE depicted in Figure 17.

Our goal in this proof is to exploit the structural similarity between DAKE and DAKE^σ . To this end, we will not analyze them directly under the standard attack scenarios of eCK^w and eCK-PFS , expressed by the predicates $F_1, \dots, F_5$ in Equation 14. Instead, we introduce a refined set of predicates, $U, V_1, \dots, V_5$ ²⁹ better suited to leverage the resemblance of the two protocols. Among these, only U requires us to analyze the responder's identification step, which differs between DAKE and DAKE^σ . For all other cases, the proof does not depend on the responder's identification and is therefore identical for both protocols. In fact, the proofs for $V_1, \dots, V_5$ can be carried out using the simplified protocol of Figure 17.

In the remaining of this section, we formally define these new predicates. The detailed analysis is then organized as follows:

- Section E.2 define some necessary notations,
- Section E.3 analyzes the case U ,
- Section E.4 covers the cases $V_1, \dots, V_5$.

Definition of the predicates.

$$\begin{aligned}
 U &= \text{AisInit} \wedge ((\overline{\text{corPbeforeT}} \wedge \overline{\text{FirstMatch}}) \vee (\text{Orig} \wedge \overline{\text{FirstMatch}})) , \\
 V_1 &= \text{AisInit} \wedge \overline{\text{FirstMatch}} \wedge \overline{\text{CorA}} \wedge (\overline{\text{CorP}} \vee \overline{\text{LeakRandO}}) \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}} , \\
 V_2 &= \text{AisResp} \wedge \overline{\text{CorP}} \wedge (\overline{\text{CorA}} \vee \overline{\text{LeakRandT}}) \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}} , \\
 V_3 &= \text{AisInit} \wedge \overline{\text{FirstMatch}} \wedge \overline{\text{CorA}} \wedge \overline{\text{LeakRandT}} \wedge (\overline{\text{CorP}} \vee \overline{\text{LeakRandO}}) \\
 &\quad \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}} , \\
 V_4 &= \text{AisResp} \wedge \overline{\text{CorP}} \wedge \text{Orig} \wedge \overline{\text{LeakRandO}} \wedge (\overline{\text{CorA}} \vee \overline{\text{LeakRandT}}) \\
 &\quad \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}} , \\
 V_5 &= \text{AisResp} \wedge \overline{\text{corPafterT}} \wedge \overline{\text{Orig}} \wedge (\overline{\text{CorA}} \vee \overline{\text{LeakRandT}}) \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}} .
 \end{aligned} \tag{16}$$

²⁹ For accuracy, we do not consider U alone, but rather $U \wedge \text{eCK}^w$ for eCK^w security, and $U \wedge \text{eCK-PFS}$ for eCK-PFS security.

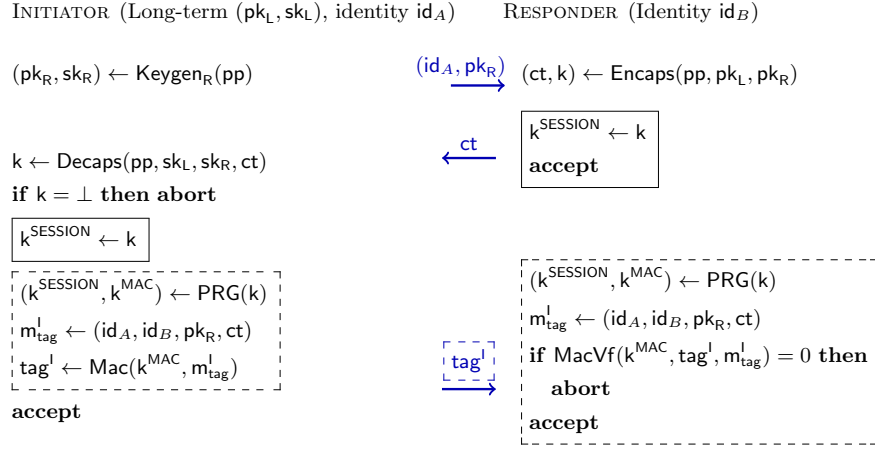


Fig.17: A simplified AKE whose security can be proven under predicates $V_1, \dots, V_5$. It is obtained from DAKE or DAKE $^\sigma$ by removing the components responsible for responder authentication. Instructions within dotted boxes must be removed to obtain the two-message version AKE $_2$.

Here, **FirstMatch** holds if and only if the test session has a first-matching session s , which is defined by the following conditions: (1) the first message send and received by s^{Test} are equal to the first messages received and sent by s , and (2) they have distinct roles and (3) $s_{\text{Actor}}^{\text{Test}} = s_{\text{Peer}} \wedge s_{\text{Actor}} = s_{\text{Peer}}^{\text{Test}}$. Note that in the two-message protocols, if s and s^{Test} are completed and first-matching, they are matching. However, this equivalence no longer holds in the three-message variants.

A direct computation shows that the predicates $U, V_1, \dots, V_5$ are all mutually disjoint and:

$$eCK^w = (U \wedge eCK^w) \vee \left(\bigvee_{i=1}^4 V_i \right) \quad eCK\text{-PFS} = (U \wedge eCK\text{-PFS}) \vee \left(\bigvee_{i=1}^5 V_i \right). \quad (17)$$

It is thus possible to study separately the security of the AKEs for each of these predicates. More precisely, for any of the AKEs AKE and any adversary $\mathcal{A}$, we have:

$$\begin{aligned} \text{Adv}_{\text{AKE}}^{eCK^w}(\mathcal{A}) &\leq \text{Adv}_{\text{AKE}}^{(U \wedge eCK^w)}(\mathcal{A}) + \sum_{i=1}^4 \text{Adv}_{\text{AKE}}^{V_i}(\mathcal{A}), \\ \text{Adv}_{\text{AKE}}^{eCK\text{-PFS}}(\mathcal{A}) &\leq \text{Adv}_{\text{AKE}}^{(U \wedge eCK\text{-PFS})}(\mathcal{A}) + \sum_{i=1}^5 \text{Adv}_{\text{AKE}}^{V_i}(\mathcal{A}). \end{aligned}$$

E.2 Notations Used for the Proofs

First-Origin Session, First-Partner Session and First-Match Session.

A session s is said to be a *first-origin session* of s^{Test} if the first message it created is identical to the first message received by s^{Test} . It is important to distinguish this from an *origin session* because, in the three-message protocol, a first-origin session might not necessarily be an origin session. Making this distinction will highlight how this proof can be readily adapted to the three-messages protocol.

Similarly, a session s is said to be a *first-partner* of s^{Test} if s^{Test} is a first-origin session of s and s is a first-origin session of s^{Test} .

Finally, a session s is said to be a *first-match* of s^{Test} if (1) s^{Test} is a first-partner of s , and (2) they have distinct roles and (3) $s_{\text{Actor}}^{\text{Test}} = s_{\text{Peer}} \wedge s_{\text{Actor}} = s_{\text{Peer}}^{\text{Test}}$.

Elements of a session. Note that for a session s , we will denote the elements computed or received using this convention: $s.k^{\text{SESSION}}$, $s.ct$, $s.pk_R$, $s.sk_R$, *etc.* These values are set to $\perp$ if they have not yet been computed or received. We also recall that s_{ActorId} (*resp.* s_{PeerId}) denotes the identity of the actor (*resp.* peer) of the session s .

E.3 Key Secrecy for Predicates $U \cap \text{eCK}^w$ and $U \cap \text{eCK-PFS}$

Proposition 2 ($((U \cap \text{eCK}^w)\text{-KeySec}$ and $(U \cap \text{eCK-PFS})\text{-KeySec-Security of DAKE}$). *We consider an adversary $\mathcal{A}$ against the Key Secrecy of DAKE₂ or DAKE₃. We suppose that $\mathcal{A}$ activates less than N parties and that for each fixed party P , it activates less than l sessions that have either actor or peer P . We also consider $\delta' \geq 0$ such that $\widetilde{\text{KEM}}$ is δ' -correct. Then, in the standard model, for $X \in \{\text{Adv}_{\text{DAKE}_2}^{U \cap \text{eCK}^w\text{-KeySec}}(\mathcal{A}), \text{Adv}_{\text{DAKE}_3}^{U \cap \text{eCK}^w\text{-KeySec}}(\mathcal{A}), \text{Adv}_{\text{DAKE}_3}^{U \cap \text{eCK-PFS-KeySec}}(\mathcal{A})\}$ there exist adversaries $\mathcal{B}, \mathcal{C}, \mathcal{D}, \mathcal{E}$ such that,*

$$\begin{aligned} X \leq & \text{Adv}_{\text{MAC}}^{\text{MacColl}}(\mathcal{B}) + N \text{Adv}_{\widetilde{\text{KEM}}}^{\text{Key-Recov}}(\mathcal{C}) \\ & + N^2 l \left(2 \text{Adv}_{\widetilde{\text{KEM}}}^{\text{IND-CCA}}(\mathcal{D}) + l \delta' + \text{Adv}_{\text{MAC}}^{\text{sUnforg}}(\mathcal{E}) \right). \end{aligned}$$

Moreover, $\mathcal{D}$ queries $\mathcal{O}_{\text{CCA}}$ at most l times and $\mathcal{E}$ queries $\mathcal{O}_{\text{Mac}}$ at most l times.

Proof. We recall that, as defined in Equation (16) (page 67):

$$U = \text{AisInit} \wedge ((\overline{\text{corPbeforeT}} \wedge \overline{\text{FirstMatch}}) \vee (\text{Orig} \wedge \overline{\text{FirstMatch}})).$$

We will study the $U \cap \text{eCK}^w\text{-KeySec}$ security of DAKE₂, the proof for the $U \cap \text{eCK}^w\text{-KeySec}$ and $U \cap \text{eCK}^w\text{-KeySec}$ security of DAKE₃ being similar.

Note that this demonstration can also be reused to establish Resp-explicit authentication for DAKE₂ and DAKE₃, as explained in the following sidebar.

Reusing this demonstration to establish Resp-explicit authentication.

First, we note that the advantage in Resp-explicit authentication for DAKE₂ and DAKE₃ is exactly the probability that an instance of the Key Secrecy

game satisfies the predicate $\text{AisInit} \wedge \overline{\text{corPbeforeT}} \wedge \overline{\text{FirstMatch}}$ and that the test session completes³⁰. With this formulation, the test session corresponds to the session that the adversary must select in the **Resp**-explicit authentication game.

Using this formulation of the **Resp**-explicit authentication game, we observe that the proof of Proposition 2 establishes **Resp**-explicit authentication for DAKE_2 . Indeed, this proof bounds the probability that an instance of the game satisfies the predicate $\text{AisInit} \wedge \overline{\text{corPbeforeT}} \wedge \overline{\text{FirstMatch}}$ and that the test session completes. Moreover, the proof for DAKE_3 uses exactly the same arguments.

Game G_1 : Origin session implies first match session. We recall that G_0 is the original security game for the AKE and that certain instances of this game can be designated as invalid, which is indicated by the event EValidInstance being false. More precisely, an instance is invalid either if $\text{U} \wedge \text{eCK}^w$ is false or if no session has been tested. For an invalid instance, a random bit is returned instead of the adversary's result.

In G_1 , additional instances are considered invalid. Specifically, any instance is deemed invalid if s^{Test} is an initiator session and if there exists an origin session s^{Origin} of s^{Test} that is not a first matching session of s^{Test} .

Let us suppose that it is true and that s^{Test} completes. Then, this directly leads to an attack against the **MacColl**-security of **MAC**. Indeed, s^{Origin} creates a MAC $\tau \leftarrow \text{Mac}(s^{\text{Origin}}.\tilde{k}, m_1)$ for $m_1 = (s_{\text{PeerId}}^{\text{Origin}}, s_{\text{ActorId}}^{\text{Origin}}, s_{\text{Origin}}.\text{pk}_R, s_{\text{Origin}}.\tilde{\text{ct}}, s_{\text{Origin}}.\text{ct})$. Then, s^{Test} receives τ and verifies that $\text{MacVf}(s^{\text{Test}}.\tilde{k}, \tau, m_2)$ for $m_2 = (s_{\text{ActorId}}^{\text{Test}}, s_{\text{PeerId}}^{\text{Test}}, s^{\text{Test}}.\text{pk}_R, s^{\text{Test}}.\tilde{\text{ct}}, s^{\text{Test}}.\text{ct})$. Moreover, because s^{Origin} is not a first match, we know that $m_1 \neq m_2$. The tuple $(s^{\text{Test}}.\tilde{k}, s^{\text{Origin}}.\tilde{k}, m_1, m_2, \tau)$ is thus a solution of **MacColl**.³¹

We thus have an adversary $\mathcal{B}$ such that:

$$\text{Adv}^{G_0}(\mathcal{A}) \leq \text{Adv}_{\text{MAC}}^{\text{MacColl}}(\mathcal{B}) + \text{Adv}^{G_1}(\mathcal{A}).$$

In the valid instances of the next games, any origin session of s^{Test} will also be a first matching session of s^{Test} .

In particular, we know that for this game and the next ones, the valid instances verify the predicate $\text{AisInit} \wedge \overline{\text{corPbeforeT}} \wedge \overline{\text{Orig}}$.

Game G_2 : Removal of collisions of $\tilde{\text{pk}}$. In G_2 , additional instances are considered invalid. Specifically, any instance is deemed invalid if there exist two distinct parties with role **Resp** that have the same public static key.

This probability can be bounded by $N\text{Adv}_{\text{KEM}}^{\text{Key-Recov}}(\mathcal{B})$ for some adversary $\mathcal{C}$ against the key recovery game of **KEM**. Thus, we have:

$$\left| \text{Adv}^{G_2}(\mathcal{A}) - \text{Adv}^{G_1}(\mathcal{A}) \right| \leq N\text{Adv}_{\text{KEM}}^{\text{Key-Recov}}(\mathcal{C}).$$

Game G_3 : Guessing multiple elements.

³¹ The complete definition of **MacColl** is given in Section A.

We guess $s_{\text{Actor}}^{\text{Test}}, s_{\text{Peer}}^{\text{Test}}, s^{\text{Test}}$, we thus have: $\text{Adv}^{G_2}(\mathcal{A}) \leq N^2 l \text{Adv}^{G_3}(\mathcal{A})$.

Game G_4 : New win condition.

We create G_4 as G_3 , with a new condition for victory. An instance of G_4 is won by the adversary $\mathcal{A}$ if, and only if, s^{Test} completes without aborting and that nothing makes the instance of the game invalid (as per the conditions of G_3) until s^{Test} completes. In particular, this means that the first message received by s^{Test} contains a valid tag.

We stop the game at the moment s^{Test} completes as this is sufficient to check if it is won. Because we stop the game at the moment when s^{Test} completes, this implies that **test-session** is not even requested for s^{Test} . In particular, we know that in a valid session of G_4 and the following games, **corrupt** is not asked to $s_{\text{Peer}}^{\text{Test}}$. Thus, the valid instances verify the predicate $\text{AisInit} \wedge \text{CorP} \wedge \text{Orig}$.

Note that any winning instance of G_3 needs to satisfy the win conditions of G_4 . Thus, we have: $\text{Adv}^{G_3}(\mathcal{A}) \leq \text{Adv}^{G_4}(\mathcal{A})$.

Game G_5 : Replacement of $s^{\text{Test}}.\tilde{k}$ by a uniform key.

We create game G_5 where:

- $s^{\text{Test}}.\tilde{k}$ is replaced by a uniformly sampled key $\tilde{k}^*$,
- any responder session s with actor $s_{\text{Peer}}^{\text{Test}}$ that receives the ciphertext $s^{\text{Test}}.\tilde{ct}$ uses $\tilde{k}^*$ as $s.k$ instead of computing it with $\widetilde{\text{Decaps}}$.

To measure the difference of advantage between G_4 and G_5 , we construct an adversary $\mathcal{D}$ against the IND-CCA security of $\widetilde{\text{KEM}}$ as follows:

- $\mathcal{D}$ receives $(\text{pp}, \text{pk}_*, \text{ct}_*, k_u)$ where:
 - if $u = 0$, ct_* is an encapsulation of k_u .
 - if $u = 1$, k_u has been uniformly, independently sampled.
- $\mathcal{D}$ begins by simulating G_4 to the adversary $\mathcal{A}$, using $\widetilde{\text{Keygen}}(\text{pp})$ to create the long-term keys of the parties with role responder, except $s_{\text{Peer}}^{\text{Test}}$, that will uses pk_* . Note that in a valid instance, $s_{\text{Peer}}^{\text{Test}}$ is not corrupted, there is thus no need to know the secret key associated to pk_* .
- Instead of computing the cipher and key computed by the session s^{Test} with $\widetilde{\text{Encaps}}(\text{pp}, \text{pk}_*)$, $\mathcal{D}$ uses ct_* and k_u .
- If, at any moment, the (simulated) instance of G_4 becomes invalid, $\mathcal{D}$ stops the simulation and returns a uniform bit. We will assume in next items that this does not occur.
- For each session s such that $s_{\text{Actor}} = s_{\text{Peer}}^{\text{Test}}$:
 - If the first message received by s contains ciphertext ct_* , the decapsulation of $s.\tilde{ct} = \text{ct}_*$ computed by s is set to k_u .
 - Else, the decapsulation is computed using $\mathcal{O}_{\text{CCA}}$.

A direct analysis of $\mathcal{D}$ shows that:

- if $u = 0$, $\mathcal{D}$ almost perfectly simulates G_4 to $\mathcal{A}$. The only issue with the simulation is that the decapsulation of the ciphertext received by s^{Test} always succeeds, while in the real game, its result could be different of the encapsulated key. This difference can be measured by the correctness of $\widetilde{\text{KEM}}$, and we have: $|\Pr[\mathcal{D} \text{ outputs } 1 \mid u = 0] - \Pr[1 \leftarrow G_4(\mathcal{A})]| \leq l\delta'$.

- if $u = 1$, $\mathcal{D}$ perfectly simulates G_5 to $\mathcal{A}$. Thus, $\Pr[\mathcal{D} \text{ outputs } 1 \mid u = 1] = \Pr[1 \leftarrow G_5(\mathcal{A})]$.
Finally, we obtain:

$$\left| \text{Adv}^{G_4}(\mathcal{A}) - \text{Adv}^{G_5}(\mathcal{A}) \right| \leq 2\text{Adv}_{\text{KEM}}^{\text{IND-CCA}} + l\delta'.$$

Moreover, $\mathcal{D}$ queries the oracle $\mathcal{O}_{\text{CCA}}$ at most l times.

Analysis of G_5 . We upper bound $\text{Adv}^{G_5}(\mathcal{A})$ by the advantage of an adversary $\mathcal{E}$ winning the game sUnforg of the MAC.

$\mathcal{E}$ has access to an oracle $\text{Mac}(\mathbf{k}, \cdot)$ for a key it does not know. When queried on an entry m , this oracle provides a tag tag such that $\text{MacVf}(\mathbf{k}, \text{tag}, m) = 1$ and registers the couple (m, tag) in a database Q . In order to win, $\mathcal{E}$ needs to provide a couple of message and tag $(m, \text{tag}) \notin Q$ such that $\text{MacVf}(\mathbf{k}, \text{tag}, m) = 1$.

To do so, $\mathcal{E}$ simulates G_5 to $\mathcal{A}$ in such a way that $\mathbf{k}$ corresponds to $\mathbf{s}^{\text{Test}}.\tilde{\mathbf{k}}$ and to $\mathbf{s}.\tilde{\mathbf{k}}$ for each session $\mathbf{s}$ with actor $\mathbf{s}_{\text{Peer}}^{\text{Test}}$ that received $\mathbf{s}^{\text{Test}}.\tilde{\text{ct}}$. More precisely, for each of these sessions $\mathbf{s}$, $\mathcal{E}$ does not compute $\mathbf{s}.\tilde{\mathbf{k}}$, and uses the oracle $\text{Mac}(\mathbf{k}^{\text{MAC}}, \cdot)$ to compute the tag. They are the only situations where the oracle is used during the simulation.

When the simulation of G_5 is over, $\mathcal{E}$ returns $(\mathbf{m}, \text{tag})$ to the challenger of sUnforg , where tag is the tag received by $\mathbf{s}^{\text{Test}}$ and $\mathbf{m} = (\mathbf{s}_{\text{ActorId}}^{\text{Test}}, \mathbf{s}_{\text{PeerId}}^{\text{Test}}, \mathbf{s}^{\text{Test}}.\text{pk}_R, \mathbf{s}^{\text{Test}}.\tilde{\text{ct}}, \mathbf{s}^{\text{Test}}.\text{ct})$. If $\mathbf{s}^{\text{Test}}$ did not receive a tag, the output of $\mathcal{E}$ does not matter and it returns any couple.

We note that $\mathcal{E}$ provides a perfect simulation of G_5 to $\mathcal{A}$. Moreover, when the simulated instance of G_5 is won, we claim that $\mathcal{E}$ wins the game sUnforg . To see this, we first recall that in order for the simulated instance of G_5 to be won, the tag tag received by $\mathbf{s}^{\text{Test}}$ needs to satisfy $\text{MacVf}(\mathbf{k}, \text{tag}, M) = 1$ and $(M, \text{tag}) \notin Q$. $\text{MacVf}(\mathbf{k}, \text{tag}, M) = 1$ is true, so we focus on showing $(M, \text{tag}) \notin Q$.

By definition of $\mathcal{E}$, in a valid instance, the messages that Q contains are of the form $M^* = (\mathbf{s}_{\text{PeerId}}, \mathbf{s}_{\text{ActorId}}, \mathbf{s}.\text{pk}_R, \mathbf{s}.\tilde{\text{ct}}, \mathbf{s}.\text{ct})$ for sessions $\mathbf{s}$ with actor $\mathbf{s}_{\text{Peer}}^{\text{Test}}$ and that received $\mathbf{s}^{\text{Test}}.\tilde{\text{ct}}$.

Because these sessions $\mathbf{s}$ are not origin session of $\mathbf{s}^{\text{Test}}$, we can see that $(M^*, \mathbf{s}.\text{tag}!) \neq (M, \text{tag})$. As a result, $(M, \text{tag}) \notin Q$.

We thus have $\text{Adv}^{G_5}(\mathcal{A}) \leq \text{Adv}^{\text{sUnforg}}(\mathcal{E})$.

Proposition 3 ($(U \wedge \text{eCK}^w)$ -KeySec and $(U \wedge \text{eCK-PFS})$ -KeySec-Security of DAKE^σ). *We consider an adversary $\mathcal{A}$ against the Key Secrecy of DAKE_2^σ or DAKE_3^σ . We suppose that $\mathcal{A}$ activates less than N parties and that for each fixed party P , it activates less than l sessions that have either actor or peer P . Then, in the standard model, for $\text{Advantage} \in \{\text{Adv}_{\text{DAKE}_2^\sigma}^{U \wedge \text{eCK}^w\text{-KeySec}}(\mathcal{A}), \text{Adv}_{\text{DAKE}_3^\sigma}^{U \wedge \text{eCK}^w\text{-KeySec}}(\mathcal{A}), \text{Adv}_{\text{DAKE}_3^\sigma}^{U \wedge \text{eCK-PFS-KeySec}}(\mathcal{A})\}$ there exist adversaries $\mathcal{B}, \mathcal{C}, \mathcal{D}$ such that,*

$$\begin{aligned} \text{Advantage} \leq & \text{Adv}_{\text{Sign}}^{\text{MBS}}(\mathcal{B}) + \text{Adv}_{\text{Sign}}^{\text{M-S-UEO}}(\mathcal{C}) + N\text{Adv}_{\text{Sign}}^{\text{Key-Recov}}(\mathcal{D}) \\ & + N^2 l \text{Adv}_{\text{Sign}}^{\text{sUF-CMA}}(\mathcal{E}). \end{aligned}$$

Moreover, $\mathcal{E}$ queries $\mathcal{O}_{\text{Sign}}$ at most l times.

Proof. We study the $U \wedge \text{eCK}^w\text{-KeySec}$ security of DAKE_2^σ , the proof for the $U \wedge \text{eCK}^w\text{-KeySec}$ and $U \wedge \text{eCK}^w\text{-KeySec}$ security of DAKE_3^σ being similar.

Note that this demonstration can also be reused to establish **Resp-explicit** authentication for DAKE_2^σ and DAKE_3^σ , as explained in the following sidebar:

Reusing this demonstration to establish Resp-explicit authentication.

First, we note that the advantage in **Resp-explicit** authentication for the protocols DAKE_2^σ and DAKE_3^σ is exactly the probability that an instance of the Key Secrecy game satisfies the predicate $\text{AisInit} \wedge \overline{\text{corPbeforeT}} \wedge \overline{\text{FirstMatch}}$ and that the test session completes³². With this formulation, the test session corresponds to the session that the adversary must select in the **Resp-explicit** authentication game.

Using this formulation of the **Resp-explicit** authentication game, we observe that the proof of Proposition 3 also establishes **Resp-explicit** authentication for DAKE_2^σ . Indeed, this proof bounds the probability that an instance of the game satisfies the predicate $\text{AisInit} \wedge \overline{\text{corPbeforeT}} \wedge \overline{\text{FirstMatch}}$ and that the test session completes. Moreover, the proof for DAKE_3^σ uses exactly the same arguments.

Game G_1 : (Origin session implies first Match session) $\wedge$ (no signature collision). We recall that G_0 is the original security game for the AKE and that certain instances of this game can be designated as invalid. We call G_0 is the original security game for the AKE. We recall that certain instances of this game can be designated as invalid, which is indicated by the event $\text{E}_{\text{ValidInstance}}$ being false. An instance are invalid either if $U \wedge \text{eCK}^w\text{-KeySec}$ is false or if no session has been tested. For an invalid instance, a random bit is returned instead of the adversary's result.

In G_1 , additional instances are considered invalid. Specifically, any instance is deemed invalid if one of these two conditions is true :

- There exist two distinct sessions $\bar{s}, \tilde{s}$ with same responder party computing the same signature but for distinct messages.
- There exists a session s which is an origin session of s^{Test} but not a first match of s^{Test} .

The first item directly creates a solution to the MBS game³³ by using an adversary that simulates G_0 . The second item gives a signature $\sigma = s.\sigma$, two messages

$$\begin{aligned} m_1 &= (s_{\text{ActorId}}^{\text{Test}}, s_{\text{PeerId}}^{\text{Test}}, s^{\text{Test}}.\text{pk}_R, s^{\text{Test}}.\text{ct}), \\ m_2 &= (s_{\text{PeerId}}, s_{\text{ActorId}}, s.\text{pk}_R, s.\text{ct}), \end{aligned}$$

and two public keys: pk_1 the public key of $s_{\text{Actor}}^{\text{Test}}$ and pk_2 the public key of s_{Actor} , such that :

$$\begin{aligned} \text{SignVf}(\text{pp}^{\text{sign}}, \text{pk}_1, \sigma, m_1) &= 1 \wedge \text{SignVf}(\text{pp}^{\text{sign}}, \text{pk}_2, \sigma, m_2) = 1, \\ (\text{pk}_1 \neq \text{pk}_2) \vee (m_1 \neq m_2), \end{aligned}$$

³³ The complete definition of MBS and M-S-UEO are given in Section A.

The inequalities being a direct consequence of the fact s is not a first matching session of s^{Test} .

We see that if $\text{pk}_1 = \text{pk}_2$, this leads to a solution of MBS while, if $\text{pk}_1 \leq \text{pk}_2$, this leads to a solution of M-S-UEO.

Thus, we can create two adversaries $\mathcal{B}, \mathcal{C}$ such that:

$$\left| \text{Adv}^{G_2}(\mathcal{A}) - \text{Adv}^{G_1}(\mathcal{A}) \right| \leq \text{Adv}_{\text{Sign}}^{\text{MBS}}(\mathcal{B}) + \text{Adv}_{\text{Sign}}^{\text{M-S-UEO}}(\mathcal{C}).$$

Game G_2 : Removal of collisions of pk^{sign} . In G_2 , additional instances are considered invalid. Specifically, any instance is deemed invalid if there exist two distinct parties with role **Resp** that have the same public static key,

This probability can be bounded by $N \text{Adv}_{\text{Sign}}^{\text{Key-Recov}}(\mathcal{B})$ for some adversary $\mathcal{D}$ of the key recovery game of **Sign**. Thus, we have:

$$\left| \text{Adv}^{G_2}(\mathcal{A}) - \text{Adv}^{G_1}(\mathcal{A}) \right| \leq N \text{Adv}_{\text{Sign}}^{\text{Key-Recov}}(\mathcal{D}).$$

Game G_3 : Guessing multiple elements. Same game than the game 3 of the proof of Proposition 2. We guess $s_{\text{Actor}}^{\text{Test}}, s_{\text{Peer}}^{\text{Test}}, s^{\text{Test}}$. We have $\text{Adv}^{G_2}(\mathcal{A}) \leq N^2 / \text{Adv}^{G_3}(\mathcal{A})$.

Game G_4 : New win condition.

Same game than the game 4 of the proof of Proposition 2.

We create G_4 as G_3 , with a new condition for victory. An instance of G_4 is won by the adversary $\mathcal{A}$ if and only if s^{Test} complete without aborting and that nothing makes the instance of the game invalid (as per the conditions of G_3) until s^{Test} completes. In particular, this means that the first message received by s^{Test} contains valid signature.

We stop the game at the moment s^{Test} completes as this is sufficient to check if it is won. Because we stop the game at the moment when s^{Test} completes, this implies that **test-session** is not even requested for s^{Test} . In particular, we know that in a valid session of G_4 and the following games, **corrupt** is not asked to $s_{\text{Peer}}^{\text{Test}}$. Thus, the valid instances verify the predicate $\text{AisInit} \wedge \overline{\text{CorP}} \wedge \overline{\text{Orig}}$.

For G_3 to be won, any instance of G_3 needs to satisfy the win conditions of G_4 . Thus, we have:

$$\text{Adv}^{G_3}(\mathcal{A}) \leq \text{Adv}^{G_4}(\mathcal{A}).$$

Analysis of G_4 . We upper bound $\text{Adv}^{G_4}(\mathcal{A})$ by the advantage of an adversary $\mathcal{D}$ winning the strong forgery game **sUF-CMA** of the signature and using at most l times the oracle $\mathcal{O}_{\text{Sign}}$.

$\mathcal{E}$ receives public parameters and a challenge public signature key $(\text{pp}^{\text{sign}}, \text{pk}_*)$, moreover, it has access to an oracle $\mathcal{O}_{\text{Sign}}$ that can give signatures for the secret key associated to pk_* . When $\mathcal{O}_{\text{Sign}}$ construct a signature σ' for a message m' , it registers the couple (m', σ') in a database Q .

$\mathcal{E}$ simulates the game G_4 using pk_* as the public key of $s_{\text{Peer}}^{\text{Test}}$ —note that this party is not corrupted in valid instances. When $\mathcal{D}$ needs to simulate the construction of the signature of a responder session with actor $s_{\text{Peer}}^{\text{Test}}$, it uses the oracle $\mathcal{O}_{\text{Sign}}$.

We denote by σ the signature received by s^{Test} . This signature is verified with public key $s^{\text{Test}}.\text{pk}^{\text{sign}}$ for the message $m = (s_{\text{ActorId}}^{\text{Test}}, s_{\text{PeerId}}^{\text{Test}}, s^{\text{Test}}.\text{pk}_R, s^{\text{Test}}.\text{ct})$. $\mathcal{D}$ returns (m, σ) .

When the simulated instance wins G_5 , (m, σ) wins the game sUF-CMA . Indeed, we already know that in this case, $\text{SignVf}(\text{pp}^{\text{sign}}, \text{pk}^{\text{sign}}, m, \sigma) = 1$. Moreover, because there is no origin session of s^{Test} , we can see that $(m, \sigma) \notin Q$.

We thus have $\text{Adv}^{G_4}(\mathcal{A}) \leq \text{Adv}^{\text{sUF-CMA}}(\mathcal{E})$.

E.4 Key Secrecy for Predicates $V_1, \dots, V_5$

As explained in E.1, the proof we are presenting now is the same for DAKE, its variant DAKE^σ , and even the protocol described in Figure 17, which has no responder authentication.³⁴ This is because we never rely on the authentication of the responder.

We provide the proof only for the protocol of Figure 17—the less secure one—which we denote by AKE, and by AKE_2 or AKE_3 when it is necessary to distinguish the two- or three-message variants.

We consider $\mathcal{A}$ as a PPT adversary participating in the KeySec game for AKE. We assume that $\mathcal{A}$ activates fewer than N parties, and that for each fixed party P , it activates fewer than l sessions, with P acting either as the actor or as the peer. Moreover, we recall that 2K-KEM is δ -correct and γ -spread for some $\delta \geq 0$ and $\gamma > 0$.

The security analysis made in this section proceeds as follows:

- For $V_1, \dots, V_4$ we analyze AKE_2 ; the proof extends verbatim to AKE_3 .
- For V_5 we analyze AKE_3 , since in this case the security of AKE_2 does not hold.

V_1 -KeySec Security in the Standard Model for the Two- and Three-Message Variants. We recall that, as defined in (16) (page 67):

$$V_1 = \text{AisInit} \wedge \text{FirstMatch} \wedge \overline{\text{CorA}} \wedge (\overline{\text{CorP}} \vee \overline{\text{LeakRandO}}) \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}.$$

We study the security of AKE_3 , whose only difference from AKE_2 is that the fourth game (which relies on the security of the PRG) is required only for AKE_3 .

Assume that dKEM is δ -correct and γ -spread. We will show that, in the standard model, there exist adversaries $\mathcal{B}, \mathcal{C}$ and $\mathcal{D}$ such that:

$$\begin{aligned} \text{Adv}_{\text{AKE}_3}^{V_1\text{-KeySec}}(\mathcal{A}) &\leq Nl \text{Adv}_{\text{dKEM}}^{[\cdot, \text{Key-Recov}]}(\mathcal{B}) + Nl^2 (\delta + 2^{-\gamma+1}) + \\ &\quad + 2Nl^2 (\text{Adv}_{\text{dKEM}}^{[\text{IND-CCA}, \cdot]}(\mathcal{C}) + \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{D})). \end{aligned}$$

Moreover, $\mathcal{C}$ queries the oracles $\mathcal{O}_{\text{leak}_R}$ and $\mathcal{O}_{\text{CCA}_L}$ at most $(l-1)$ times.

Note that the desired bound for AKE_2 is obtained simply by removing the advantage of IND-PRG.

³⁴ Note that this protocol is not DUAKE: DUAKE adds a hash value in the first message sent by the responder.

Proof. **Game G_1 : Removal of collisions with pk_R and ct .**

We recall that G_0 is the original security game for the AKE and that certain instances of this game can be designated as invalid, which is indicated by the event $E_{ValidInstance}$ being false. For an invalid instance, a random bit is returned instead of the adversary's result.

In G_1 , additional instances are considered invalid. Specifically, any instance is deemed invalid if the following conditions hold:

- For any party P with role $Init$, there is a collision in the right key pk_R computed by the sessions with P as the actor.
- For any party P with role $Init$, there is a collision in the ciphertexts computed by the sessions with P as the peer.
- For any party P with role $Init$, a ciphertext ct exists such that:
 - A session with P as the actor receives a first message with ciphertext ct .
 - Later, a session with peer P , sends its first message, this message having ciphertext ct .

The probability of the first item can be bounded by $N!Adv_{dKEM}^{[\cdot, Key-Recov]}(\mathcal{B})$ for some adversary $\mathcal{B}$. For the second and third items, we leverage the spreadness of $dKEM$ to bound the probability of these events by $2Nl^22^{-\gamma}$.

Thus, we have:

$$\left| Adv^{G_0}(\mathcal{A}) - Adv^{G_1}(\mathcal{A}) \right| \leq N!Adv_{dKEM}^{[\cdot, Key-Recov]}(\mathcal{B}) + 2Nl^22^{-\gamma}.$$

Note that in a valid instance of G_1 , there exists a first matching session s^{FirstM} of s^{Test} . It is in particular an origin session of s^{Test} because s^{Test} only receives one message. Moreover, it is the only origin session of s^{Test} .

Game G_2 : Guessing s_{Actor}^{Test} , s^{Test} and s^{FirstM} .

Note that, for a fixed party P , in any instance of G_1 , we can enumerate each session s where P is either the actor or the peer, based on its order of creation. We denote its order by $l_{ActorOrPeer=P}(s)$. By definition, we have $l_{ActorOrPeer=P}(s) \in \text{Range}(l)$. G_2 is similar to G_1 with two modifications:

- We begin by uniformly sampling three numbers from respectively $\text{Range}(N)$, $\text{Range}(l)$ and $\text{Range}(l)$.
- If these numbers do not match the numbers $s_{ActorInternId}^{Test}$, $l_{ActorOrPeer=s_{Actor}^{Test}}(s^{Test})$ and $l_{ActorOrPeer=s_{Actor}^{Test}}(s^{FirstM})$ respectively, the session is invalid. In such cases, instead of producing $\mathcal{A}$'s result, we output a uniformly random bit.

Thus, we have $Adv^{G_2}(\mathcal{A}) \leq Nl^2Adv^{G_1}(\mathcal{A})$.

Note that, because they are guessed at the beginning of the game, the long-term key pk_R for s_{Actor}^{Test} , the ephemeral key pk_R for s^{Test} and the key and encapsulation computed by s^{Test} can be precomputed.

Game G_3 : Replacement of $s^{FirstM}.k$ and $s^{Test}.k$ by a uniform key.

We create game G_3 where the keys $s^{Test}.k$ and $s^{FirstM}.k$ are replaced by a uniformly sampled key k .

To measure the difference of advantage between G_2 and G_3 , we construct an adversary $\mathcal{C}$ against the $[IND-CCA, -]$ security of $dKEM$ as follows:

- $\mathcal{C}$ receives $(pp, pk_{R,*}, sk_{R,*}, r_{R,*}, pk_{L,*}, ct_*, k_u)$ where:

- if $u = 0$, ct_* is an encapsulation of k_u with public keys $\text{pk}_{L,*}$ and $\text{pk}_{L,*}$.
- if $u = 1$, k_u has been uniformly, independently sampled.

$\mathcal{C}$ starts by simulating G_2 to the adversary $\mathcal{A}$, using pp as public parameters of the protocol, $\text{pk}_{L,*}$ as the static public key of $s_{\text{Actor}}^{\text{Test}}$, $\text{pk}_{R,*}$ as $s^{\text{Test}}.\text{pk}_R$, $\text{sk}_{R,*}$ as $s^{\text{Test}}.\text{sk}_R$, ct_* as $s^{\text{Test}}.\text{ct}$, the ciphertext created by s^{FirstM} and k_u as the key encapsulated by s^{FirstM} under ct_* . $\mathcal{C}$ also uses k_u as the key decapsulated by s^{Test} , instead of computing its decapsulation.

- If, at any moment, the (simulated) instance of G_2 becomes invalid, $\mathcal{C}$ stops the simulation and returns a uniform bit. We will assume in next items that this does not occur.
- For each session $s \neq s^{\text{Test}}$ such that $s_{\text{Actor}} = s_{\text{Actor}}^{\text{Test}}$, each couple of keys $(\text{pk}_R, \text{sk}_R)$, normally created by $\text{Keygen}_R(\text{pp})$ in the first message of s , is instead computed by $\mathcal{C}$ with $\mathcal{O}_{\text{leak}_R}$. This allows $\mathcal{C}$ to use $\mathcal{O}_{\text{CCA}_L}$ on them.³⁵
- For each session $s \neq s^{\text{Test}}$ such that $s_{\text{Actor}} = s_{\text{Actor}}^{\text{Test}}$, $\mathcal{C}$ uses $\mathcal{O}_{\text{CCA}_L}$ to compute the decapsulation.

A direct analysis of $\mathcal{C}$ shows that:

- if $u = 0$, $\mathcal{C}$ almost perfectly simulates G_2 to $\mathcal{A}$. The only issue with the simulation is that the decapsulation of the ciphertext received by s^{Test} always succeeds, while in the real game, its result could be different of the encapsulated key. This difference can be measured by the correctness of dKEM, and we have: $|\Pr[\mathcal{C} \text{ outputs } 1 \mid u = 0] - \Pr[1 \leftarrow G_2(\mathcal{A})]| \leq \delta$.
 - if $u = 1$, $\mathcal{C}$ perfectly simulates G_3 to $\mathcal{A}$. Thus, $\Pr[\mathcal{C} \text{ outputs } 1 \mid u = 1] = \Pr[1 \leftarrow G_3(\mathcal{A})]$.
- Finally, we have³⁶:

$$\left| \text{Adv}^{G_3}(\mathcal{A}) - \text{Adv}^{G_2}(\mathcal{A}) \right| \leq 2\text{Adv}_{\text{dKEM}}^{\text{IND-CCA}, -}(\mathcal{C}) + \delta.$$

We also note that, by construction, $\mathcal{C}$ uses at most $l - 1$ times the oracles $\mathcal{O}_{\text{leak}_R}$ and $\mathcal{O}_{\text{CCA}_L}$.

Game G_4 : Replacement of $s^{\text{Test}}.k^{\text{SESSION}}$ and $s^{\text{FirstM}}.k^{\text{SESSION}}$ by a uniform key (together with their MAC keys).

We create game G_4 where:

- the keys $s^{\text{Test}}.k^{\text{SESSION}}$ and $s^{\text{FirstM}}.k^{\text{SESSION}}$ are replaced by a uniformly sampled key.
- the keys $s^{\text{Test}}.k^{\text{MAC}}$ and $s^{\text{FirstM}}.k^{\text{MAC}}$ are replaced by a uniformly sampled key.

To measure the difference of advantages between G_3 and G_4 , we construct an adversary $\mathcal{D}$ against IND-PRG:

- We denote by u the bit sampled by the challenger. The adversary is given out_u and tries to guess u where :

³⁵ Note that $\mathcal{O}_{\text{leak}_R}$ also provides the randomness used to compute them, which can then be returned when eph-key is queried.

³⁶ Note that in the theorems summarizing the security of the protocol, we use $[\text{IND-CK-CCA}, -]$ security in order to simplify the formulae, this notion is stronger than $[\text{IND-CCA}, -]$ security : the adversary simply computes $(\text{pk}_{R,*}, \text{sk}_{R,*}, r_{R,*})$ with $\mathcal{O}_{\text{leak}_R}$ instead of receiving them from the challenger.

- $\text{out}_1 = \text{PRG}(k)$ for a key k uniformly taken in the set of session keys.
 - out_0 is uniformly taken in the image of PRG .
 - $\mathcal{D}$ simulates G_4 , but uses out_u instead of $(s^{\text{Test},k^{\text{SESSION}}}, s^{\text{Test},k^{\text{MAC}}})$ and of $(s^{\text{FirstM},k^{\text{SESSION}}}, s^{\text{FirstM},k^{\text{MAC}}})$.
 - If, at any moment, the—simulated—instance of G_4 becomes invalid, $\mathcal{D}$ returns a uniform bit.
 - At the end of the simulation, $\mathcal{D}$ returns 1 if, and only if, $\mathcal{A}$ wins.
- We can see that:
- When $u = 0$, $\mathcal{D}$ perfectly simulates G_3 to $\mathcal{A}$. Hence, $\Pr[\mathcal{D} \text{ outputs } 1 \mid u = 0] = \Pr[1 \leftarrow G_3(\mathcal{A})]$.
 - When $u = 1$, $\mathcal{D}$ perfectly simulates G_4 to $\mathcal{A}$. Hence, $\Pr[\mathcal{D} \text{ outputs } 1 \mid u = 1] = \Pr[1 \leftarrow G_4(\mathcal{A})]$.
- Finally, we have:

$$\left| \text{Adv}^{G_4}(\mathcal{A}) - \text{Adv}^{G_3}(\mathcal{A}) \right| \leq 2\text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{D}).$$

Study of G_4 and conclusion. In G_4 , whether b is equal to 0 or 1, the session key of the tested session is uniformly sampled. Consequently, $\text{Adv}^{G_4}(\mathcal{A}) = 0$, allowing us to conclude.

V₂-KeySec Security in the Standard Model for the Two- and Three-Message Variants. We recall that, as defined in (16) (page 67):

$$V_2 = \text{AisResp} \wedge \overline{\text{CorP}} \wedge (\overline{\text{CorA}} \vee \overline{\text{LeakRandT}}) \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}.$$

We study the security of AKE_3 , whose only difference from AKE_2 is that the fourth game (which relies on the security of the PRG) is required only for AKE_3 .

Assume that dKEM is δ -correct and γ -spread. We will show that, in the standard model, there exist adversaries $\mathcal{B}$ and $\mathcal{C}$ such that:

$$\begin{aligned} \text{Adv}_{\text{AKE}_2}^{V_2\text{-KeySec}}(\mathcal{A}) &\leq Nl \text{Adv}_{\text{dKEM}}^{[\cdot, \text{Key-Recov}]}(\mathcal{B}) + 2Nl^2 (\delta + 2^{-\gamma}) + \\ &\quad + 4Nl^2 (\text{Adv}_{\text{dKEM}}^{[\text{IND-CK-CCA}, -]}(\mathcal{C}) + \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{D})). \end{aligned}$$

Moreover, $\mathcal{D}$ queries the oracles $\mathcal{O}_{\text{leak}_R}$ and $\mathcal{O}_{\text{CCA}_L}$ at most l times.

Note that the desired bound for AKE_2 is obtained simply by removing the advantage of IND-PRG .

Proof. **Game G_1 : Removal of some collisions with pk_R and ct .**

It is the same game as G_1 for V_1 (page 76), the difference of advantage being the same.

Thanks to G_1 , we know that if a first-partner session exists, it is unique. If so, we will denote it by s^{FirstP} .

Game G_2 : Guessing multiple elements.

Using the same method as in G_2 for V_1 (see page 76), we guess $s_{\text{Peer}}^{\text{Test}}$, s^{Test} , whether s^{FirstP} exists (so, we guess a if yes or not the s^{FirstP} exists) and, if so, we also guess s^{FirstP} .

We have $\text{Adv}^{G_1}(\mathcal{A}) \leq 2Nl^2 \text{Adv}^{G_2}(\mathcal{A})$.

Note that, because they are guessed at the beginning of the game:

- if s^{FirstP} exists, we can precompute the long-term key pk_L for $s_{\text{Peer}}^{\text{Test}}$, the ephemeral key pk_R of s^{FirstP} as well as the cipher and encapsulated key computed by s^{Test} .
- if s^{FirstP} does not exist, we can only precompute the long-term key pk_L for $s_{\text{Peer}}^{\text{Test}}$.

Game G_3 : Replacement of $s.k$ and $s^{\text{FirstP}}.k$ (if it exists) by a uniform key.

We create game G_3 where the keys $s^{\text{Test}}.k$ and $s^{\text{FirstP}}.k$ (if s^{FirstP} exists) are replaced by a uniformly sampled key $\tilde{k}$.

Since we make an initial guess in the game about the existence of s^{FirstP} , we can separately prove the scenarios where it exists and where it does not.

If s^{FirstP} exists:

We measure the difference of advantage between G_2 and G_3 by the advantage of an adversary $\mathcal{C}$ against the $[\text{IND-CCA}, -]$ security of dKEM, as made in the game G_3 of V_1 , and we obtain the bound

$$\left| \text{Adv}^{G_3}(\mathcal{A}) - \text{Adv}^{G_2}(\mathcal{A}) \right| \leq 2\text{Adv}_{\text{dKEM}}^{[\text{IND-CCA}, -]}(\mathcal{C}) + \delta. \quad (18)$$

Moreover, $\mathcal{C}$ queries the oracles $\mathcal{O}_{\text{leak}_R}$ and $\mathcal{O}_{\text{CCA}_L}$ at most $(l-1)$ times.

If s^{FirstP} does not exist:

In this case, we create an adversary $\mathcal{C} = (\mathcal{C}_1, \mathcal{C}_2)$ against the $[\text{IND-CK-CCA}, -]$ security of dKEM, as follows:

- $\mathcal{C}_1$ receives $(pp, pk_{L,*})$ and begins by simulating G_2 to the adversary $\mathcal{A}$, using pp as public parameters of the protocol and $pk_{L,*}$ as the static public key of $s_{\text{Peer}}^{\text{Test}}$.
- If, at any moment, the (simulated) instance of G_2 becomes invalid, $\mathcal{C}$ stops the simulation and returns a uniform bit. We will assume in next items that this does not occur.
- For each session s such that $s_{\text{Actor}} = s_{\text{Actor}}^{\text{Test}}$ (including s^{Test}), each couple of keys (pk_R, sk_R) , normally created by $\text{Keygen}_R(pp)$ in the first message of s , is instead computed by $\mathcal{C}_1$ with $\mathcal{O}_{\text{leak}_R}$. This allows $\mathcal{C}$ to use $\mathcal{O}_{\text{CCA}_L}$ on them.
- When s^{Test} receives its first message $(id, pk_{R,*})$, $\mathcal{C}_1$ sends $pk_{R,*}$ to the challenger³⁷. $\mathcal{C}_2$ then receives a ciphertext ct_* and a key k_u . We recall that u is a bit uniformly sampled by the challenger and that:
 - if $u = 0$, ct_* is an encapsulation of k_u with public keys $pk_{L,*}, pk_{R,*}$.
 - if $u = 1$, k_u has been uniformly and independently sampled. $\mathcal{C}_2$ uses ct_* and k_u respectively as $s^{\text{Test}}.ct$ and $s^{\text{Test}}.k$.
- When an initiator session s with actor $s_{\text{Actor}} = s_{\text{Peer}}^{\text{Test}}$ receives its first message, decapsulation is computed by $\mathcal{O}_{\text{CCA}_L}$. Note that because no session is a first matching session of s^{Test} , $\mathcal{O}_{\text{CCA}_L}$ will always compute a decapsulation (*i.e.*, it will not directly return $\perp$).
- If $\mathcal{C}_2$ did not return a uniform bit before $\mathcal{A}$ ends, it returns 1 if $\mathcal{A}$ wins the simulated game and else 0.

A direct analysis of $\mathcal{C}$ shows that:

³⁷ For concision, we omit **state**.

- if $u = 0$, $\mathcal{C}$ perfectly simulates G_2 to $\mathcal{A}$. Thus, $\Pr[\mathcal{C} \text{ outputs } 1 \mid u = 1] = \Pr[1 \leftarrow G_2(\mathcal{A})]$.
- if $u = 1$, $\mathcal{C}$ perfectly simulates G_3 to $\mathcal{A}$. Thus, $\Pr[\mathcal{C} \text{ outputs } 1 \mid u = 1] = \Pr[1 \leftarrow G_3(\mathcal{A})]$.

$$\left| \text{Adv}^{G_3}(\mathcal{A}) - \text{Adv}^{G_2}(\mathcal{A}) \right| \leq 2\text{Adv}_{\text{dKEM}}^{[\text{IND-CK-CCA}, -]}(\mathcal{C}). \quad (19)$$

Moreover, $\mathcal{C}$ queries the oracles $\mathcal{O}_{\text{leak}_R}$ and $\mathcal{O}_{\text{CCA}_L}$ at most l times.

Conclusion of the two scenarios:

Using (18) and (19), and using the fact that chosen-key is stronger than non-chosen-key security, we can create an adversary $\mathcal{C}$ such that:

$$\left| \text{Adv}^{G_3}(\mathcal{A}) - \text{Adv}^{G_2}(\mathcal{A}) \right| \leq 2\text{Adv}_{\text{dKEM}}^{[\text{IND-CK-CCA}, -]}(\mathcal{C}) + \delta. \quad (20)$$

Moreover, $\mathcal{C}$ queries the oracles $\mathcal{O}_{\text{leak}_R}$ and $\mathcal{O}_{\text{CCA}_L}$ at most l times.

Game G_4 : Replacement of $s^{\text{Test}, k^{\text{SESSION}}}$ and $s^{\text{FirstP}, k^{\text{SESSION}}}$ (if it exists) by a uniform key (together with their MAC keys).

We create game G_4 where:

- the key $s^{\text{Test}, k^{\text{SESSION}}}$ is replaced by a uniformly sampled key. If s^{FirstP} exists, $s^{\text{FirstP}, k^{\text{SESSION}}}$ is also replaced by the same key.
- the key $s^{\text{Test}, k^{\text{MAC}}}$ is replaced by a uniformly sampled key. If s^{FirstP} exists, $s^{\text{FirstP}, k^{\text{MAC}}}$ is also replaced by the same key.

To measure the difference of advantages between G_3 and G_4 , we construct an adversary $\mathcal{D}$ against IND-PRG, as done in the G_4 for the study of V_1 -KeySec (page 77).

This gives:

$$\left| \text{Adv}^{G_4}(\mathcal{A}) - \text{Adv}^{G_3}(\mathcal{A}) \right| \leq 2\text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{D}).$$

Study of G_4 and conclusion. In G_3 , whether b is equal to 0 or 1, the session key of the tested session is uniformly sampled. Consequently, $\text{Adv}^{G_3}(\mathcal{A}) = 0$, allowing us to conclude.

V_3 -KeySec Security in Both the Standard Model and the ROM for the Two- and Three-Message Variants We recall that, as defined in (16) (page 67), the predicate V_3 is equal to

$$\text{AisInit} \wedge \text{FirstMatch} \wedge \text{CorA} \wedge \overline{\text{LeakRandT}} \wedge (\overline{\text{CorP}} \vee \overline{\text{LeakRandO}}) \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}.$$

We study the security of AKE_3 , whose only difference from AKE_2 is that the fourth game (which relies on the security of the PRG) is required only for AKE_3 .

Assume that dKEM is δ -correct and γ -spread. We will show that in the standard model, there exist adversaries $\mathcal{B}$, and $\mathcal{C}$ such that:

$$\begin{aligned} \text{Adv}_{\text{AKE}_2}^{V_3\text{-KeySec}}(\mathcal{A}) &\leq Nl \text{Adv}_{\text{dKEM}}^{[\cdot, \text{Key-Recov}]}(\mathcal{B}) + Nl^2 (\delta + 2^{-\gamma+1}) + \\ &\quad + 2Nl^2 \left(\text{Adv}_{\text{dKEM}}^{[-, \text{IND-CPA}]}(\mathcal{C}) + \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{D}) \right). \end{aligned}$$

Note that the desired bound for AKE_2 is obtained simply by removing the advantage of IND-PRG.

Proof. **Game G_1 : Removal of some collisions with pk_R and ct.**

It is the same game as G_1 for V_1 (page 76), the difference of advantage being the same. Consequently, in any valid instance of G_1 , there exists a unique origin session of s^{Test} , which is also a first matching session of s^{Test} . We denote this session as s^{FirstM} .

Game G_2 : Guessing $s_{\text{Actor}}^{\text{Test}}$, s^{Test} and s^{FirstM} .

Using the same method as in G_2 for V_1 (see page 76), we guess $s_{\text{Actor}}^{\text{Test}}$, s^{Test} and s^{FirstM} . We have: $\text{Adv}^{G_1}(\mathcal{A}) \leq Nl^2 \text{Adv}^{G_2}(\mathcal{A})$. Guessing those sessions at the beginning of the game allows to precompute the long-term key pk_L for $s_{\text{Actor}}^{\text{Test}}$, the ephemeral key pk_R for s^{Test} , the session key as well as the encapsulation computed by s^{FirstM} .

Game G_3 : Replacement of $s^{\text{Test}}.k$ and $s^{\text{FirstM}}.k$ by a uniform key.

We create game G_3 where the keys $s^{\text{Test}}.k$ and $s^{\text{FirstM}}.k$ are replaced by a uniformly sampled key $\tilde{k}$. To measure the difference of advantage between G_2 and G_3 , we construct an adversary $\mathcal{C}$ against the $[-, \text{IND-CPA}]$ security of dKEM as follows:

- $\mathcal{C}$ receives $(\text{pp}, \text{pk}_{L,*}, \text{sk}_{L,*}, \text{r}_{L,*}, \text{pk}_{R,*}, \text{ct}_*, k_u)$, where u is a bit uniformly sampled by the challenger such that:
 - If $u = 0$, ct_* is an encapsulation of k_u with public keys $\text{pk}_{L,*}$ and $\text{pk}_{R,*}$.
 - If $u = 1$, k_u is uniformly and independently sampled.
- $\mathcal{C}$ starts by simulating G_2 to the adversary $\mathcal{A}$, using pp as public parameters of the protocol, $(\text{pk}_{L,*}, \text{sk}_{L,*})$ as the static public and private keys of $s_{\text{Actor}}^{\text{Test}}$, $\text{pk}_{R,*}$ as $s^{\text{Test}}.\text{pk}_R$, ct_* as $s^{\text{FirstM}}.\text{ct}$, the ciphertext created by s^{FirstM} and k_u as the key encapsulated by s^{FirstM} under ct_* . $\mathcal{C}$ also uses k_u as the key decapsulated by s^{Test} , instead of computing its decapsulation—note that s^{Test} receives the cipher ct_* because s^{FirstM} is its first match session. Note also that $\mathcal{C}$ can give sk_L to the adversary on a $\text{corrupt}(s_{\text{Actor}}^{\text{Test}})$ query.
- If anything makes the simulated session of G_2 invalid, $\mathcal{C}$ aborts and returns a uniform bit.
- If $\mathcal{C}$ did not return a uniform bit before $\mathcal{A}$ ends, it returns 1 if $\mathcal{A}$ wins the simulated game and else 0.

A direct analysis of $\mathcal{C}$ shows that:

- If $u = 0$, $\mathcal{C}$ almost perfectly simulates G_2 to $\mathcal{A}$. The only difference being that the decapsulation of the ciphertext received by s^{Test} always succeeds, while in the real game, its result might differ from the encapsulated key. The probability that this happens can be upper bounded by the correctness of dKEM, hence $|\Pr[\mathcal{C} \text{ outputs } 1 \mid u = 0] - \Pr[1 \leftarrow G_2(\mathcal{A})]| \leq \delta$.
- If $u = 1$, $\mathcal{C}$ perfectly simulates G_3 to $\mathcal{A}$, hence $\Pr[\mathcal{C} \text{ outputs } 1 \mid u = 1] = \Pr[1 \leftarrow G_3(\mathcal{A})](\mathcal{A})$.

Finally, we deduce that:

$$\left| \text{Adv}^{G_3}(\mathcal{A}) - \text{Adv}^{G_2}(\mathcal{A}) \right| \leq 2\text{Adv}_{\text{dKEM}}^{[-, \text{IND-CPA}]}(\mathcal{C}) + \delta.$$

Game G_4 : Replacement of $s^{\text{Test}}.k^{\text{SESSION}}$ and $s^{\text{FirstM}}.k^{\text{SESSION}}$ by a uniform key (together with their MAC keys).

We create game G_4 where:

- the keys $s^{\text{Test}}.k^{\text{SESSION}}$ and $s^{\text{FirstM}}.k^{\text{SESSION}}$ are replaced by a uniformly sampled key.
- the keys $s^{\text{Test}}.k^{\text{MAC}}$ and $s^{\text{FirstM}}.k^{\text{MAC}}$ are replaced by a uniformly sampled key.

To measure the difference of advantages between G_3 and G_4 , we construct an adversary $\mathcal{D}$ against IND-PRG, as done in the G_4 for the study of V_1 -KeySec (page 77).

This gives:

$$\left| \text{Adv}^{G_4}(\mathcal{A}) - \text{Adv}^{G_3}(\mathcal{A}) \right| \leq 2\text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{D}).$$

Study of G_4 and conclusion. In G_3 , whether b is equal to 0 or 1, the session key of the tested session is uniformly sampled. Consequently, $\text{Adv}^{G_3}(\mathcal{A}) = 0$, allowing us to conclude.

V_4 -KeySec Security in Both the Standard Model and the ROM for the Two- and Three-Message Variants We recall that, as defined in (16) (page 67):

$$V_4 = \text{AisResp} \wedge \text{CorP} \wedge \text{Orig} \wedge \overline{\text{LeakRandO}} \wedge (\overline{\text{CorA}} \vee \overline{\text{LeakRandT}}) \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}},$$

We study the security of AKE_3 , whose only difference from AKE_2 is that the fourth game (which relies on the security of the PRG) is required only for AKE_3 .

Assume that dKEM is δ -correct and γ -spread. We will show that, in the standard model, there exist adversaries $\mathcal{B}$, $\mathcal{C}$ and $\mathcal{D}$ such that:

$$\begin{aligned} \text{Adv}_{\text{AKE}_2}^{V_4\text{-KeySec}}(\mathcal{A}) &\leq Nl \text{Adv}_{\text{dKEM}}^{[\cdot, \text{Key-Recov}]}(\mathcal{B}) + Nl^2 (\delta + 2^{-\gamma+1}) + \\ &\quad + 2Nl^2 \left(\text{Adv}_{\text{dKEM}}^{[\cdot, \text{IND-CCA}]}(\mathcal{C}) + \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{D}) \right). \end{aligned}$$

Moreover, $\mathcal{C}$ does not query $\mathcal{O}_{\text{leak}_L}$ and queries $\mathcal{O}_{\text{CCA}_R}$ at most one time.

Note that the desired bound for AKE_2 is obtained simply by removing the advantage of IND-PRG.

Proof. **Game G_1 : Removal of some collisions with pk_R and ct .**

It is the same game as G_1 for V_1 (page 76), the difference of advantage being the same. Consequently, in any valid instance of G_1 , there exists a unique origin session for s^{Test} . We denote this session as s^{Origin} . Note that, however, it is possible that this session is not a first-partner session.

Game G_2 : Guessing $s_{\text{Actor}}^{\text{Test}}$, s^{Test} and s^{Origin} .

Using the same method as in G_2 for V_1 (see page 76), we guess $s_{\text{Actor}}^{\text{Test}}$, s^{Test} and s^{Origin} . We have $\text{Adv}^{G_1}(\mathcal{A}) \leq Nl^2 \text{Adv}^{G_2}(\mathcal{A})$.

Guessing those sessions at the beginning of the game allows to precompute the long-term key pk_L for $s_{\text{Actor}}^{\text{Test}}$, the ephemeral key pk_R for s^{Test} , the session key as well as the encapsulation computed by s^{Origin} .

Game G_3 : Replacement of $s^{\text{Test}}.k$ and $s^{\text{Origin}}.k$ (if it is first-partner) by a uniform key.

We create game G_3 where:

- $s^{\text{Test}}.k$ is replaced by a uniformly sampled key $\tilde{k}$.
- if the cipher $s^{\text{Origin}}.\text{ct}$ received by s^{Origin} is equal to the cipher $s^{\text{Test}}.\text{ct}$ computed by s^{Test} , the key $s^{\text{Origin}}.k$ is also replaced by $\tilde{k}$. Note that this condition is equivalent to s^{Origin} being a first-partner session of s^{Test} .

To measure the difference of advantage between G_2 and G_3 , we construct an adversary $\mathcal{C}$ against the $[-, \text{IND-CCA}]$ security of dKEM as follows:

- $\mathcal{C}$ receives $(\text{pp}, \text{pk}_{L,*}, \text{sk}_{L,*}, \text{r}_{L,*}, \text{pk}_{R,*}, \text{ct}_*, k_u)$ where:
 - if $u = 0$, ct_* is an encapsulation of k_u with public keys $\text{pk}_{L,*}$ and $\text{pk}_{L,*}$.
 - if $u = 1$, k_u has been uniformly, independently sampled.
- $\mathcal{C}$ starts by simulating G_2 to the adversary $\mathcal{A}$, using pp as public parameters of the protocol, $\text{pk}_{R,*}$ as $s^{\text{Origin}}.\text{pk}_R$ and $(\text{pk}_{L,*}, \text{sk}_{L,*})$ as the long-term keys of $s_{\text{Actor}}^{\text{Origin}}$ —note in particular that, because $\mathcal{C}$ knows $\text{sk}_{L,*}$, it can give it to $\mathcal{A}$ if $\mathcal{A}$ uses `corrupt` on it.
- If, at any moment, the (simulated) instance of G_2 becomes invalid, $\mathcal{C}$ stops the simulation and returns a uniform bit. We will assume in next items that this does not occur.
- When s^{Test} computes its first message, $\mathcal{A}$ uses ct_* as $s^{\text{Test}}.\text{ct}$ and k_u as $s^{\text{Test}}.k$.
- When s^{Origin} receives its first message, if the cipher is equal to ct_* , $\mathcal{C}$ uses k_u as the key decapsulated by s^{Origin} , instead of computing its decapsulation. Else, $\mathcal{C}$ uses the oracle $\mathcal{O}_{\text{CCA}_R}$ to compute the decapsulation.

A direct analysis of $\mathcal{C}$ shows that:

- if $u = 0$, $\mathcal{C}$ almost perfectly simulates G_2 to $\mathcal{A}$. The only issue with the simulation is that the decapsulation of the ciphertext received by s^{Origin} always succeeds, while in the real game, its result could be different of the encapsulated key. This difference can be measured by the correctness of dKEM, and we have: $|\Pr[\mathcal{C} \text{ outputs } 1 \mid u = 0] - \Pr[1 \leftarrow G_2(\mathcal{A})]| \leq \delta$.
- if $u = 1$, $\mathcal{C}$ perfectly simulates G_3 to $\mathcal{A}$. Thus, $\Pr[\mathcal{C} \text{ outputs } 1 \mid u = 1] = \Pr[1 \leftarrow G_3(\mathcal{A})]$.

Finally, we deduce that:

$$\left| \text{Adv}^{G_3}(\mathcal{A}) - \text{Adv}^{G_2}(\mathcal{A}) \right| \leq 2\text{Adv}_{\text{dKEM}}^{[-, \text{IND-CCA}]}(\mathcal{C}) + \delta.$$

Moreover, $\mathcal{C}$ does not query $\mathcal{O}_{\text{leak}_R}$ and queries $\mathcal{O}_{\text{CCA}_L}$ at most one time.

We create game G_4 where:

- the keys $s^{\text{Test}}.k^{\text{SESSION}}$ and $s^{\text{Origin}}.k^{\text{SESSION}}$ are replaced by a uniformly sampled key.
- the keys $s^{\text{Test}}.k^{\text{MAC}}$ and $s^{\text{Origin}}.k^{\text{MAC}}$ are replaced by a uniformly sampled key.

To measure the difference of advantages between G_3 and G_4 , we construct an adversary $\mathcal{D}$ against IND-PRG, as done in the G_4 for the study of V_1 -KeySec (page 77).

This gives:

$$\left| \text{Adv}^{G_4}(\mathcal{A}) - \text{Adv}^{G_3}(\mathcal{A}) \right| \leq 2\text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{D}).$$

Study of G_4 and conclusion. In G_4 , whether b is equal to 0 or 1, the session key of the tested session is uniformly sampled. Consequently, $\text{Adv}^{G_4}(\mathcal{A}) = 0$, allowing us to conclude.

V_5 -KeySec Security in the Standard Model for the Three-Message Variant. We recall that, as defined in (16) (page 67):

$$V_5 = \text{AisResp} \wedge \text{corPafterT} \wedge \overline{\text{Orig}} \wedge (\overline{\text{CorA}} \vee \overline{\text{LeakRandT}}) \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}.$$

We study the security of AKE_3 , as AKE_2 is not secure against V_5 -KeySec. Assume that dKEM is δ -correct and γ -spread. We show that, in the standard model, there exist adversaries $\mathcal{B}, \mathcal{C}, \mathcal{D}$ and $\mathcal{E}$ such that:

$$\begin{aligned} \text{Adv}_{\text{AKE}_3}^{V_5\text{-KeySec}}(\mathcal{A}) &\leq Nl \text{Adv}_{\text{dKEM}}^{[\cdot, \text{Key-Recov}]}(\mathcal{B}) + 2Nl^2 (\delta + 2^{-\gamma}) \\ &\quad + 2Nl^2 \left(2 \text{Adv}_{\text{dKEM}}^{[\text{IND-CK-CCA}, -]}(\mathcal{C}) + 2 \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{D}) + \text{Adv}_{\text{MAC}}^{\text{sUnforg}}(\mathcal{E}) \right). \end{aligned}$$

Moreover, $\mathcal{C}$ queries $\mathcal{O}_{\text{leakR}}$ and $\mathcal{O}_{\text{CCA}_L}$ at most l times, and $\mathcal{E}$ queries $\mathcal{O}_{\text{Mac}}$ at most one time.

Proof. We observe that this proof can also be reused to establish Init-explicit authentication for DAKE_3 , DAKE_3^σ , and DUAKE_3

Reusing this demonstration to establish Init-explicit authentication.

First, we note that the advantage in Init-explicit authentication for the protocols DAKE_3 , DAKE_3^σ and DUAKE_3 is exactly the probability that an instance of the Key Secrecy game satisfies the predicate $\text{AisInit} \wedge \overline{\text{corPbeforeT}} \wedge \text{Match}$ and that the test session completes, since in these protocols the initiator sessions send the final message. With this formulation, the test session corresponds to the session that the adversary must select in the **Resp**-explicit authentication game.

We now consider the case of DAKE_3 and DAKE_3^σ ; the case of DUAKE_3 will be treated at the end of this sidebar.

We assume here that the reader is already familiar with the proof of V_5 , and we only highlight the modifications needed to obtain **Resp**-explicit authentication.

Although V_5 imposes additional restrictions on adversarial queries, the only restriction actually used in the proof is that **corrupt** is not applied to the peer of s^{Test} before s^{Test} completes.

To bound the probability of $\text{AisInit} \wedge \overline{\text{corPbeforeT}} \wedge \overline{\text{Match}}$, we can therefore reuse the games G_1, G_2, G_4 without any modification. Unlike in the proof of V_5 , the use of G_3 is unnecessary here: we do not change the winning condition, which remains $\text{AisInit} \wedge \overline{\text{corPbeforeT}} \wedge \text{Match}$ together with the completion of the test session.

The analysis of G_5 can also be reused, but here we crucially rely on the value of the message contained in the MAC constructed by the initiator, whereas in the proof of V_5 the exact value of this message is not relevant. Specifically:

- The proof of V_5 would work identically if the protocol were modified so that the message of the initiator's MAC were always 0.
- Here, however, the value of the message is essential: in G_5 it ensures that if an honest initiator generated the MAC received by s^{Test} , then this honest session is in fact a matching session with s^{Test} .

The case of DUAKE₃.

We observe that DUAKE is extremely close to the protocol AKE₃ (Figure 17), the only difference being that DUAKE includes an additional hash. This similarity between DUAKE₃ and AKE₃ implies that the argument for Resp-authentication explained above applies identically to DUAKE₃, since the additional hash is not relevant in this context.

Game G_1 : Removal of some collisions with pk_R and ct .

It is the same game as G_1 for V_1 (page 76), the difference of advantage being the same.

Note that by the definition of V_5 , there is no origin session for the test session in a valid instance of the game. However, it is still possible that a first-partner session exists.

Thanks to G_1 , we know that if a first-partner session exists, it is unique. If such a session exists, we will denote it as s^{FirstP} .

Game G_2 : Guessing multiple elements.

Using the same method as in G_2 for V_1 (see page 76), we guess $s_{\text{Peer}}^{\text{Test}}, s^{\text{Test}}$, if s^{FirstP} exists and, if so, s^{FirstP} .

We have $\text{Adv}^{G_1}(\mathcal{A}) \leq 2Nl^2 \text{Adv}^{G_2}(\mathcal{A})$.

Game G_3 : New win condition.

We create G_3 as G_2 , with a new condition for victory. An instance of G_3 is won by the adversary $\mathcal{A}$ if, and only if, s^{Test} receives a valid tag (and thus completes) and nothing makes the instance of the game invalid (as per the conditions of G_2) until s^{Test} receives the valid tag.

We stop the game at the moment when s^{Test} completes—this implies that test-session is not even requested for s^{Test} .

For G_2 to be won, any instance of G_2 needs to satisfy the win conditions of G_3 . Thus, we have $\text{Adv}^{G_2}(\mathcal{A}) \leq \text{Adv}^{G_3}(\mathcal{A})$.

An important consequence of the definition of G_3 is that in all valid instances, $s_{\text{Peer}}^{\text{Test}}$ remains uncorrupted.

Game G_4 : Replacement of $s.k$ and $s^{\text{FirstP}}.k$ (if it exists) by a uniform key.

We create game G_4 where the keys $s^{\text{Test}}.k$ and $s^{\text{FirstP}}.k$ (if s^{FirstP} exists) are replaced by a uniformly sampled key $\tilde{k}$.

To study the difference in advantage between G_3 and G_4 , we use the same argument as in the game G_3 for the study of $V_2\text{-KeySec}$ (page 78). This argument shows that we can construct an adversary $\mathcal{C} = (\mathcal{C}_1, \mathcal{C}_2)$ against the $[\text{IND-CK-CCA}, -]$ security of dKEM such that:

$$\left| \text{Adv}^{G_4}(\mathcal{A}) - \text{Adv}^{G_3}(\mathcal{A}) \right| \leq 2\text{Adv}_{\text{dKEM}}^{[\text{IND-CK-CCA}, -]}(\mathcal{C}) + \delta.$$

Moreover, $\mathcal{C}$ queries the oracles $\mathcal{O}_{\text{leak}_R}$ and $\mathcal{O}_{\text{CCA}_L}$ at most l times.

Game G_5 : Replacement of $s^{\text{Test}}.k^{\text{MAC}}$ and $s^{\text{FirstP}}.k^{\text{MAC}}$ (if it exists) by a uniform key (together with their session keys).

We create game G_5 where:

- the key $s^{\text{Test}}.k^{\text{SESSION}}$ is replaced by a uniformly sampled key. If s^{FirstM} exists, $s^{\text{FirstP}}.k^{\text{SESSION}}$ is also replaced by the same key.
- the key $s^{\text{Test}}.k^{\text{MAC}}$ is replaced by a uniformly sampled key. If s^{FirstM} exists, $s^{\text{FirstP}}.k^{\text{MAC}}$ is also replaced by the same key.

To measure the difference of advantages between G_4 and G_5 , we construct an adversary $\mathcal{D}$ against IND-PRG, as done in the G_4 for the study of $V_1\text{-KeySec}$ (page 77).

This gives:

$$\left| \text{Adv}^{G_5}(\mathcal{A}) - \text{Adv}^{G_4}(\mathcal{A}) \right| \leq 2\text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{D}).$$

Analysis of G_5 . We upper bound $\text{Adv}^{G_5}(\mathcal{A})$ by the advantage of an adversary $\mathcal{F}$ winning the game sUnforg of the MAC and that queries at most one times the oracle $\mathcal{O}_{\text{Mac}}$.

$\mathcal{F}$ has access to an oracle $\mathcal{O}_{\text{Mac}} = \text{Mac}(k, \cdot)$ for a key it does not know. When queried on an entry m , this oracle provides a tag tag such that $\text{MacVf}(k, \text{tag}, m) = 1$ and registers the couple (m, tag) in a database Q . In order to win, $\mathcal{F}$ needs to provide a couple of message and tag $(m, \text{tag}) \notin Q$ such that $\text{MacVf}(k, \text{tag}, m) = 1$.

To do so, $\mathcal{F}$ simulates G_5 to $\mathcal{A}$ with one key difference: if s^{FirstP} exists, $\mathcal{F}$ uses the oracle $\mathcal{O}_{\text{Mac}}$ with message $(s_{\text{ActorId}}^{\text{FirstP}}, s_{\text{PeerId}}^{\text{FirstP}}, s^{\text{FirstP}}.pk_R, s^{\text{FirstP}}.ct)$ in order to compute the tag computed by s^{FirstP} . It is the only situation where the oracle is used during the simulation.

Finally, when s^{Test} receives a tag tag , $\mathcal{F}$ returns $(m^{\text{test}}, \text{tag})$, to the challenger of sUnforg, where ³⁸

$$m^{\text{test}} = (s_{\text{PeerId}}^{\text{Test}}, s_{\text{ActorId}}^{\text{Test}}, s^{\text{Test}}.pk_R, s^{\text{Test}}.ct) .$$

³⁸ Note that the value of the message used to compute this tag differs across AKE_3 , DAKE_3 , DAKE_3^σ , and DUAKE_3 . However, in each case it consists precisely of the transcript of the sent and received elements (except for the resulting tag).

We note that $\mathcal{F}$ provides a perfect simulation of G_4 to $\mathcal{A}$ until it stops. Moreover, when the simulated instance of G_5 is won, we claim that $\mathcal{F}$ wins the game sUnforg .

To see this, we first recall that in a valid instance of G_4 , the tag tag received by s^{Test} satisfies $\text{MacVf}(k, \text{tag}, m^{\text{test}}) = 1$. We now need to verify that $(m^{\text{test}}, \text{tag}) \notin Q$.

There are two possible cases:

- s^{FirstP} exists. In this situation, we know that the tag sent to s^{Test} is not equal to the tag created with the oracle—else, the session would be invalid as s^{FirstP} would be an origin session. Consequently, $(m^{\text{test}}, \text{tag}) \notin Q$.
- s^{FirstP} does not exist. As a result, $Q = \emptyset$, and thus $(m^{\text{test}}, \text{tag}) \notin Q$.

Hence, $\text{Adv}^{G_5}(\mathcal{A}) \leq \text{Adv}_{\text{MAC}}^{\text{sUnforg}}(\mathcal{F})$ and this concludes the proof.

E.5 Key Secrecy of DUAKE

We can observe that DUAKE is extremely close of the protocol depicted in Figure 17 (Appendix E.1), the only difference being that DUAKE uses an additional hash.

Moreover, the predicates $F_1^U, \dots, F_5^U$ (defined in Proposition 1 of Section C.4) are close to the predicates $V_1, \dots, V_5$ under which the protocol of Figure 17 is studied (see Appendix E.1). More precisely, the only relevant difference is that F_1^U and F_3^U uses Orig while V_1 and V_3 use FirstMatch .³⁹

As a result, we can reuse the security proof of the protocol of Figure 17 if we can show that, in F_1^U and F_3^U , the addition of the hash in DUAKE allows to say that:

$$(\text{Aislnit} \wedge \text{Orig}) \Rightarrow (\text{Aislnit} \wedge \text{FirstMatch}) .$$

We can directly see that this is true if there are no collisions between the hash computed by any of the responder sessions of the game. The security proof under $G_1, \dots, G_5$ of the AKE of Figure 17 can thus be reused by just inserting an additional game to avoid the collisions of H .

As a result, we proved the Theorem 8.

F Explicit Authentication of DAKE and its Variants

In this section, we precisely study the explicit authentication of the protocols DAKE, DAKE^σ , or DUAKE. This analysis will establish that:

- Init-explicit authentication is achieved by DAKE_3 , DAKE_3^σ and DUAKE_3 .
- Resp-explicit authentication is achieved by DAKE_3 and DAKE_3^σ .

³⁹ They are also some differences concerning requests that have absolutely no effect. More precisely, concerning using eph-key or corrupt on a responder session—which only return $\perp$ and thus gives no advantage to the adversary.

The complete bounds are given in Theorems 2 (stated in Section 3.3 and restated below for readability), 9 and 10.

In order to prove them, we will actually reuse the work made during the proof of key secrecy of these protocols. More precisely,

- The proof of Init-explicit authentication of $\text{DAKE}_3, \text{DAKE}_3^\sigma, \text{DUAKE}_3$ can be derived from their key secrecy proof for the predicate V_5 , as explained in the sidebar paragraph of Section E.4, page 84.
- The proof of Resp-explicit authentication of DAKE_2 and DAKE_3 can be derived from their key secrecy proof for the predicates $U \wedge \text{eCK}^w$ and $U \wedge \text{eCK-PFS}$, as explained in the sidebar paragraph of Section E.3, page 69.
- The proof of Resp-explicit authentication of DAKE_2^σ and DAKE_3^σ can be derived from their key secrecy proof for the predicates $U \wedge \text{eCK}^w$ and $U \wedge \text{eCK-PFS}$, as explained in the sidebar paragraph of Section E.3, page 73.

Theorem 2 (Explicit Authentication for DAKE). *Let $\mathcal{A}$ be a PPT adversary against Init-ExplicitAuth for DAKE_3 . Suppose it activates at most N parties and, for each party P , at most l sessions with actor or peer P . Assume that the underlying 2K-KEM is γ -spread and δ -correct. Then, in the standard model, there exist PPT adversaries $\mathcal{B}, \mathcal{C}, \mathcal{D}$ and $\mathcal{E}$ such that:*

$$\begin{aligned} \text{Adv}_{\text{DAKE}_3}^{\text{Init-ExplicitAuth}}(\mathcal{A}) &\leq Nl \text{Adv}_{\text{dKEM}}^{[\cdot, \text{Key-Recov}]}(\mathcal{B}) + 2Nl^2 (\delta + 2^{-\gamma}) \\ &\quad + 2Nl^2 \left(2 \text{Adv}_{\text{dKEM}}^{\text{IND-CK-CCA}, -}(\mathcal{C}) + 2 \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{D}) + \text{Adv}_{\text{MAC}}^{\text{sUnforg}}(\mathcal{E}) \right). \end{aligned}$$

Moreover, $\mathcal{C}$ queries $\mathcal{O}_{\text{leak}_R}$ and $\mathcal{O}_{\text{CCA}_L}$ at most l times, and $\mathcal{E}$ queries $\mathcal{O}_{\text{Mac}}$ at most one time.

Now let $\Pi \in \{\text{DAKE}_2, \text{DAKE}_3\}$ and suppose that $\mathcal{A}$ is a PPT adversary against Resp-ExplicitAuth for Π . Assume the same conditions on $\mathcal{A}$ as above, and that the standard KEM is δ' -correct. Then, in the standard model, there exist PPT adversaries $\mathcal{B}, \mathcal{C}, \mathcal{D}, \mathcal{E}$ such that $\text{Adv}_{\Pi}^{\text{Resp-ExplicitAuth}}(\mathcal{A})$ is upper-bounded by:

$$\text{Adv}_{\text{MAC}}^{\text{MacColl}}(\mathcal{B}) + N \text{Adv}_{\text{KEM}}^{\text{Key-Recov}}(\mathcal{C}) + N^2 l \left(2 \text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{D}) + l\delta' + \text{Adv}_{\text{MAC}}^{\text{sUnforg}}(\mathcal{E}) \right).$$

Moreover, $\mathcal{D}$ queries $\mathcal{O}_{\text{CCA}}$ at most l times and $\mathcal{E}$ queries $\mathcal{O}_{\text{Mac}}$ at most l times.

Theorem 9 (Resp-Explicit Authentication for DAKE_3^σ and DAKE_2^σ). *Let $\Pi \in \{\text{DAKE}_3^\sigma, \text{DAKE}_2^\sigma\}$ and let $\mathcal{A}$ be a PPT adversary against Resp-ExplicitAuth for Π . Suppose it activates at most N parties and, for each party P , at most l sessions with actor or peer P . Then, in the standard model, there exist PPT adversaries $\mathcal{B}, \mathcal{C}, \mathcal{D}$ and $\mathcal{E}$ such that:*

$$\begin{aligned} \text{Adv}_{\Pi}^{\text{Resp-ExplicitAuth}}(\mathcal{A}) &\leq \text{Adv}_{\text{Sign}}^{\text{MBS}}(\mathcal{B}) + \text{Adv}_{\text{Sign}}^{\text{M-S-UEO}}(\mathcal{C}) + N \text{Adv}_{\text{Sign}}^{\text{Key-Recov}}(\mathcal{D}) \\ &\quad + N^2 l \text{Adv}_{\text{Sign}}^{\text{sUF-CMA}}(\mathcal{E}). \end{aligned}$$

Moreover, $\mathcal{E}$ queries $\mathcal{O}_{\text{Sign}}$ at most l times.

Theorem 10 (Init-Explicit Authentication for DAKE_3^σ , and DUAKE_3). *Let $\Pi \in \{\text{DAKE}_3^\sigma, \text{DUAKE}_3\}$ and let $\mathcal{A}$ be a PPT adversary against Init-ExplicitAuth for Π . Suppose it activates at most N parties and, for each party P , at most l sessions with actor or peer P . Assume that the underlying 2K-KEM is γ -spread and δ -correct. Then, in the standard model, there exist PPT adversaries $\mathcal{B}, \mathcal{C}, \mathcal{D}$ and $\mathcal{E}$ such that:*

$$\begin{aligned} \text{Adv}_{\Pi}^{\text{Init-ExplicitAuth}}(\mathcal{A}) &\leq Nl \text{Adv}_{\text{dKEM}}^{[\cdot, \text{Key-Recov}]}(\mathcal{B}) + 2Nl^2 (\delta + 2^{-\gamma}) \\ &\quad + 2Nl^2 \left(2 \text{Adv}_{\text{dKEM}}^{[\text{IND-CK-CCA}, -]}(\mathcal{C}) + 2 \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{D}) + \text{Adv}_{\text{MAC}}^{\text{sUnforg}}(\mathcal{E}) \right). \end{aligned}$$

Moreover, $\mathcal{C}$ queries $\mathcal{O}_{\text{leak}_R}$ and $\mathcal{O}_{\text{CCA}_L}$ at most l times, and $\mathcal{E}$ queries $\mathcal{O}_{\text{Mac}}$ at most one time.

G Instantiating the Xue et al. AKE with One-Sided Chosen-Key Secure Double-KEMs

In [59], Xue *et al.* define an AKE that performs two encapsulations using a 2K-KEM, rather than one encapsulation with a 2K-KEM and another with a standard KEM, as in DAKE. In [59, Theorem 1], they require the 2K-KEM to satisfy both $[\text{OW-CK-CCA}, -]$ and $[-, \text{OW-CK-CPA}]$ security. However, Maul does not meet $[-, \text{OW-CK-CPA}]$, providing only $[-, \text{OW-CPA}]$ security instead.^{40,41}

However, we show in this section that a careful study of the demonstration reveals that [59, Theorem 1] still holds if we replace the $[-, \text{OW-CK-CPA}]$ security requirement with $[-, \text{OW-CPA}]$. As a result, Maul can be used with their AKE. The rest of this section is devoted to explaining this.

The security of the AKE in [59] is proven in [59, Theorem 1]. We can directly see that the only place in the proof where the 2K-KEM's security is invoked is [59, Lemma 2], and [59, Table 4, p.12] shows that $[-, \text{OW-CK-CPA}]$ is used only to bound the event $\text{AskH} \wedge E_5$. We therefore focus on this part of the proof, which appears in [59, Appendix 1, p.31].

To bound the probability of $\text{AskH} \wedge E_5$, the proof constructs an adversary $\mathcal{S}$ for the $[-, \text{OW-CK-CPA}]$ game. $\mathcal{S}$ receives a right-hand key pair $(\text{pk}_{A0}, \text{sk}_{A0})$ from the $[-, \text{OW-CK-CPA}]$ challenger, then uses the $\mathcal{O}_{\text{leak}_L}$ ⁴² oracle to obtain a left-hand key pair $(\text{pk}_A, \text{sk}_A)$, and finally provides pk_A to the challenger to obtain a challenge ciphertext and its corresponding (decapsulated) key. Since this is the $[-, \text{OW-CK-CPA}]$ game, the challenger only generates $(\text{pk}_{A0}, \text{sk}_{A0})$; the left-hand key pk_A must be provided by $\mathcal{S}$. However, the key observation is that in this setting, $\mathcal{S}$ could equivalently generate $(\text{pk}_A, \text{sk}_A)$ using Keygen_L ⁴³ instead of $\mathcal{O}_{\text{leak}_L}$.

⁴⁰ In [59], the notation $[-, \text{OW-CPA}]$ actually refers to chosen-key security. Here, we use our notation to distinguish between chosen-key security ($[-, \text{OW-CK-CPA}]$) and standard one-way security ($[-, \text{OW-CPA}]$).

⁴¹ Note that indistinguishability security directly implies one-way security.

⁴² Called $\mathcal{O}_{\text{leak}_1}$ in [59].

⁴³ Note that in [59] they instead use the notation KeyGen1 for Keygen_L , and KeyGen0 for Keygen_R .

We thus propose the following change to the proof: replace the $[-, \text{OW-CK-CPA}]$ game with the $[-, \text{OW-CPA}]$ game, where the $[-, \text{OW-CPA}]$ challenger generates both key pairs $(\text{pk}_{A_0}, \text{sk}_{A_0})$ and $(\text{pk}_A, \text{sk}_A)$ via Keygen_L , and sends $(\text{pk}_A, \text{sk}_A)$ to $\mathcal{S}$, along with pk_{A_0} , the challenge ciphertext, and its corresponding key. To ensure that using $[-, \text{OW-CPA}]$ leads to the same conclusion, we observe that in $[-, \text{OW-CK-CPA}]$, creating $(\text{pk}_A, \text{sk}_A)$ via Keygen_L is equivalent to creating it via $\mathcal{O}_{\text{leak1}}$, $\mathcal{O}_{\text{leak1}}$ being impactful only in the CCA game $[-, \text{OW-CK-CCA}]$ ⁴⁴. Thus, in this case, using the $[-, \text{OW-CK-CPA}]$ game with $\mathcal{S}$ calling $\mathcal{O}_{\text{leak1}}$ is equivalent to using the $[-, \text{OW-CPA}]$ game with the challenger providing $(\text{pk}_A, \text{sk}_A)$ by calling Keygen_L . This change preserves the logic of the proof and therefore $[-, \text{OW-CPA}]$ security suffices.

With this small modification to the proof, the 2K-KEM is only required to be $[\text{OW-CK-CCA}, -]$ and $[-, \text{OW-CPA}]$, making the AKE of [59] fully compatible with Maul.

H Additional Details Regarding Hint-MLWE and Maul_{pKE}

H.1 Hint-MLWE Hardness

Lattices and Gaussians A lattice $\Lambda \subset \mathbb{R}^n$ is defined as a discrete subgroup of $\mathbb{R}^n$. For a lattice $\Lambda \subset \mathbb{R}^n$, $\text{span}(\Lambda)$ is the smallest linear subspace that contains Λ . The n -dimensional Gaussian function $\rho : \mathbb{R}^n \rightarrow (0, 1]$ is defined as $\rho(\mathbf{x}) := \exp(-\pi \|\mathbf{x}\|^2)$. Given a positive definite matrix $\Sigma = \mathbf{B}\mathbf{B}^\top$, we can apply the invertible linear transformation $\mathbf{B}$ to define a Gaussian function $\rho_{\sqrt{\Sigma}}(\mathbf{x}) := \rho(\mathbf{B}^{-1}\mathbf{x}) = \exp(-\pi \mathbf{x}^\top \Sigma^{-1} \mathbf{x})$. For any $\mathbf{c} \in \text{span}(\Sigma)$, the Gaussian function centered at $\mathbf{c}$ with parameter $\sqrt{\Sigma}$ is $\rho_{\sqrt{\Sigma}, \mathbf{c}}(\mathbf{x}) := \rho_{\sqrt{\Sigma}}(\mathbf{x} - \mathbf{c})$. The discrete Gaussian distribution over a lattice Λ centered at $\mathbf{c}$ with parameter $\sqrt{\Sigma}$ is defined by the probability mass function $D_{\Lambda, \sqrt{\Sigma}, \mathbf{c}}(\mathbf{x}) := \frac{\rho_{\sqrt{\Sigma}, \mathbf{c}}(\mathbf{x})}{\rho_{\sqrt{\Sigma}, \mathbf{c}}(\Lambda)}$ for all $\mathbf{x} \in \Lambda$, where $\rho_{\sqrt{\Sigma}, \mathbf{c}}(\Lambda) := \sum_{\mathbf{x} \in \Lambda} \rho_{\sqrt{\Sigma}, \mathbf{c}}(\mathbf{x})$. When $\mathbf{c} = \mathbf{0}$, we omit the center from the notation. When $\Sigma = \sigma^2 \cdot \mathbf{I}_n$ for $\sigma > 0$ and $\mathbf{I}_n$ the $n \times n$ identity matrix, we write $D_{\Lambda, \sigma, \mathbf{c}}$ and refer to σ as the width parameter of $D_{\Lambda, \sigma, \mathbf{c}}$. We write $x \leftarrow \$ D_{\mathbb{Z}^n, \sigma, \mathbf{c}}$ for $x \in \mathcal{R}$ to denote that the corresponding coefficient vector of x is sampled from $D_{\mathbb{Z}^n, \sigma, \mathbf{c}}$. For any positive $\varepsilon > 0$, the smoothing parameter $\eta_\varepsilon(\Lambda)$ is the smallest $s > 0$ such that $\rho(s \cdot \Lambda^*) \leq 1 + \varepsilon$, where $\Lambda^* := \{\mathbf{x} \in \text{span}(\Lambda) \mid \langle \mathbf{x}, \Lambda \rangle \in \mathbb{Z}\}$ is the dual lattice of Λ .

Hint-MLWE Hardness In the following section, we work in the coefficient embedding, meaning that elements of $\mathcal{R}$ can be mapped to negacyclic matrices in $\mathbb{Z}^{n \times n}$ or to vectors in $\mathbb{Z}^n$.

Lemma 6 (Lemma 5 [38]). *For $0 < \varepsilon < 1/2$, a lattice Λ and a positive-definite matrix Σ , we have*

$$s_1(\Sigma^{-1}) \leq \eta_\varepsilon(\Lambda)^{-2} \quad \Rightarrow \quad \sqrt{\Sigma} \geq \eta_\varepsilon(\Lambda).$$

⁴⁴ The keys that can be used by the decapsulation oracle depends on the queries made to $\mathcal{O}_{\text{leak1}}$.

Lemma 7 (Simplified Convolution Lemma [38]). *Let Σ_1, Σ_2 be positive definite matrices such that*

$$\Sigma_3^{-1} := \Sigma_1^{-1} + \Sigma_2^{-1}$$

satisfies $\sqrt{\Sigma_3} \geq \eta_\varepsilon(\mathbb{Z}^n)$ for $0 < \varepsilon < 1/2$. Then for an arbitrary $\mathbf{c} \in \mathbb{Z}^n$, the distribution

$$\{\mathbf{x}_1 + \mathbf{x}_2 \mid \mathbf{x}_1 \leftarrow D_{\mathbb{Z}^n, \sqrt{\Sigma_1}}, \mathbf{x}_2 \leftarrow D_{\mathbb{Z}^n, \mathbf{c}, \sqrt{\Sigma_2}}\}$$

is within statistical distance 2ε of $D_{\mathbb{Z}^n, \mathbf{c}, \sqrt{\Sigma_1 + \Sigma_2}}$.

Lemma 8 (Derived from [38]). *Let ℓ, d be positive integers, $\mathbf{H}$ be a $d \times \ell$ matrix sampled from $\mathcal{F}$ in $\mathbb{Z}^{d \times \ell}$, $\sigma_1, \sigma_2 > 0$ be two real numbers and $\Sigma_0 := (\frac{1}{\sigma_1^2} \cdot \mathbf{I}_\ell + \frac{1}{\sigma_2^2} \mathbf{H}^\top \cdot \mathbf{H})^{-1}$. If Σ_0 is positive-definite, then the following two distributions over $\mathbb{Z}^\ell \times \mathbb{Z}^d$ are statistically identical:*

$$\begin{aligned} & \{(\mathbf{r}, \mathbf{z}) \mid \mathbf{r} \leftarrow D_{\mathbb{Z}^\ell, \sigma_1}, \mathbf{y} \leftarrow D_{\mathbb{Z}^d, \sigma_2}, \mathbf{z} = \mathbf{H} \cdot \mathbf{r} + \mathbf{y}\}, \\ & \left\{(\hat{\mathbf{r}}, \mathbf{z}) \mid \mathbf{r} \leftarrow D_{\mathbb{Z}^\ell, \sigma_1}, \mathbf{y} \leftarrow D_{\mathbb{Z}^d, \sigma_2}, \mathbf{z} = \mathbf{H} \cdot \mathbf{r} + \mathbf{y}, \mathbf{c} = \frac{1}{\sigma_2^2} \Sigma_0 \cdot \mathbf{H}^\top \mathbf{z}, \hat{\mathbf{r}} \leftarrow D_{\mathbb{Z}^\ell, \mathbf{c}, \Sigma_0}\right\}. \end{aligned}$$

Proof. The probability that the first random variable outputs $(\mathbf{v}, \mathbf{w}) \in \mathbb{Z}^\ell \times \mathbb{Z}^d$ can be written as follows:

$$\begin{aligned} & \Pr[\mathbf{r} = \mathbf{v}, \mathbf{H} \cdot \mathbf{r} + \mathbf{y} = \mathbf{w} \mid \mathbf{r} \leftarrow D_{\mathbb{Z}^\ell, \sigma_1}, \mathbf{y} \leftarrow D_{\mathbb{Z}^d, \sigma_2}] \\ &= D_{\mathbb{Z}^\ell, \sigma_1}(\mathbf{v}) \cdot D_{\mathbb{Z}^d, \sigma_2}(\mathbf{w} - \mathbf{H}\mathbf{v}) \\ &\propto \exp \left[-\pi \left(\frac{1}{\sigma_1^2} \cdot \mathbf{v}^\top \mathbf{v} + \frac{1}{\sigma_2^2} \cdot (\mathbf{w} - \mathbf{H}\mathbf{v})^\top (\mathbf{w} - \mathbf{H}\mathbf{v}) \right) \right] \\ &= \exp \left[-\pi \left((\mathbf{v} - \mathbf{c})^\top \Sigma_0^{-1} (\mathbf{v} - \mathbf{c}) - \mathbf{c}^\top \Sigma_0^{-1} \mathbf{c} + \frac{1}{\sigma_2^2} \cdot \mathbf{w}^\top \mathbf{w} \right) \right] \end{aligned}$$

where $\mathbf{c} = \frac{1}{\sigma_2^2} \Sigma_0 \cdot \mathbf{H}^\top \mathbf{w}$. Hence, the conditional probability $\Pr[\mathbf{r} = \mathbf{v} \mid \mathbf{H} \cdot \mathbf{r} + \mathbf{y} = \mathbf{w}]$ is proportional to $\exp[-\pi(\mathbf{v} - \mathbf{c})^\top \Sigma_0^{-1}(\mathbf{v} - \mathbf{c})]$, which implies

$$\Pr[\mathbf{r} = \mathbf{v} \mid \mathbf{H} \cdot \mathbf{r} + \mathbf{y} = \mathbf{w}] \equiv \rho_{\sqrt{\Sigma_0}}(\mathbf{v} - \mathbf{c}) \equiv \Pr[\hat{\mathbf{r}} = \mathbf{v} \mid \mathbf{H} \cdot \mathbf{r} + \mathbf{y} = \mathbf{w}].$$

Therefore, the given two distributions are statistically identical. $\square$

Theorem 4 (Hardness of Hint-MLWE, adapted from [38]).

Let k, m, ℓ, d, q, J be as in Definition 2, and let $\mathcal{F}$ be a distribution over $\mathcal{S}_B := \{\mathbf{H} \in \mathcal{R}_q^{\ell \times d} \mid s_1(\mathbf{H})^2 \leq B\}$, for $B > 0$. Let $\sigma_1, \sigma_2 > 0$, and define $\sigma > 0$ via $\frac{1}{\sigma^2} = 2 \left(\frac{1}{\sigma_1^2} + \frac{B}{\sigma_2^2} \right)$. Let $\chi_h := D_{\mathbb{Z}^{\ell \cdot n}, \sigma_2}$, and let χ be a distribution over $\mathcal{R}^{k+m-d}$. Define χ_σ over $\mathcal{R}^{k+m}$ as the distribution where coordinates in J are drawn from $D_{\mathbb{Z}^{d \cdot n}, \sigma}$ and the rest from χ .

If $\sigma \geq \sqrt{2} \cdot \eta_\varepsilon(\mathbb{Z}^{d \cdot n})$ for some $0 < \varepsilon \leq 1/2$, then there exists an efficient reduction from $\text{MLWE}_{\mathcal{R}, k, m, q}^{\chi_\sigma}$ to $\text{Hint-MLWE}_{\mathcal{R}, k, m, q, \ell, d, J}^{\chi_{\sigma_1}, \chi_h, \mathcal{F}}$ that reduces the advantage by at most 2ε .

Proof. Let $\mathbf{H} \leftarrow \mathcal{F}$, $\Sigma_0 = (\frac{1}{\sigma_1^2} \cdot \mathbf{I}_{nd} + \frac{1}{\sigma_2^2} \mathbf{H}^\top \cdot \mathbf{H})^{-1}$ and $(\mathbf{A}, \mathbf{b}) \in R_q^{m \times k} \times R_q^m$ be a given $\text{MLWE}_{R,k,m,q}^{\chi_\sigma}$ instance. We suppose without loss of generality that the coordinates sample from χ are the last of $(\begin{smallmatrix} s \\ \mathbf{e} \end{smallmatrix})$. To begin, we sample:

$$\begin{aligned} \mathbf{r}_1 &\leftarrow D_{\mathbb{Z}^{dn}, \sigma_1}, \quad \mathbf{r}_2 \leftarrow \chi, \text{ and } \mathbf{y} \leftarrow D_{\mathbb{Z}^{tn}, \sigma_2}, \\ \mathbf{t}_1 &\leftarrow D_{\mathbb{Z}^{dn}, \mathbf{c}, \sqrt{\Sigma_0 - \sigma^2 \cdot \mathbf{I}_{n\ell}}} \quad \text{for } \mathbf{c} = \frac{1}{\sigma_2^2} \Sigma_0 \mathbf{H}^\top (\mathbf{H} \mathbf{r} + \mathbf{y}), \\ \mathbf{t}_2 &\leftarrow \mathbf{0}^{n \cdot (k+m-d)}. \end{aligned}$$

Define $(\mathbf{r}_1, \mathbf{r}_2)$ and $(\mathbf{t}_1, \mathbf{t}_2)$ as $\mathbf{r}$ and $\mathbf{t}$ respectively. Note that $\Sigma_0 - \sigma^2 \cdot \mathbf{I}_{nd}$ is positive-definite, since the smallest singular value of Σ_0 is higher than σ^2 (derived from $s_1(\mathbf{H})^2 \leq B$).

Then, we use the sampled polynomials to transform the given MLWE instance $(\mathbf{A}, \mathbf{b})$ into

$$(\mathbf{A}, \mathbf{b} + [\mathbf{A} \mid \mathbf{I}_{nm}] \mathbf{t}, \mathbf{H}, \mathbf{z} = \mathbf{H} \cdot \mathbf{r}_1 + \mathbf{y}).$$

We distinguish two cases: the first where $\mathbf{b}$ is a MLWE sample, and the second where it is uniform.

First, assume that $\mathbf{b} = [\mathbf{A} \mid \mathbf{I}_{nm}] \mathbf{r}'$ for $\mathbf{r}' \leftarrow \chi_\sigma$. Then, we have

$$\mathbf{b} + [\mathbf{A} \mid \mathbf{I}_{nm}] \mathbf{t} = [\mathbf{A} \mid \mathbf{I}_{nm}] (\mathbf{r}' + \mathbf{t}),$$

where $\mathbf{r}' + \mathbf{t}$ follows the distribution $(D_{\mathbb{Z}^{nd}, \sigma} + D_{\mathbb{Z}^{nd}, \mathbf{c}, \sqrt{\Sigma_0 - \sigma^2 \cdot \mathbf{I}_{nd}}}) \times \chi$.

To use the Convolution Lemma, we aim to show that $\sqrt{\Sigma_3} \geq \eta_\varepsilon(\mathbb{Z}^{nd})$, where $\Sigma_3^{-1} := \sigma^{-2} \cdot \mathbf{I}_{nd} + (\Sigma_0 - \sigma^2 \cdot \mathbf{I}_{nd})^{-1}$. By Lemma 6, it suffices to show that $s_1(\Sigma_3^{-1}) \leq \eta_\varepsilon(\mathbb{Z}^{nd})^{-2}$.

Recall that the smallest singular value of $\Sigma_0 - \sigma^2 \cdot \mathbf{I}_{nd}$ is at least σ^2 . Therefore, we obtain

$$s_1(\Sigma_3^{-1}) \leq \sigma^{-2} + s_1((\Sigma_0 - \sigma^2 \cdot \mathbf{I}_{nd})^{-1}) \leq \sigma^{-2} + \sigma^{-2} = 2\sigma^{-2} \leq \eta_\varepsilon(\mathbb{Z}^{nd})^{-2}.$$

So the distributions $D_{\mathbb{Z}^{nd}, \sigma} + D_{\mathbb{Z}^{nd}, \mathbf{c}, \sqrt{\Sigma_0 - \sigma^2 \cdot \mathbf{I}_{nd}}}$ are within the statistical distance 2ε of $D_{\mathbb{Z}^{nd}, \mathbf{c}, \sqrt{\Sigma_0}}$. Therefore, the distribution of

$$(\mathbf{A}, \mathbf{b} + [\mathbf{A} \mid \mathbf{I}_{nm}] \mathbf{t}, \mathbf{H}, \mathbf{z} = \mathbf{H} \cdot \mathbf{r}_1 + \mathbf{y})$$

is within statistical distance 2ε of

$$(\mathbf{A}, [\mathbf{A} \mid \mathbf{I}_{nm}] \tilde{\mathbf{r}}, \mathbf{H}, \mathbf{z} = \mathbf{H} \cdot \mathbf{r}_1 + \mathbf{y}) \quad \text{for } \tilde{\mathbf{r}} \leftarrow D_{\mathbb{Z}^{nd}, \mathbf{c}, \sqrt{\Sigma_0}} \times \chi.$$

As the last step, we apply Lemma 8 on $(\tilde{\mathbf{r}}, \mathbf{z})$, then by independence its distribution is identical to that of $(\mathbf{r}_1, \mathbf{r}_2, \mathbf{z})$. As a result, the distribution of

$$(\mathbf{A}, [\mathbf{A} \mid \mathbf{I}_{nm}] \tilde{\mathbf{r}}, \mathbf{H}, \mathbf{z} = \mathbf{H} \cdot \mathbf{r}_1 + \mathbf{y}),$$

is identical to that of

$$(\mathbf{A}, [\mathbf{A} \mid \mathbf{I}_{nm}]\mathbf{r}, \mathbf{H}, \mathbf{z} = \mathbf{H} \cdot \mathbf{r}_1 + \mathbf{y}),$$

which exactly follows the distribution of samples from $\text{Hint-MLWE}_{\mathcal{R},k,m,q,\ell,d,J}^{\chi_{\sigma_1},\chi_h,\mathcal{F}}$ when $\mathbf{b}$ is not sample uniformly.

Secondly, if $\mathbf{b} \leftarrow \mathcal{U}(R_q^m)$, then $(\mathbf{A}, \mathbf{b} + [\mathbf{A} \mid \mathbf{I}_{nm}]\mathbf{t}, \mathbf{H}, \mathbf{z} = \mathbf{H} \cdot \mathbf{r}_1 + \mathbf{y})$ follows the same distribution as $(\mathbf{A}, \mathbf{u}, \mathbf{H}, \mathbf{z} = \mathbf{H} \cdot \mathbf{r}_1 + \mathbf{y})$ where $\mathbf{u} \leftarrow \mathcal{U}(R_q^m)$.

The reduction is then correct and reduces the advantage by at most 2ε . $\square$

H.2 Complete Correctness and Security Proofs for Maul_{PKE}

In this section, we provide the complete proofs of Lemma 1 and Theorem 5.

Lemma 1 (Maul_{PKE} correctness). *Using the notation from Figure 8, the scheme Maul_{PKE} is δ -correct provided that*

$$\Pr \left[\left\| \begin{pmatrix} f_L + e_{c_L} + \mathbf{s}_C^T \mathbf{e}_L - (\mathbf{e}_C + \mathbf{e}_{cc})^T \mathbf{s}_L \\ f_R + e_{c_R} + \mathbf{s}_C^T \mathbf{e}_R - (\mathbf{e}_C + \mathbf{e}_{cc})^T \mathbf{s}_R \end{pmatrix} \right\|_{\infty} \leq \frac{q}{4} \right] > 1 - \delta,$$

where, $\mathbf{e}_{cc} = \text{Decomp}_{(q,d_u)}(\mathbf{c}_C) - \mathbf{A}^T \mathbf{s}_C + \mathbf{e}_C$, $e_{c_L} = \text{Decomp}_{(q,d_v)}(\mathbf{c}_L) - \mathbf{s}_C^T \mathbf{t}_L + f_L + m_L$, and $e_{c_R} = \text{Decomp}_{(q,d_v)}(\mathbf{c}_R) - \mathbf{s}_C^T \mathbf{t}_R + f_R + m_R$.

Proof. For a well-formed ciphertext $(\mathbf{c}_C, \mathbf{c}_L, \mathbf{c}_R)$, define:

$$\begin{aligned} \mathbf{v}_L &:= \text{Decomp}_{(q,d_v)}(\mathbf{c}_L) - \text{Decomp}_{(q,d_u)}(\mathbf{c}_C)^T \mathbf{s}_L \\ &= \lceil q/2 \rceil \cdot m_L + e_{c_L} + \mathbf{s}_C^T \mathbf{t}_L + f_L - (\mathbf{e}_{cc} + \mathbf{A}^T \mathbf{s}_C + \mathbf{e}_C)^T \mathbf{s}_L \\ &= \lceil q/2 \rceil \cdot m_L + e_{c_L} + \mathbf{s}_C^T \mathbf{e}_L + f_L - (\mathbf{e}_{cc} + \mathbf{e}_C)^T \mathbf{s}_L, \end{aligned}$$

and similarly $\mathbf{v}_R := \lceil q/2 \rceil \cdot m_R + e_{c_R} + \mathbf{s}_C^T \mathbf{e}_R + f_R - (\mathbf{e}_{cc} + \mathbf{e}_C)^T \mathbf{s}_R$.

Thus, correct decryption—*i.e.* recovering $(m_L, m_R) = \lfloor (2/q) \cdot (\mathbf{v}_L, \mathbf{v}_R) \rfloor$ —is guaranteed if $\|\mathbf{v}_L - \lceil q/2 \rceil \cdot m_L\|_{\infty} \leq q/4$ and $\|\mathbf{v}_R - \lceil q/2 \rceil \cdot m_R\|_{\infty} \leq q/4$. $\square$

Theorem 5 ([IND-CPA, IND-CPA] Security of Maul_{PKE}). *Let consider $\mathsf{X} \in \{[\text{IND-CPA}, -], [-, \text{IND-CPA}]\}$. Let $\mathcal{A}$ be a PPT adversary against the X security of Maul_{PKE} , there exist PPT adversaries $\mathcal{B}, \mathcal{C}$ with same time complexity as $\mathcal{A}$ such that:*

$$\text{Adv}_{\text{Maul}_{\text{PKE}}}^{\mathsf{X}}(\mathcal{A}) \leq \text{Adv}_{k,k,\mathcal{B}^k,\mathcal{B}^k}^{\text{MLWE}}(\mathcal{B}) + \text{Adv}_{\mathcal{R},k,k+1,q,1,2k,J}^{\text{Hint-MLWE}_{\chi,\chi_h,\mathcal{F}}}(\mathcal{C}).$$

Where $\mathcal{F} = \mathcal{B}^k \times (-\mathcal{B}^k)$, $\chi_h = \mathcal{D}$, $\chi = \mathcal{B}^{2k} \times \mathcal{D}$ and the subset J being the $2k$ first elements.

Proof. By symmetry of the construction, it suffices to prove $[\text{IND-CPA}, -]$ security. Furthermore, it is enough to consider the variant of the scheme in which the **Compress** and **Decompress** functions are replaced by the identity; we assume this is the case in the following.

$G_0(\mathcal{A}) = [\text{IND-CPA}, -](\mathcal{A})$	$G_2(\mathcal{A})$
$\mathbf{A} \leftarrow \mathcal{U}(\mathcal{R}_q^{k \times k})$	$\mathbf{B} \leftarrow \mathcal{U}(\mathcal{R}_q^{(k+1) \times k})$
$\text{sk}_L = \mathbf{s}_L \leftarrow \mathcal{B}^k; \text{sk}_R = \mathbf{s}_R \leftarrow \mathcal{B}^k$	$\bar{\mathbf{s}} = \mathbf{s}_C \leftarrow \mathcal{X} = \mathcal{B}^k$
$\mathbf{e}_L \leftarrow \mathcal{B}^k; \mathbf{e}_R \leftarrow \mathcal{B}^k$	$\bar{\mathbf{e}} = (\mathbf{e}_C^\top, f_L)^\top \leftarrow \mathcal{X} = \mathcal{B}^k \times \mathcal{D}$
$\text{pk}_L = \mathbf{t}_L \leftarrow \mathbf{A}\mathbf{s}_L + \mathbf{e}_L$	$(\tilde{\mathbf{c}}_C^\top, \tilde{\mathbf{c}}_L)^\top \leftarrow \mathbf{B} \bar{\mathbf{s}} + \bar{\mathbf{e}}$
$\text{pk}_R = \mathbf{t}_R \leftarrow \mathbf{A}\mathbf{s}_R + \mathbf{e}_R$	$\mathbf{H} = (\mathbf{e}_R^\top, -\mathbf{s}_R^\top) \leftarrow \mathcal{F} = \mathcal{B}^k \times (-\mathcal{B}^k)$
$(\text{state}, m_{L,0}, m_{L,1}, m_R) \leftarrow$ $\mathcal{A}_1(\mathbf{A}, \text{pk}_R, (\text{sk}_R, \mathbf{e}_R), \text{pk}_L)$	$f_R^\top \leftarrow \mathcal{X}_h = \mathcal{D}$
$(\mathbf{s}_C, \mathbf{e}_C) \leftarrow \mathcal{B}^k \times \mathcal{B}^k$	$\mathbf{r} = (\mathbf{s}_C^\top, \mathbf{e}_C^\top)^\top$
$(f_L, f_R) \leftarrow \mathcal{D} \times \mathcal{D}$	$\mathbf{z} = \mathbf{H}\mathbf{r} + f_R$
$\mathbf{c}_C \leftarrow \mathbf{A}^\top \mathbf{s}_C + \mathbf{e}_C$	we define $\mathbf{A} \in \mathcal{R}_q^{k \times k}$ and
$\mathbf{c}_{L,b} \leftarrow \mathbf{s}_C^\top \mathbf{t}_L + f_L + \lceil q/2 \rceil m_{L,b}$	$\mathbf{t}_L \in \mathcal{R}_q^k$ such that $\mathbf{B}^\top = (\mathbf{A} \mid \mathbf{t}_L)$
$\mathbf{c}_R \leftarrow \mathbf{s}_C^\top \mathbf{t}_R + f_R + \lceil q/2 \rceil m_R$	$\mathbf{t}_R \leftarrow \mathbf{A}\mathbf{s}_R + \mathbf{e}_R; \text{pk}_R = \mathbf{t}_R, \text{sk}_R = \mathbf{s}_R$
$b \leftarrow \{0, 1\}$	$(\text{state}, m_{L,0}, m_{L,1}, m_R) \leftarrow$ $\mathcal{A}_1(\mathbf{A}, \text{pk}_R, (\text{sk}_R, \mathbf{e}_R), \text{pk}_L)$
$b' \leftarrow \mathcal{A}_2(\text{state}, (\mathbf{c}_C, \mathbf{c}_{L,b}, \mathbf{c}_R))$	$\mathbf{c}_{L,b} \leftarrow \tilde{\mathbf{c}}_L + \lceil q/2 \rceil m_{L,b}$
return $b = b'$	$\mathbf{c}_R \leftarrow \tilde{\mathbf{c}}_C^\top \mathbf{s}_R + \mathbf{z} + \lceil q/2 \rceil m_R$
$G_1(\mathcal{A})$	$b \leftarrow \{0, 1\}$
Two differences with G_0 :	$b' \leftarrow \mathcal{A}_2(\text{state}, (\mathbf{c}_C, \mathbf{c}_{L,b}, \mathbf{c}_R))$
- $\text{sk}_L, \mathbf{e}_L$ are no more defined	return $b = b'$
- $\text{pk}_L = \mathbf{t}_L \leftarrow \mathcal{R}_q^k$	$G_3(\mathcal{A})$
	One difference with G_2 :
	$(\tilde{\mathbf{c}}_C^\top, \tilde{\mathbf{c}}_L)^\top \leftarrow \mathcal{R}_q^{(k+1)}$

Fig. 18: Complete games for the proof of Theorem 5.

We proceed with a game-based proof, with the full sequence of games provided in Figure 18.

Game G_0 . The initial $[\text{IND-CPA}, -]$ game.

Game G_1 . In G_1 , the vector $\mathbf{t}_L$ is sampled uniformly at random, and the vectors $\mathbf{e}_L$ and $\mathbf{s}_L$ are no longer used. We can therefore construct an adversary $\mathcal{B}$ such that: $|\text{Adv}^{G_1}(\mathcal{A}) - \text{Adv}^{[\text{IND-CPA}, -]}(\mathcal{A})|$ is bounded by $\text{Adv}^{\text{MLWE}_{\mathcal{R}, k, k, q}^{2k}}(\mathcal{B})$.

Game G_2 . In G_2 , we equivalently recompute several elements from G_1 as follows. Let $\mathbf{A} \in \mathcal{R}^{k \times k}$ and $\mathbf{t}_L \in \mathcal{R}^k$ be such that $\mathbf{B}^\top = (\mathbf{A} \mid \mathbf{t}_L)$ for a uniformly sampled matrix $\mathbf{B} \in \mathcal{R}^{(k+1) \times k}$.

This implies that $\mathbf{c}_C$ and $\tilde{\mathbf{c}}_L = \mathbf{s}_C^\top \mathbf{t}_L + f_L$ can be expressed as $(\mathbf{c}_C^\top, \tilde{\mathbf{c}}_L)^\top = \mathbf{B}\mathbf{s}_C + (\mathbf{e}_C^\top, f_L)^\top$.

Moreover, the term $\mathbf{s}_C^\top \mathbf{t}_R + f_R$ is replaced with the term $\mathbf{c}_C^\top \mathbf{s}_R + z$, where $z = (\mathbf{s}_C^\top, \mathbf{e}_C^\top) \begin{pmatrix} \mathbf{e}_R \\ -\mathbf{s}_R \end{pmatrix} + f_R$. These two expressions are indeed equivalent, since:

$$\begin{aligned} \mathbf{s}_C^\top \mathbf{t}_R + f_R &= \mathbf{s}_C^\top (\mathbf{A} \mathbf{s}_R + \mathbf{e}_R) + f_R = (\mathbf{s}_C^\top \mathbf{A} + \mathbf{e}_C^\top) \mathbf{s}_R - \mathbf{e}_C^\top \mathbf{s}_R + \mathbf{s}_C^\top \mathbf{e}_R + f_R \\ &= \mathbf{c}_C^\top \mathbf{s}_R + (\mathbf{s}_C^\top, \mathbf{e}_C^\top) \begin{pmatrix} \mathbf{e}_R \\ -\mathbf{s}_R \end{pmatrix} + f_R = \mathbf{c}_C^\top \mathbf{s}_R + z. \end{aligned}$$

Thus, the advantage does not change.

Game G_3 . This game is identical to G_2 , except that $(\mathbf{c}_C^\top, \tilde{\mathbf{c}}_L)^\top$ is now sampled uniformly at random. Consequently, $c_{L,b} = \tilde{\mathbf{c}}_L + \text{Encode}(m_{L,b})$ is indistinguishable from a uniformly random element for the adversary $\mathcal{A}$. Hence, $\text{Adv}^{G_3}(\mathcal{A}) = 0$.

The only difference between G_2 and G_3 lies in how $(\mathbf{c}_C^\top, \tilde{\mathbf{c}}_L)^\top$ is generated: in G_2 , it is computed as $\mathbf{B} \mathbf{s}_C + (\mathbf{e}_C^\top, f_L)^\top$, whereas in G_3 it is drawn uniformly from $\mathcal{R}_q^{(k+1)}$. This change directly corresponds to the distributional shift underlying the Hint-MLWE problem with the parameters specified in the theorem statement, and we may therefore conclude.

H.3 Further Parameter Selection and Heuristic Discussion for Maul

Various trade-offs arise when selecting parameters, primarily governed by one consideration: decryption failure rate. To ensure proper security, we emphasize a conservative parameter-selection approach. Specifically, parameters are chosen such that the failure rate remains strictly below the defined security threshold. Nevertheless, accurately quantifying the security degradation associated with these attacks remains a tough question. Recent studies, notably [20], have begun to address this challenge.

In this section, we present several parameter sets to accommodate different security requirements and performance trade-offs. Beyond simply tuning the decryption failure rate, accounting for or disregarding the previously outlined heuristics can yield substantially different parameter choices. Accordingly, we propose three parameter sets: one that reuses the ML-KEM parameters to preserve key and implementation compatibility—though with a higher decryption failure rate at NIST levels 1 and 5, while remaining sufficiently low at level 3—and two more conservative sets that do not rely on Heuristic 1.

Maul-Kyber. The main drawback of the conservative approach lies in its incompatibility with ML-KEM/Kyber implementations. To complement the core parameter sets, Table 7 also includes a set aligned with ML-KEM’s parameters. However, this set exhibits a higher decryption failure rate for at NIST levels 1 and 5, potentially resulting in effective security below the target levels, while remaining sufficiently low for Maul768-K. The latter therefore represents a practical choice when implementation compatibility with ML-KEM-768 is desired. As for Maul512-K and Maul1024-K, following the analysis of [20], we emphasize that such parameter choices may be risky, particularly in our setting where one of the two keys is static, since even moderate failure probabilities could be exploitable.

Table 7: Parameter sets for **Maul-K** (Kyber-like), including sizes (in bytes).

	bikz	n	k	q	$\mathcal{B}$	$\mathcal{D}$	(d_u, d_v)	δ	ν	$ \mathbf{pk} $	$ \mathbf{ct} $	$ \mathbf{pk} + \mathbf{ct} $
Maul512-K	371	256	2	3329	$\mathcal{B}_3$	$\mathcal{D}_{48}$	$(-, 7)$	2^{-101}	384	749	1196	1945
Maul768-K	591	256	3	3329	$\mathcal{B}_2$	$\mathcal{D}_{32}$	$(-, 7)$	2^{-184}	512	1124	1572	2696
Maul1024-K	829	256	4	3329	$\mathcal{B}_2$	$\mathcal{D}_{32}$	$(-, 7)$	2^{-153}	640	1498	1946	3444

Heuristic Discussions on Discrete Gaussians Convolution. Continuous Gaussian distributions satisfy a crucial property: preservation under convolution. Namely, $\mathcal{D}_\sigma + \mathcal{D}_\sigma = \mathcal{D}_{\sigma\sqrt{2}}$ for a given standard deviation σ . However, this property does not hold for discrete Gaussians, although accurate approximations can be obtained, particularly when considering parameters above the smoothing parameter, as detailed in [46, Lemma 2.4]. In our case, the standard deviations reported in Tables 5 and 7 lie below the smoothing parameter.

To better quantify the deviation, we developed a script to compute the Rényi divergence between $\mathcal{D}_{\mathbb{Z},\sigma} + \mathcal{D}_{\mathbb{Z},\sigma}$ and $\mathcal{D}_{\mathbb{Z},\sigma\sqrt{2}}$, allowing us to assess the security loss and propose more conservative parameters accordingly.

Definition 4 (Rényi divergence [47]). Let $\mathcal{P}$ and $\mathcal{Q}$ be two distributions such that $\text{Supp}(\mathcal{P}) \subset \text{Supp}(\mathcal{Q})$. For $a \in (1, \infty)$, we define the Rényi divergence of order a by:

$$R_a(\mathcal{P}||\mathcal{Q}) = \left(\sum_{x \in \text{Supp}(\mathcal{P})} \frac{\mathcal{P}(x)^a}{\mathcal{Q}(x)^{a-1}} \right)^{\frac{1}{a-1}}.$$

In addition, we define the Rényi divergence of order ∞ by :

$$R_\infty(\mathcal{P}||\mathcal{Q}) = \max_{x \in \text{Supp}(\mathcal{P})} \frac{\mathcal{P}(x)}{\mathcal{Q}(x)}.$$

Using the Rényi divergence, we quantify the security loss introduced by the use of **Hint-MLWE** with the distribution $D_{\mathbb{Z},\sigma\sqrt{2}}$. Specifically, given oracle access to the distributions $D_{\mathbb{Z},\sigma} + D_{\mathbb{Z},\sigma}$ and $D_{\mathbb{Z},\sigma\sqrt{2}}$, an adversary $\mathcal{A}$ making q_s queries can distinguish these distributions with an advantage at most $2^{-\lambda}$, provided the condition $R_a(D_{\mathbb{Z},\sigma} + D_{\mathbb{Z},\sigma}||D_{\mathbb{Z},\sigma\sqrt{2}}) \leq 1 + \frac{1}{4q_s}$ holds true for $a = 2(\lambda + 1)$ [47, Equation 4].

In our scheme, querying these distributions equates to executing **Maul**; hence, we define $q_s = 2^{64}$ as the maximum number of queries to **Maul**. Additionally, we replace centered binomial distributions with discrete Gaussian distributions according to Heuristic 1. To establish an accurate equivalence, we set $D_{\mathbb{Z},\sigma} \sim \mathcal{B}_\eta$ with variance $\sigma^2 = \frac{\eta}{2}$.

The resulting Rényi divergence profile, based on these distributions, is illustrated in Figure 19. This profile allows to determine an optimal σ for targeted

Table 8: Parameter sets for **Maul-C** (conservative standard deviation), including sizes (in bytes).

	bikz	n	k	q	$\mathcal{B}$	$\mathcal{D}$	(d_u, d_v)	δ	ν	$ \text{pk} $	$ \text{ct} $	$ \text{pk} + \text{ct} $
Maul512-C	347	256	2	7681	$\mathcal{B}_6$	$\mathcal{D}_{96}$	(12, 7)	2^{-129}	384	826	1216	2042
Maul768-C	548	256	3	10753	$\mathcal{B}_6$	$\mathcal{D}_{96}$	(12, 6)	2^{-193}	512	1286	1536	2822
Maul1024-C	755	256	4	13313	$\mathcal{B}_6$	$\mathcal{D}_{96}$	(13, 7)	2^{-258}	640	1754	2112	3866

security levels (128, 192, or 256 bits). Given that queries directly invoke **Maul** instead of a generic oracle, we maintain $q_s = 2^{64}$ across all security targets. Utilizing Figure 19, the required Rényi divergence bound of $\frac{1}{4q_s} = 2^{-66}$ is satisfied for $\sigma = 1.7$, resulting in the corresponding binomial parameter $\eta = 6$.

Table 9: Parameter sets for **Maul-R** (Rényi-based), including sizes (in bytes).

	bikz	n	k	q	$\mathcal{B}$	$\mathcal{D}$	(d_u, d_v)	δ	ν	$ \text{pk} $	$ \text{ct} $	$ \text{pk} + \text{ct} $
Maul512-R	330	256	2	15361	$\mathcal{B}_{12}$	$\mathcal{D}_{192}$	(12, 6)	2^{-128}	384	890	1280	2170
Maul768-R	523	256	3	22273	$\mathcal{B}_{12}$	$\mathcal{D}_{192}$	(12, 6)	2^{-194}	512	1387	1536	2923
Maul1024-R	728	256	4	26113	$\mathcal{B}_{12}$	$\mathcal{D}_{192}$	(14, 8)	2^{-256}	640	1879	2304	4183

In Table 8, we highlight a set of parameters following previous Rényi analysis ensuring low decryption failure rate. Meanwhile in Table 9, we present a fully conservative set of parameters taking into account an additional factor of $\sqrt{2}$ from Hint-MLWE reduction to MLWE leading to $\eta = 12$. Note that these different conservative choices degrade the estimated security measured through bikz size.

I Additional Uses Cases for DUAKE and Maul

As previously noted, the DUAKE₂ protocol offers unilateral implicit authentication, which can be straightforwardly upgraded to explicit authentication by appending a key confirmation round, as in DUAKE₃. This type of UAKE protocol is particularly well-suited to two prominent use cases: KEM-based TLS handshakes (see Figure 21) and authenticated continuous key agreement.

Authenticated Continuous Key Agreement. A Continuous Key Agreement (CKA) is a secure messaging subprotocol, initially introduced in [2], that

⁴⁴ <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/call-for-proposals>

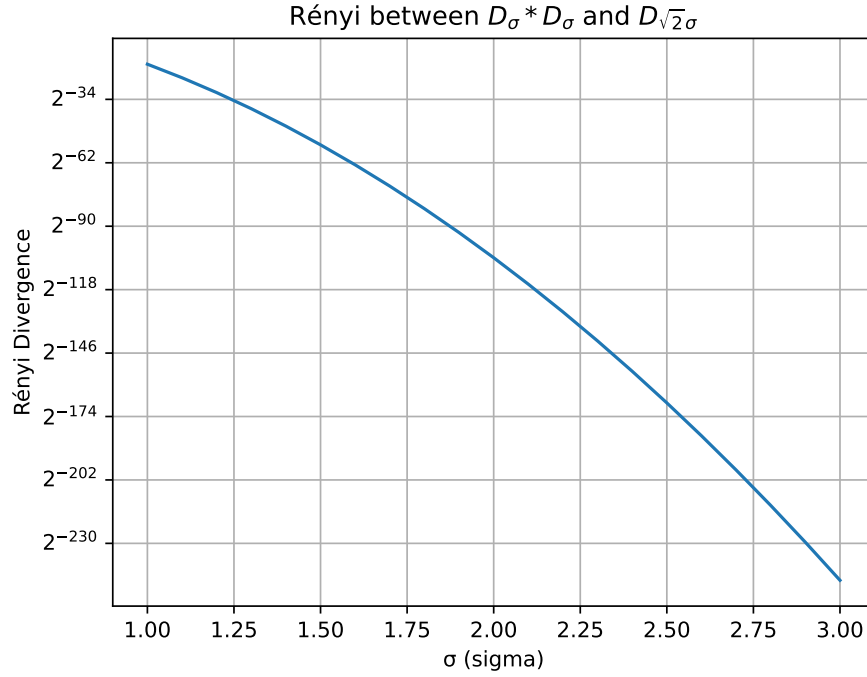


Fig. 19: Rényi divergence between $D_{\mathbb{Z},\sigma} + D_{\mathbb{Z},\sigma}$ and $D_{\mathbb{Z},\sigma\sqrt{2}}$ at increasing standard deviation σ (subtracted by 1).

enables secure messaging protocols to achieve both forward secrecy and post-compromise security.⁴⁵ Standard secure messaging protocols typically begin with a (non-interactive) authenticated key exchange (AKE) to establish an initial shared secret known as the root key. This root key is then regularly updated using the CKA subroutine to provide continued security.

However, as emphasized by [23], an adversary who compromises one party may not remain passive but can actively launch a man-in-the-middle (MitM) attack by conducting its own key exchanges with both parties. Given that secure messaging sessions may last for years, such attacks can have enduring consequences. A natural countermeasure is to periodically execute an *authenticated* key exchange instead of relying solely on unauthenticated updates, leading to what [23] defines as an *Authenticated Continuous Key Agreement (ACKA)*.⁴⁶

In parallel, post-quantum (PQ) hybridization has gained traction in secure messaging protocols, with several efforts incorporating PQ ratchets alongside

⁴⁵ Forward secrecy ensures that past communications remain secure even if long-term secrets are compromised. Post-compromise security refers to the ability of the protocol to recover and regain security after such a compromise.

⁴⁶ Note that the ACKA approach in [23] differs from ours, as it provides continuous authentication using a distinct mechanism rather than periodic AKEs.

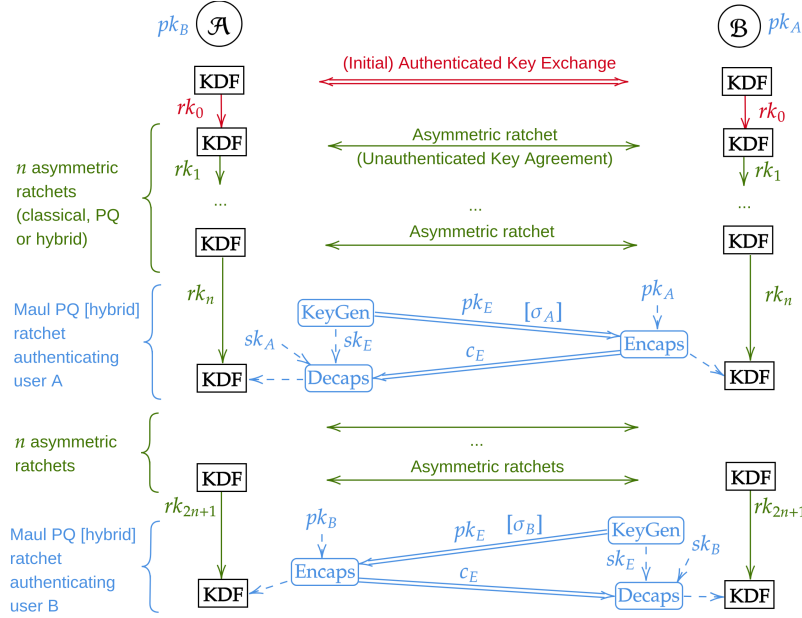


Fig. 20: Authenticated Continuous Key Agreement (ACKA) using our DUAKE (in blue) for periodic unilateral implicit authentication. In green are unauthenticated ratchets performed between authenticated ones, whose number is protocol-dependent. The variable rk_i denotes the i -th generation of the shared secret.

classical Diffie-Hellman key exchanges. For instance, Apple’s iMessage implements the PQ3 protocol [4], which hybridizes classical and PQ cryptography not only during the initial AKE but also within the CKA phase, introducing one PQ ratchet approximately every 50 classical ratchets [35]. More recently, [22] introduced the Triple-Ratchet protocol, an efficient hybrid CKA based on a novel *ratcheted-KEM*⁴⁷ called Katana, designed for integration into future versions of the Signal protocol.

A post-quantum ACKA requires periodic KEM-based implicit authentications⁴⁸ alongside ephemeral PQ ratchets. Each such authentication would traditionally incur the transmission of an additional ciphertext. However, our UAE construction DUAKE₂, instantiated with the 2K-KEM scheme Maul, is particularly well-suited in this setting, as it reduces the communication cost of unilateral authentication from two ciphertexts to one, significantly improving communication efficiency, as shown in Table 3.

⁴⁷ A ratcheted-KEM is a KEM introduced in [22] for PQ CKAs, where the ciphertext of one encapsulation can serve as a public key in subsequent encapsulations.

⁴⁸ We exclude PQ signature-based authentication due to its high communication overhead and computational cost compared to KEM-based methods.

Moreover, full mutual authentication need not occur simultaneously. Instead, it can be distributed over successive ratchets, further reducing communication overhead. Specifically, the encapsulating party uses **Maul** with the recipient’s ephemeral public key (previously received) and static authentication public key to produce a single ciphertext. The recipient decrypts this ciphertext, gaining a shared key and implicitly confirming the sender’s identity. This authentication is made explicit upon the sender’s first encrypted message using the derived key. Later, the recipient can reciprocate using the same mechanism, completing the mutual authentication in a staggered fashion.

Figure 20 illustrates this interplay between authenticated and unauthenticated PQ (or hybrid) ratchets within an ACKA protocol. We underline that the PQ authentication ratchet implemented with **Maul** can easily be hybridized, if needed, with a classical authentication mechanism such as a signature scheme.

Maul-Based TLS Handshake. Unilaterally authenticated key exchanges are widely employed in TLS handshakes, where typically only the server is authenticated. In light of the well-known drawbacks of post-quantum signature schemes, several works [51, 52] have proposed a KEM-based variant of TLS, known as KEMTLS, which is currently under study by the IETF [58]. This mechanism combines two KEMs in parallel and would thus benefit significantly from the use of our **Maul** double-key KEM, which achieves the same functionality with reduced communication overhead.

However, adapting KEMTLS to benefit from **Maul** would require protocol changes: specifically, enabling the client to perform the double encapsulation, in contrast to the current version where the server performs one encapsulation followed by a client encapsulation.

Figure 21, shows the message flow of a TLS handshake using our **Maul 2K-KEM**.

The reversed order of exchanges introduced by our scheme, compared with KEMTLS, has notable implications. Indeed, in the regular TLS 1.3 handshake [48], as well as in KEMTLS, the **ServerHello** message is immediately followed by the encrypted transmission of the **EncryptedExtensions** and the server’s certificate. This early encryption—though only authenticated later—enhances the privacy guarantees of the handshake.

In contrast, with **Maul**’s double encapsulation, the key exchange and server authentication occur simultaneously, which precludes this interim unauthenticated encryption. As a consequence:

- The server’s certificate must be transmitted in the clear;
- The **EncryptedExtensions** message must either be sent in the clear as well, or deferred until the subsequent **ServerFinished** message, once the server has decrypted the client’s ciphertext.

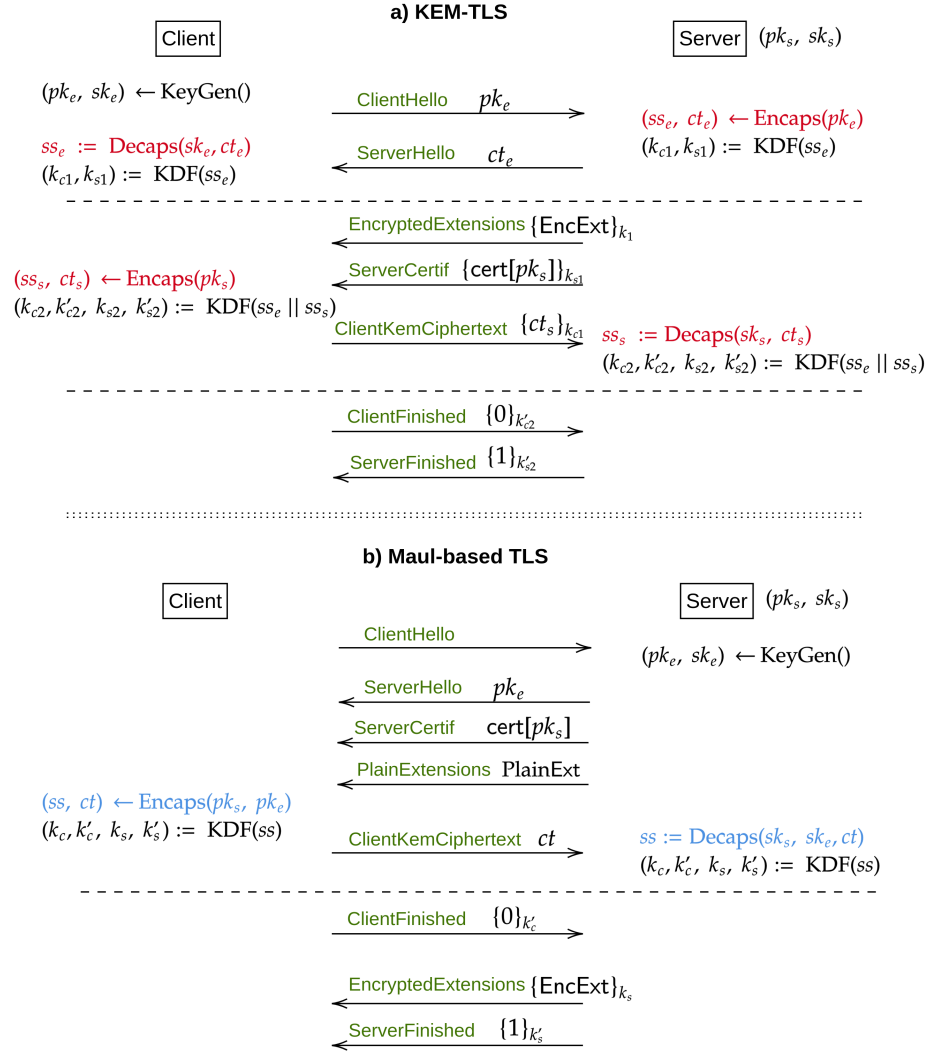


Fig.21: Compared TLS handshakes (server unilateral-authentication) with KEMTLS (top) and with our Maul double-key KEM (bottom).

Table of Contents

DAKE: Bandwidth-Efficient (U)AKE from Double-KEM	1
<i>Hugo Beguinet, Céline Chevalier, Guirec Lebrun, Thomas Legavre,</i>	
<i>Thomas Ricosset, Maxime Roméas, and Éric Sageloli</i>	
1 Introduction	1
1.1 Our Contributions	3
1.2 Technical Overview	4
1.3 Related Works	10
2 Preliminaries	11
3 Authenticated Key Exchanges from Double-KEMs	14
3.1 The eCK Security Model and Its Variants	14
3.2 Our AKE Protocols	17
3.3 Security of Our AKE Protocols	17
4 Chosen-key Fujisaki–Okamoto Transform	21
4.1 Combining Two PKE to Obtain a 2K2M-PKE	21
4.2 The CK-FO Transform and Its Security	22
5 Compact Lattice-based Double-KEM	24
5.1 Preliminaries on Lattices, MLWE, and Hint-MLWE Problems	24
5.2 $\mathsf{Maul}_{\mathsf{PKE}}$: A $[\mathsf{IND}\text{-}\mathsf{CPA}, \mathsf{IND}\text{-}\mathsf{CPA}]$ -Secure 2K2M-PKE	25
5.3 Parameter Selection	27
5.4 Security Estimation	28
5.5 Concrete Security Loss	29
A Additional Preliminaries	36
B Proof of the CK-FO Transform	41
B.1 Proof in the Random Oracle Model	42
B.2 Proof in the Quantum-Accessible Random Oracle Model	47
C The eCK Model and its Variants	55
C.1 Modelization of Protocol, Participants and Adversarial Queries	55
C.2 Correctness, Key Secrecy and Freshness predicates	59
C.3 Definition of eCK^w and eCK-PFS Models	59
C.4 Unilateral Versions of eCK^w ($\mathsf{u}\text{-}\mathsf{eCK}^w$) and eCK-PFS ($\mathsf{u}\text{-}\mathsf{eCK}\text{-}\mathsf{PFS}$)	61
D Detailed Presentation of DAKE^σ and DUAKE	63
D.1 DAKE^σ : A Variant of DAKE Using a Signature	63
D.2 DUAKE : An Unilateral AKE Fitted for Maul	64
E Key Secrecy of DAKE, DAKE^σ , and DUAKE	65
E.1 Context, Rationale of the Proof and Notations	67
E.2 Notations Used for the Proofs	69
E.3 Key Secrecy for Predicates $\mathsf{U} \cap \mathsf{eCK}^w$ and $\mathsf{U} \cap \mathsf{eCK}\text{-}\mathsf{PFS}$	69
E.4 Key Secrecy for Predicates $\mathsf{V}_1, \dots, \mathsf{V}_5$	75
E.5 Key Secrecy of DUAKE	87
F Explicit Authentication of DAKE and its Variants	87

G	Instantiating the Xue et al. AKE with One-Sided Chosen-Key Secure Double-KEMs	89
H	Additional Details Regarding Hint-MLWE and Maul_{PKE}	90
	H.1 Hint-MLWE Hardness	90
	H.2 Complete Correctness and Security Proofs for Maul_{PKE}	93
	H.3 Further Parameter Selection and Heuristic Discussion for Maul . .	95
I	Additional Uses Cases for DUAKE and Maul	97